

In [1]: *# 실행 환경 확인*

```
import pandas as pd
import numpy as np
import sklearn
import scipy
import statsmodels
import mlxtend
import sys
import xgboost as xgb

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

print(sys.version)
for i in [pd, np, sklearn, scipy, mlxtend, statsmodels, xgb]:
    print(i.__name__, i.__version__)

3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)]
pandas 0.25.1
numpy 1.18.5
sklearn 0.21.3
scipy 1.5.2
mlxtend 0.15.0.0
statsmodels 0.11.1
xgboost 0.80
```

In [2]: *# 시각화 모듈 설정*
참고용 차트를 출력하기 위함

```
import matplotlib.pyplot as plt
import seaborn as sns

import matplotlib as mpl
import matplotlib.font_manager as fm
plt.rc('font', family='Malgun Gothic')
mpl.rcParams['axes.unicode_minus'] = False
```

```
In [3]: Image('2-1.png')
```

Out[3]: **문제 개요** ¶

다음은 폴더블 폰의 힌지에 들어가는 스프링 내구력을 테스트한 실험 결과이다.

스프링 측정값과 스프링에 가한 부하 정보와 함께, 테스트 통과/실패 (failure) 결과가 기재되어 있다.

개발부서는 테스트 비용을 줄이기 위해 failure 여부를 맞추는 모델을 만들고자 한다.

변수명은 보안을 위해 measurement_0과 같이 익명화되었다.

데이터 구성

학습데이터: train_prob.csv, 21,458 rows, 25 columns

테스트데이터: test_prob.csv, 5,112 rows, 24 columns,

테스트정답셋: test_prob_ans.csv, 5,112 rows, 1 columns

컬럼명 설명 타입

변수명	설명	타입
id	실험 고유 번호	정수형
product_code	스프링 코드	범주형
loading	스프링에 가한 부하	실수형
attribute_0	구성 소재1	범주형
attribute_1	구성 소재2	범주형
attribute_2	구성 소재3	정수형
attribute_3	구성 소재4	정수형
measurement_0 ~ 17	측정값 0~17	실수형
failure	성공여부	이진형(0, 1)

전처리(Preprocessing)

train_prob.csv를 불러 온다. 이를 basetable이라고 한다.

```
In [4]: # 고유값인 id를 인덱스에 위치 시킵니다.
df_basetable = pd.read_csv('train_prob.csv', index_col='id')
display(df_basetable.info())
df_basetable.head()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 21458 entries, 0 to 26569
Data columns (total 25 columns):
product_code      21458 non-null object
loading           21257 non-null float64
attribute_0       21458 non-null object
attribute_1       21458 non-null object
attribute_2       21458 non-null int64
attribute_3       21458 non-null int64
measurement_0     21458 non-null int64
measurement_1     21458 non-null int64
measurement_2     21458 non-null int64
measurement_3     21146 non-null float64
measurement_4     21016 non-null float64
measurement_5     20893 non-null float64
measurement_6     20818 non-null float64
measurement_7     20692 non-null float64
measurement_8     20605 non-null float64
measurement_9     20469 non-null float64
measurement_10    20399 non-null float64
measurement_11    20278 non-null float64
measurement_12    20171 non-null float64
measurement_13    20063 non-null float64
measurement_14    19976 non-null float64
measurement_15    19855 non-null float64
measurement_16    19750 non-null float64
measurement_17    19640 non-null float64
failure           21458 non-null int64
dtypes: float64(16), int64(6), object(3)
memory usage: 4.3+ MB
```

None

```
Out[4]:
```

	product_code	loading	attribute_0	attribute_1	attribute_2	attribute_3	measurement_0	measurement_1	measurement_2	measurement_3	...	measu
id												
0	A	80.10	material_7	material_8	9	5	7	8	4	18.040	...	
1	A	84.89	material_7	material_8	9	5	14	3	3	18.213	...	
2	A	82.43	material_7	material_8	9	5	12	1	5	18.057	...	
3	A	101.07	material_7	material_8	9	5	13	2	6	17.295	...	
4	A	188.06	material_7	material_8	9	5	9	2	8	19.346	...	

5 rows x 25 columns

단계 1

basetable에 measurement_3 ~17 각각의 행이 결측인지 나타내는 파생 변수를 만든다.

파생 변수는 이진 형식이고, False는 미결측 True는 결측을 의미한다.

파생 변수의 이름은 measurement 번호에 따라 isna_3 ~ 17로 한다.

```
In [5]: df_basetable[['isna_{}'.format(i) for i in range(3, 18)]] = \
         df_basetable[['measurement_{}'.format(i) for i in range(3, 18)]].isna()
df_basetable.head()
```

```
Out[5]:
```

	product_code	loading	attribute_0	attribute_1	attribute_2	attribute_3	measurement_0	measurement_1	measurement_2	measurement_3	...	isna_8
id												
0	A	80.10	material_7	material_8	9	5	7	8	4	18.040	...	False
1	A	84.89	material_7	material_8	9	5	14	3	3	18.213	...	False
2	A	82.43	material_7	material_8	9	5	12	1	5	18.057	...	False
3	A	101.07	material_7	material_8	9	5	13	2	6	17.295	...	False
4	A	188.06	material_7	material_8	9	5	9	2	8	19.346	...	False

5 rows × 40 columns

```
In [6]: df_basetable.columns
```

```
Out[6]: Index(['product_code', 'loading', 'attribute_0', 'attribute_1', 'attribute_2',
              'attribute_3', 'measurement_0', 'measurement_1', 'measurement_2',
              'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6',
              'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10',
              'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14',
              'measurement_15', 'measurement_16', 'measurement_17', 'failure',
              'isna_3', 'isna_4', 'isna_5', 'isna_6', 'isna_7', 'isna_8', 'isna_9',
              'isna_10', 'isna_11', 'isna_12', 'isna_13', 'isna_14', 'isna_15',
              'isna_16', 'isna_17'],
              dtype='object')
```

```
In [7]: df_basetable[['isna_{}'.format(i) for i in range(3, 18)]]
```

Out[7]:

	isna_3	isna_4	isna_5	isna_6	isna_7	isna_8	isna_9	isna_10	isna_11	isna_12	isna_13	isna_14	isna_15	isna_16	isna_17
--	--------	--------	--------	--------	--------	--------	--------	---------	---------	---------	---------	---------	---------	---------	---------

id

0	False	False	False	False	False	False	False	False	False	False	False	True	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	True	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
26565	False	False	False	False	False	False	True	False	False	False	False	False	False	True	False
26566	False	True	False	False	False	False	False	False	False	False	False	False	False	False	False
26567	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
26568	False	True	False	False	False	False	False	False	False	False	False	False	False	False	False
26569	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False

21458 rows × 15 columns

In [8]: `type(df_basetable[['isna_{}'.format(i) for i in range(3, 18)]])`

Out[8]: `pandas.core.frame.DataFrame`

In [9]:

```
'''
아래 코드는 데이터프레임(df_basetable)의 특정 열들을 사용하여 결측치 여부를 나타내는 새로운 열들을 만드는 과정을 나타냅니다.
1. 새로운 열 생성:
- `df_basetable[['isna_{}'.format(i) for i in range(3, 18)]]`은
  `df_basetable` 데이터프레임에서 'measurement_3'부터 'measurement_17'까지의 열에 대해 결측치 여부를 나타내는 DataFrame이다.
- `[['measurement_{}'.format(i) for i in range(3, 18)]].isna()`은 선택된 열들에 대해 결측치 여부를 검사합니다.
  `isna()` 함수는 각 셀의 값이 결측치인지를 확인하고, 결과를 불리언(Boolean) 형태로 반환합니다.
2. 결과 확인:
- `.sum()`은 각 열에 대해 결측치(True 값)의 개수를 합산합니다.
- `.to_frame()`은 결과를 데이터프레임 형태로 변환합니다.
- `.T`는 결과를 전치하여(Transpose) 열과 행을 바꿉니다. 이렇게 함으로써 결과 데이터프레임이 가로 방향으로 출력되도록 합니다.
'''
```

따라서, 이 코드는 원본 데이터프레임의 특정 열들을 이용하여 결측치 여부를 나타내는 새로운 열들을 생성하고, 각 열에 대한 결측치 개수를 요약하여 출력합니다.

```
...
```

```
df_basetable[['isna_{}'.format(i) for i in range(3, 18)]].sum().to_frame().T
```

```
Out[9]:
```

	isna_3	isna_4	isna_5	isna_6	isna_7	isna_8	isna_9	isna_10	isna_11	isna_12	isna_13	isna_14	isna_15	isna_16	isna_17
0	312	442	565	640	766	853	989	1059	1180	1287	1395	1482	1603	1708	1818

```
In [10]: Image('2-3.png')
```

Out[10]: **단계 2**

이 과제를 맡은 데이터분석가 지희는 measurement_3~17의 결측치 처리 방안을 고민하던 중,

개발부서에서 measurement_17은 product_code별로 failure를 예측하기 위해

measurement_3 ~ measurement_9을 다음과 같이 선형 조합하여 생성한 값이라는 정보를 받았다.

$$measurement_{17} = \beta_0 + \beta_3 measurement_3 + \beta_4 measurement_4 + \dots + \beta_9 measurement_9$$

이는 즉,

$$measurement_3 = \beta'_0 + \beta'_4 measurement_4 + \beta'_5 measurement_5 + \dots + \beta'_9 measurement_9 + \beta'_{17} measurement_{17}$$

...

$$measurement_9 = \beta''_0 + \beta''_3 measurement_3 + \beta''_4 measurement_4 + \dots + \beta''_8 measurement_8 + \beta''_{17} measurement_{17}$$

와 같이 measurement_3 ~ measurement_9의 각 변수들도 나머지 변수들과 선형 관계를 지닌다.

이 점을 이용하여 대상 변수를 번갈아 가면서 예측 모델을 만들어 최대한 원래 값에 가깝게 복원할 수 있다.

이러한 반복적인 결측치 복원 방법을 사내 데이터분석 연구소에 문의 했더니 다음과 같은 가이드를 주었다.

sklearn 모듈에 아직은 실험 단계이지만, 비슷한 경우에 문제 없이 사용했던 사례가 있어 의견을 드립니다.

from sklearn.experimental import enable_iterative_imputer 구문을 사용하여 실험 단계인 모듈을 활성화하고,

sklearn.impute.IterativeImputer를 사용한다면 원하는 결과를 얻을 수 있습니다.

가이드의 내용을 참조하여 basetable의 measurement_3~9와 measurement_17 결측치를 복원하라.

입력 변수] measurement_3 ~ 9, measurement_17 (입력 변수 순서에 유의)

함수가이드


```
sklearn.experimental.enable_iterative_imputer
```

```
sklearn.impute.IterativeImputer, random_state=123
```

```
sklearn.linear_model.LinearRegression
```

문제 지시사항 외 Default 값 사용

```
In [11]: '''
`IterativeImputer`는 `scikit-learn` 라이브러리의 `impute` 모듈에 있는 클래스입니다.
결측값을 다중 대체 방식으로 채우는 데 사용되며, 각 특성을 다른 특성의 회귀 모델을 사용하여 반복적으로 예측합니다.

### 클래스의 인자
1. estimator:
    - 유형: 객체 (기본값은 `BayesianRidge()`)
    - 설명: 각 특성의 결측값을 예측하기 위해 사용할 추정기입니다. 기본값은 Bayesian Ridge Regression입니다.
2. max_iter:
    - 유형: 정수 (기본값은 10)
    - 설명: 모든 특성에 대해 결측값 대체를 반복할 최대 횟수입니다.
3. tol:
    - 유형: 실수 (기본값은 1e-3)
    - 설명: 수렴 기준으로 사용되는 허용 오차입니다.
4. n_nearest_features:
    - 유형: 정수 (기본값은 None)
    - 설명: 결측값 대체를 위해 사용할 가장 가까운 특성의 수입니다. None인 경우, 모든 특성을 사용합니다.
5. initial_strategy:
    - 유형: 문자열 (기본값은 'mean')
    - 설명: 초기 결측값 대체를 위한 전략입니다. 사용할 수 있는 값은 'mean', 'median', 'most_frequent', 'constant'입니다.
6. imputation_order:
    - 유형: 문자열 (기본값은 'ascending')
    - 설명: 각 반복에서 특성을 대체할 순서입니다. 사용할 수 있는 값은 'ascending', 'descending', 'roman', 'arabic', 'random'입니다.
7. skip_complete:
    - 유형: 부울 (기본값은 False)
    - 설명: 결측값이 없는 특성을 건너뛰는 여부를 나타냅니다.
8. min_value:
    - 유형: 실수 또는 배열 (기본값은 None)
    - 설명: 특성별 최소 허용 값입니다. None인 경우 제한이 없습니다.
9. max_value:
    - 유형: 실수 또는 배열 (기본값은 None)
    - 설명: 특성별 최대 허용 값입니다. None인 경우 제한이 없습니다.
10. verbose:
```

```

- 유형: 정수 (기본값은 0)
- 설명: 출력 메시지의 상세 수준입니다. 0은 메시지를 출력하지 않고, 1 이상은 상세한 메시지를 출력합니다.
11. random_state:
- 유형: 정수, `RandomState` 인스턴스 또는 None (기본값은 None)
- 설명: 난수 생성의 시드를 설정합니다.
12. add_indicator:
- 유형: 부울 (기본값은 False)
- 설명: `MissingIndicator`를 추가할지 여부를 나타냅니다. 결측값이 있는 특성의 정보를 포함합니다.

### 리턴값
`IterativeImputer`는 직접적으로 리턴값을 가지지 않지만, 다음의 메서드와 속성을 통해
결측값을 대체한 데이터와 관련된 정보를 얻을 수 있습니다:

### 주요 메서드
1. fit(X, y=None):
- 설명: 모델을 데이터에 적합시킵니다.
- 인자:
  - `X`: 결측값을 포함한 입력 데이터 배열.
  - `y`: 레이블 데이터 (필요하지 않음).
- 리턴값: `self` (적합된 인스턴스).
2. transform(X):
- 설명: 적합된 모델을 사용하여 결측값을 대체합니다.
- 인자:
  - `X`: 결측값을 포함한 입력 데이터 배열.
- 리턴값: 결측값이 대체된 배열.
3. fit_transform(X, y=None):
- 설명: 모델을 데이터에 적합시키고, 결측값을 대체합니다.
- 인자:
  - `X`: 결측값을 포함한 입력 데이터 배열.
  - `y`: 레이블 데이터 (필요하지 않음).
- 리턴값: 결측값이 대체된 배열.

### 주요 속성
1. initial_imputer_:
- 설명: 초기 결측값 대체 전략을 구현한 SimpleImputer 인스턴스.
2. imputation_sequence_:
- 설명: 각 반복에서 수행된 대체 작업의 순서와 방법을 나타내는 리스트.
3. n_iter_:
- 설명: 데이터가 수렴할 때까지의 반복 횟수.
4. random_state_:
- 설명: `random_state` 인자를 처리한 RandomState 인스턴스.

```

```

### 예제
아래 예제에서는 결측값이 포함된 데이터를 생성하고, `IterativeImputer`를 사용하여 결측값을 대체합니다.
`fit` 메서드로 모델을 적합시키고, `transform` 메서드로 결측값을 대체한 결과를 얻습니다.
...

from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
import numpy as np

# 결측값을 포함한 데이터 생성
X = np.array([[1, 2, np.nan],
              [3, 4, 3],
              [np.nan, 6, 5],
              [7, 8, 9]])

# IterativeImputer 생성 및 적합
imputer = IterativeImputer(max_iter=10, random_state=0)
imputer.fit(X)

# 결측값 대체
X_imputed = imputer.transform(X)

print("원본 데이터:\n", X)
print("대체된 데이터:\n", X_imputed)

```

원본 데이터:

```

[[ 1.  2. nan]
 [ 3.  4.  3.]
 [nan  6.  5.]
 [ 7.  8.  9.]]

```

대체된 데이터:

```

[[ 1.00000000e+00  2.00000000e+00 -2.64865933e-06]
 [ 3.00000000e+00  4.00000000e+00  3.00000000e+00]
 [ 4.53846180e+00  6.00000000e+00  5.00000000e+00]
 [ 7.00000000e+00  8.00000000e+00  9.00000000e+00]]

```

In [12]: # 위 설명에 대한 예제 코드

```

import numpy as np
import pandas as pd
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

```

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import BayesianRidge

# 데이터 생성
data = load_diabetes()
X, y = data.data, data.target
X = pd.DataFrame(X)
y = pd.DataFrame(y)

# 결측값 삽입
np.random.seed(0)
mask = np.random.rand(*X.shape) < 0.1
X[mask] = np.nan

# 모델 초기화
imputer = IterativeImputer(estimator=BayesianRidge(), max_iter=10, random_state=0)

# 결측값 대체 모델 학습 및 변환
X_imputed = imputer.fit_transform(X)

# 결과 확인
print(f'결측값 대체 전 데이터:\n{X.head()}')
print(f'결측값 대체 후 데이터:\n{pd.DataFrame(X_imputed).head()}')
```

결측값 대체 전 데이터:

	0	1	2	3	4	5	6 \
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401
1	-0.001882	-0.044642	-0.051474	-0.026328	NaN	NaN	NaN
2	0.085299	0.050680	0.044451	-0.005671	-0.045599	-0.034194	-0.032356
3	-0.089063	-0.044642	-0.011595	-0.036656	NaN	0.024991	-0.036038
4	0.005383	-0.044642	-0.036385	NaN	0.003935	0.015596	0.008142

	7	8	9
0	-0.002592	0.019908	-0.017646
1	-0.039493	-0.068330	-0.092204
2	-0.002592	0.002864	-0.025930
3	0.034309	0.022692	-0.009362
4	-0.002592	-0.031991	-0.046641

결측값 대체 후 데이터:

	0	1	2	3	4	5	6 \
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.030308	-0.017450	0.030749
2	0.085299	0.050680	0.044451	-0.005671	-0.045599	-0.034194	-0.032356
3	-0.089063	-0.044642	-0.011595	-0.036656	0.016386	0.024991	-0.036038
4	0.005383	-0.044642	-0.036385	-0.029870	0.003935	0.015596	0.008142

	7	8	9
0	-0.002592	0.019908	-0.017646
1	-0.039493	-0.068330	-0.092204
2	-0.002592	0.002864	-0.025930
3	0.034309	0.022692	-0.009362
4	-0.002592	-0.031991	-0.046641

In [13]:

```
'''
아래 코드는 반복문을 사용하여 IterativeImputer를 이용하여 결측치를 대체하는 과정을 나타냅니다.

1. IterativeImputer 설정:
- `IterativeImputer`는 반복적으로 결측치를 추정하기 위해 다른 특성을 사용하는 회귀 모델을 사용합니다.
  여기서는 선형 회귀 모델(LinearRegression)을 사용합니다.
- `estimator` 매개변수는 결측치를 예측하는 데 사용되는 추정기(회귀 모델)를 지정합니다. 이 경우 선형 회귀 모델이 사용됩니다.
- `random_state`는 재현 가능한 결과를 생성하기 위해 사용되는 난수 생성기의 시드입니다.

2. 결측치 대체:
- `X_imp`는 결측치를 대체할 열들의 이름을 나타내는 리스트입니다.
  여기서는 'measurement_3'부터 'measurement_10'까지와 'measurement_17' 열을 대상으로 합니다.
- `df_basetable['product_code'].unique()`를 통해 고유한 제품 코드를 가져옵니다. 이를 통해 각 제품 코드별로 데이터를 처리할 것은
'''
```

- 반복문을 통해 각 제품 코드별로 데이터를 처리합니다.
- ``s_bidx = df_basetable['product_code'] == i``를 사용하여 현재 제품 코드에 해당하는 행을 선택합니다.
- ``df_basetable.loc[s_bidx, X_imp]``을 통해 현재 제품 코드에 해당하는 행과 선택한 열들을 가져옵니다.
- ``imp.fit_transform()```을 사용하여 선택된 데이터에 대해 결측치를 대체합니다. 대체된 결과는 원래 데이터프레임에 할당됩니다.

이렇게 설정된 코드는 `IterativeImputer`를 사용하여 각 제품 코드별로 결측치를 대체하는 과정을 반복문을 통해 수행합니다.

```

from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import LinearRegression

# 방법 1: 반복문을 통한 방법입니다.
imp = IterativeImputer(
    estimator=LinearRegression(fit_intercept=True),
    random_state=123
)
X_imp = ['measurement_{}'.format(i) for i in range(3, 10)] + ['measurement_17']
print('X_imp = ', X_imp)
print('-----')

# product_code 별로 처리를 합니다.
for i in df_basetable['product_code'].unique():
    print('i = ', i)
    s_bidx = df_basetable['product_code'] == i
    print(s_bidx)
    print('-----')
    # 값을 대입할 때, ---대입에 대상이 되는 데이터프레임을
    # df_basetable.loc[s_bidx][X_imp] 이런식으로 filtering
    # -> 컬럼선택 2단계로 할 경우 reference 경고를 받게 됩니다.
    # df_basetable.loc[s_bidx, X_imp] filtering과 columns 선택을 하여
    # 중간에 reference DataFrame 없이 처리하도록 합니다.
    df_basetable.loc[s_bidx, X_imp] = imp.fit_transform(df_basetable.loc[s_bidx, X_imp])

```

```
X_imp = ['measurement_3', 'measurement_4', 'measurement_5', 'measurement_6', 'measurement_7', 'measurement_8', 'measurement_9',
'measurement_17']
-----
i = A
id
0      True
1      True
2      True
3      True
4      True
...
26565  False
26566  False
26567  False
26568  False
26569  False
Name: product_code, Length: 21458, dtype: bool
-----
i = B
id
0      False
1      False
2      False
3      False
4      False
...
26565  False
26566  False
26567  False
26568  False
26569  False
Name: product_code, Length: 21458, dtype: bool
-----
i = C
id
0      False
1      False
2      False
3      False
4      False
...
26565  False
```

```

26566    False
26567    False
26568    False
26569    False
Name: product_code, Length: 21458, dtype: bool
-----
i = E
id
0      False
1      False
2      False
3      False
4      False
...
26565     True
26566     True
26567     True
26568     True
26569     True
Name: product_code, Length: 21458, dtype: bool
-----

```

In [14]:

```

'''
아래 코드는 방법 1과 비슷하게 IterativeImputer를 사용하여 결측치를 대체하는데,
이번에는 product_code 별로 데이터를 처리하고, 그 결과를 바로 적용하는 방법을 사용합니다.

1. IterativeImputer 설정:
    - `IterativeImputer`는 반복적으로 결측치를 추정하기 위해 다른 특성을 사용하는 회귀 모델을 사용합니다.
      여기서는 선형 회귀 모델(LinearRegression)을 사용합니다.
    - `estimator` 매개변수는 결측치를 예측하는 데 사용되는 추정기(회귀 모델)를 지정합니다. 이 경우 선형 회귀 모델이 사용됩니다.

2. 결측치 대체:
    - `X_imp`는 결측치를 대체할 열들의 이름을 나타내는 리스트입니다.
      여기서는 'measurement_3'부터 'measurement_10'까지와 'measurement_17' 열을 대상으로 합니다.
    - `df_basetable.groupby('product_code')[X_imp].apply()`를 통해 product_code 별로 데이터를 그룹화하고, 각 그룹에 대해 함수를 적
    - `lambda x: pd.DataFrame(imp.fit_transform(x), index=x.index, columns=x.columns)`는 각 그룹에 대해 수행되는 함수입니다.
      여기서는 IterativeImputer를 사용하여 결측치를 대체한 후, 그 결과를 새로운 DataFrame으로 만듭니다.
    - `x.index`와 `x.columns`를 사용하여 새로운 DataFrame의 인덱스와 열 이름을 원래의 그룹과 동일하게 설정합니다.
    - 이렇게 만들어진 결측치가 처리된 DataFrame이 각 그룹에 대해 반환됩니다.
    - `df_basetable[X_imp]`은 이 반환된 결과를 원래 데이터프레임에 바로 적용합니다.

이렇게 설정된 코드는 product_code 별로 IterativeImputer를 사용하여 결측치를 대체하고, 그 결과를 바로 적용하는 과정을 수행합니다.
'''

```



```

...

from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import LinearRegression

# 방법 2: product_code 별로 IterativeImputer를 통한 결측처리를 하고 바로 적용합니다.
imp = IterativeImputer(
    estimator=LinearRegression(fit_intercept=True),
    random_state=123
)
X_imp = ['measurement_{}'.format(i) for i in range(3, 10)] + ['measurement_17']
# apply에서 transform에서 numpy array를 넘겨 주므로,
# DataFrame을 만들어 반환시켜주면,
# apply에서 DataFrame으로 재구성하여 넘겨줍니다.
# 모델을 적용하여 결측치 처리된 데이터프레임을 만듭니다.
df_basetable[X_imp] = df_basetable.groupby('product_code')[X_imp].apply(
    lambda x: pd.DataFrame(imp.fit_transform(x), index=x.index, columns=x.columns)
)

```

In [15]: # 방법 1과 방법 2의 비교

```

...

방법 1과 방법 2의 주요 차이점은 데이터프레임의 처리 방식과 코드의 구성입니다.
1. 데이터프레임 처리 방식:
- 방법 1: 반복문을 사용하여 각 제품 코드별로 데이터를 처리합니다.
  이를 위해 `for` 반복문을 사용하여 각 제품 코드에 해당하는 행을 선택하고, 그에 따라 결측치를 대체합니다.
  따라서 데이터프레임은 원래의 형태를 유지하며, 각 반복마다 IterativeImputer가 새로운 데이터프레임을 반환합니다.
- 방법 2: `groupby` 메서드를 사용하여 product_code를 기준으로 데이터를 그룹화하고, 각 그룹에 대해 결측치 처리를 진행합니다.
  이를 통해 데이터프레임을 한 번에 처리하고, 그룹별로 결측치가 처리된 새로운 데이터프레임을 반환합니다.

2. 코드 구성:
- 방법 1: 반복문을 사용하여 각 제품 코드별로 데이터를 처리하는 구조로 되어 있습니다.
  이에 따라 코드는 반복문 내에서 결측치 대체 과정을 담고 있습니다.
  이러한 구조는 반복문의 각 반복에서 IterativeImputer를 다시 초기화하고, 각 그룹에 대해 결측치가 처리됩니다.
- 방법 2: `groupby` 메서드를 사용하여 그룹화된 데이터프레임에 함수를 적용하는 형태로 되어 있습니다.
  이에 따라 코드는 함수 내에서 각 그룹에 대해 IterativeImputer를 초기화하고, 결측치가 처리된 새로운 데이터프레임을 반환합니다.

따라서, 두 방법은 결측치 처리를 수행하는 접근 방식에서 차이가 있으며,
방법 2는 벡터화된 연산을 통해 데이터프레임을 한 번에 처리하여 코드의 효율성을 높이는 반면,
방법 1은 각 제품 코드별로 결측치 처리를 수행하므로 데이터프레임이 반복적으로 변경되는 점이 다릅니다.

```

```
'''
print('')
```

In [16]:

```
'''
아래 코드는 3번째 방법으로, 각 제품 코드(product_code)별로 IterativeImputer 모델을 생성하고,
해당 모델을 사용하여 결측치를 처리하는 과정을 나타냅니다.

1. IterativeImputer 모델 생성 함수:
- `create_imp()` 함수는 IterativeImputer 모델을 생성하는 함수입니다.
  이 함수는 매번 호출될 때마다 새로운 IterativeImputer 모델을 생성하여 반환합니다.
  모델은 선형 회귀(Linear Regression)를 사용합니다.

2. 결측치 처리 모델 학습:
- `s_imp` 변수는 각 제품 코드별로 IterativeImputer 모델을 생성하여 저장하는 시리즈(Series)입니다.
  `groupby`를 사용하여 product_code를 기준으로 데이터를 그룹화하고, 각 그룹에 대해 `create_imp().fit(x)`를 적용하여
  모델을 생성하고 학습시킵니다. 이 때, 각 그룹은 해당 제품 코드(product_code)에 해당합니다.

3. 결측치 처리:
- `df_basetable.groupby('product_code')[X_imp].apply()`를 통해 product_code 별로 데이터를 그룹화하고, 각 그룹에 대해 함수를 적용
- `lambda x: pd.DataFrame(s_imp.loc[x.name].transform(x), index=x.index, columns=x.columns)`는 각 그룹에 대해 수행되는 함수입니다.
  여기서는 해당 product_code에 해당하는 IterativeImputer 모델을 가져와서 그룹 데이터에 대해 결측치를 대체합니다.
  이렇게 만들어진 결측치가 처리된 DataFrame이 각 그룹에 대해 반환됩니다.
- 반환된 결과는 원래의 데이터프레임에 열 이름과 인덱스가 그대로 유지되면서 적용됩니다.

이렇게 설정된 코드는 각 제품 코드별로 IterativeImputer 모델을 생성하고
해당 모델을 사용하여 결측치를 대체하여 데이터프레임을 업데이트하는 과정을 나타냅니다.
'''

from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import LinearRegression

# 방법 3: product_code 별로 IterativeImputer를 학습시키고,
# product_code 별로 결측을 처리합니다.

# 결측치 처리 모델을 생성 시킵니다.
def create_imp():
    return IterativeImputer(
        estimator=LinearRegression(fit_intercept=True),
        random_state=123
    )
```

```
X_imp = ['measurement_{}'.format(i) for i in range(3, 10)] + ['measurement_17']
# product_code 별로 모델을 **생성**시키고, 모델을 학습시킵니다.
s_imp = df_basetable.groupby('product_code')[X_imp].apply(lambda x: create_imp().fit(x))

# apply의 lambda x: 에서 넘겨주는 DataFrame의 .name attribute에는 해당 product_code
# 값이 있습니다.
# 이를 활용하여 s_imp에 있는 모델을 가져옵니다.
df_basetable[X_imp] = df_basetable.groupby('product_code')[X_imp].apply(
    lambda x: pd.DataFrame(s_imp.loc[x.name].transform(x), index=x.index, columns=x.columns)
)
```

In [17]: # 방법1, 방법2, 방법3의 차이점 비교

...

이 세 가지 방법은 모두 결측치를 처리하는 접근 방식이 다릅니다.

1. 방법 1: 반복문을 통한 방법

- 개요: 각 제품 코드(product_code)별로 반복문을 사용하여 데이터를 처리합니다.
데이터프레임을 여러 번 반복하여 결측치를 대체하는 방식입니다.
- 구현 방식: 반복문을 통해 각 제품 코드별로 데이터를 선택하고, 해당 데이터에 대해 **IterativeImputer**를 적용하여 결측치를 처리합니다.
- 특징: 반복문을 사용하므로 데이터프레임이 반복적으로 변경되며, 결측치 대체가 여러 번 수행됩니다.
제품 코드가 많을수록 처리 시간이 더 오래 걸릴 수 있습니다.

2. 방법 2: groupby를 통한 방법

- 개요: **groupby** 메서드를 사용하여 제품 코드별로 데이터를 그룹화하고, 각 그룹에 대해 결측치를 처리합니다.
벡터화된 연산을 사용하여 효율적으로 처리합니다.
- 구현 방식: **groupby** 메서드를 사용하여 각 제품 코드별로 데이터를 그룹화하고, 각 그룹에 대해 함수를 적용하여 결측치를 처리합니다.
- 특징: 벡터화된 연산을 사용하므로 처리 속도가 빠르고, 데이터프레임이 한 번에 처리됩니다.
반복문을 사용하지 않기 때문에 코드가 간결해지고, 가독성이 높아집니다.

3. 방법 3: 함수를 사용하여 방법 2 개선

- 개요: 방법 2와 비슷하지만, 함수를 사용하여 각 제품 코드별로 모델을 생성하고 학습시킵니다.
- 구현 방식: 함수를 정의하여 **IterativeImputer** 모델을 생성하고, 각 제품 코드별로 모델을 학습시킵니다.
그 후에, **groupby**를 사용하여 데이터를 그룹화하고, 각 그룹에 대해 함수를 적용하여 결측치를 처리합니다.
- 특징: 방법 2와 비슷하지만, 모델을 별도의 함수로 분리하여 가독성을 높이고, 재사용성을 높입니다.
모델 학습을 미리 수행하여 코드 실행 속도를 높일 수 있습니다.

...

print('')

Tip product_code 별로 선형모델의 회귀 계수 / 절편 구해보기

1. measurement_3~9와 measurement_17 에서 하나도 결측이 없는 레코드를 뽑아

df_tip 이라는 데이터프레임을 만듭니다.

2. product_code별로 measurement_3 ~ 9를 입력 변수로 하여 measurement_17을

예측하는 절편이 있는 선형 회귀 모델을 만듭니다.

3. product_code별로 제작된 회귀 모델의 회귀 계수와 절편을 구해봅니다.

4. product_code별로 제작된 선형회귀모델을 사용하여 measurement_17에 대한

예측값을 만들어 봅니다.

In [18]:

```
'''
아래 코드는 특정 제품 코드별로 선형 회귀 모델을 학습시키고, 그 모델의 계수와 절편을 분석하기 쉽게 데이터프레임으로 구성합니다.

1. 데이터 프레임 생성:
이 줄에서는 `df_basetable` 데이터프레임에서 `measurement_3`부터 `measurement_9`까지의 열들과
`measurement_17`, `product_code` 열을 선택하여 새로운 데이터프레임 `df_tip`을 만듭니다.
선택한 열들에 결측치가 있는 행은 모두 제거합니다.

2. 특징 열 설정:
이 줄에서는 특징 열(feature columns)의 이름들을 리스트 `X_lr`에 저장합니다.
이 리스트는 선형 회귀 모델의 독립 변수들로 사용됩니다.

3. 그룹별 선형 회귀 모델 학습:
이 줄에서는 `product_code`별로 데이터프레임을 그룹화한 후, 각 그룹에 대해 선형 회귀 모델을
학습시킵니다. `apply` 함수를 사용하여 각 그룹에 대해 다음의 작업을 수행합니다:
- 독립 변수 `X_lr`에 해당하는 데이터를 추출합니다.
- 종속 변수 `measurement_17`에 대한 선형 회귀 모델을 학습시킵니다.
- 학습된 모델 객체를 반환합니다.

4. 계수와 절편 추출:
이 줄에서는 학습된 모델들에서 계수와 절편을 추출하여 데이터프레임으로 구성합니다.
- `s_lr.apply(lambda x: pd.Series(x.coef_, index=X_lr))`는 각 모델의 계수를 `pd.Series` 객체로 변환하고,
  특징 열 이름을 인덱스로 설정하여 데이터프레임으로 만듭니다.
- `s_lr.apply(lambda x: x.intercept_).rename('intercept')`는 각 모델의 절편을 추출하여 'intercept'라는 이름의 시리즈로 만듭니다.
- `pd.concat(..., axis=1)`는 계수와 절편 데이터프레임을 열 방향으로 합쳐서 하나의 데이터프레임으로 만듭니다.
'''
```

결과적으로, 이 코드는 각 `product\_code`별로 학습된 선형 회귀 모델의 계수와 절편을 포함하는 데이터프레임을 생성합니다.
이 데이터프레임을 통해 각 제품 코드에 대한 모델의 계수와 절편을 쉽게 분석할 수 있습니다.

```
...

df_tip = df_basetable[['measurement_{}'.format(i) for i in range(3, 10)] + ['measurement_17', 'product_code']].dropna()

X_lr = ['measurement_{}'.format(i) for i in range(3, 10)]
print('X_lr = ', X_lr)
```

```
X_lr = ['measurement_3', 'measurement_4', 'measurement_5', 'measurement_6', 'measurement_7', 'measurement_8', 'measurement_9']
```

```
In [19]: # product_code 별로 선형 회귀모델을 학습시킵니다.
s_lr = df_basetable.groupby('product_code').apply(lambda x: LinearRegression().fit(x[X_lr], x['measurement_17']))

# 계수와 절편을 가져와서, 분석에 용이한 데이터프레임을 구성합니다,.
pd.concat([
    s_lr.apply(lambda x: pd.Series(x.coef_, index=X_lr)),
    s_lr.apply(lambda x: x.intercept_).rename('itercept')
], axis=1)
```

```
Out[19]:
```

	measurement_3	measurement_4	measurement_5	measurement_6	measurement_7	measurement_8	measurement_9	itercept
product_code								
A	-0.009574	14.879578	59.776506	26.728124	25.166765	77.356760	0.000282	-2732.403839
B	18.526145	44.469572	34.972121	0.000650	87.467582	-0.004309	26.745787	-2079.755058
C	0.013919	-0.006074	52.476859	7.178322	41.665776	91.211977	36.498332	-2964.515769
E	0.015304	58.691611	66.623768	77.092424	-0.024752	73.144951	19.946191	-4097.913515

단계 3

measurement_10~16까지의 결측치는 모두 product_code별 평균으로 대체한다.

```
In [20]: '''
`transform()`과 `apply()`를 사용할 때의 차이점은 다음과 같이 요약할 수 있습니다:

### `transform()` vs `apply()`
```

- ``transform()``:
 - 목적: 주어진 함수를 그룹의 각 요소에 적용하여 원래 데이터와 동일한 크기의 객체를 반환합니다.
 - 반환 값: 원래의 데이터와 동일한 크기의 객체 (시리즈 또는 데이터프레임).
 - 적용 방식: 그룹 내의 각 값에 대해 함수를 개별적으로 적용하고, 결과를 원래의 크기와 동일하게 유지합니다.
 - 효율성: 대부분의 경우, 더 빠르고 효율적입니다.
- ``apply()``:
 - 목적: 그룹 객체에 주어진 함수를 적용하여 임의의 크기의 객체를 반환할 수 있습니다.
 - 반환 값: 다양한 형태의 객체를 반환할 수 있습니다 (스칼라, 시리즈, 데이터프레임 등).
 - 적용 방식: 더 유연하지만, 반환되는 객체의 크기와 형식이 입력과 다를 수 있습니다.
 - 효율성: 일반적으로 ``transform()`` 보다 느립니다.

차이점 예시

``transform()``을 사용한 경우

```
X_mean = ['measurement_{}'.format(i) for i in range(10, 17)]
df_basetable[X_mean] = df_basetable.groupby('product_code')[X_mean].transform(
    lambda x: x.fillna(x.mean())
)
```

- ``transform()``을 사용하면, ``groupby`` 객체의 각 그룹에 대해 ``x.fillna(x.mean())``을 적용합니다.
- 결과는 원래 데이터프레임과 동일한 크기를 가집니다.
- 각 그룹 내의 결측값이 해당 그룹의 평균값으로 대체됩니다.
- 효율적이고 빠르게 결측값을 채웁니다.

``apply()``를 사용한 경우

```
X_mean = ['measurement_{}'.format(i) for i in range(10, 17)]
df_basetable[X_mean] = df_basetable.groupby('product_code')[X_mean].apply(
    lambda x: pd.DataFrame(x.fillna(x.mean()), index=x.index, columns=x.columns)
)
```

- ``apply()``를 사용하면, ``groupby`` 객체의 각 그룹에 대해 ``pd.DataFrame(x.fillna(x.mean()), index=x.index, columns=x.columns)``을 적용합니다.
- 이 경우, 각 그룹에 대해 결측값을 대체한 후, 원래의 데이터프레임 형식과 동일하게 반환합니다.
- ``apply()``는 더 유연하지만, ``transform()``보다 효율성이 떨어질 수 있습니다.

차이점 요약

1. 결과 형식:

- ``transform()``은 입력과 동일한 크기의 객체를 반환합니다.
- ``apply()``는 더 유연하여 다양한 형식의 객체를 반환할 수 있습니다.

2. 효율성 :

- ``transform()``은 일반적으로 더 빠르고 효율적입니다.
- ``apply()``는 더 유연하지만, 경우에 따라 느릴 수 있습니다.

3. 사용 목적 :

- ``transform()``은 그룹 내의 각 요소에 함수를 적용하고, 결과를 원래 크기와 동일하게 유지하려는 경우에 적합합니다.
- ``apply()``는 더 복잡한 연산이나 변환을 수행할 때 유용합니다.

결과적으로, 두 방법 모두 각 그룹 내에서 결측값을 해당 그룹의 평균값으로 대체하지만, ``transform()``이 더 간단하고 효율적입니다. ``apply()``는 유연성을 제공하지만, 필요하지 않은 경우 사용하지 않는 것이 좋습니다.

```
import pandas as pd
import numpy as np

# 데이터프레임 생성
data = {
    'product_code': ['A', 'A', 'A', 'B', 'B', 'B'],
    'measurement_10': [1, 2, np.nan, 4, 5, np.nan],
    'measurement_11': [np.nan, 2, 3, 4, np.nan, 6],
    'measurement_12': [1, 2, 3, 4, 5, np.nan]
}
df_basetable = pd.DataFrame(data)

# transform()을 사용한 결측값 대체
X_mean = ['measurement_{}'.format(i) for i in range(10, 13)]
df_transform = df_basetable.copy()
df_transform[X_mean] = df_transform.groupby('product_code')[X_mean].transform(
    lambda x: x.fillna(x.mean())
)

# apply()을 사용한 결측값 대체
df_apply = df_basetable.copy()
df_apply[X_mean] = df_apply.groupby('product_code')[X_mean].apply(
    lambda x: pd.DataFrame(x.fillna(x.mean()), index=x.index, columns=x.columns)
)

print("Original DataFrame:\n", df_basetable)
print("\nDataFrame after transform():\n", df_transform)
print("\nDataFrame after apply():\n", df_apply)
```

```
# 결과
Original DataFrame:
  product_code  measurement_10  measurement_11  measurement_12
0           A             1.0             NaN             1.0
1           A             2.0             2.0             2.0
2           A             NaN             3.0             3.0
3           B             4.0             4.0             4.0
4           B             5.0             NaN             5.0
5           B             NaN             6.0             NaN

DataFrame after transform():
  product_code  measurement_10  measurement_11  measurement_12
0           A             1.0             2.5             1.0
1           A             2.0             2.0             2.0
2           A             1.5             3.0             3.0
3           B             4.0             4.0             4.0
4           B             5.0             5.0             5.0
5           B             4.5             6.0             4.5

DataFrame after apply():
  product_code  measurement_10  measurement_11  measurement_12
0           A             1.0             2.5             1.0
1           A             2.0             2.0             2.0
2           A             1.5             3.0             3.0
3           B             4.0             4.0             4.0
4           B             5.0             5.0             5.0
5           B             4.5             6.0             4.5
...

print('')
```

```
In [21]: '''
아래 코드는 `df_basetable` 데이터프레임에서 `measurement_10`부터 `measurement_16`까지의
열들에 대해 결측치를 해당 `product_code` 그룹의 평균값으로 채우는 작업을 수행합니다.
```

1. 특징 열 설정:

이 줄에서는 `measurement\_10`부터 `measurement\_16`까지의 열 이름을 리스트 `X\_mean`에 저장합니다.
이 리스트는 결측치를 채울 열들을 나타냅니다.

2. 그룹별 결측치 채우기:

이 부분에서는 `df\_basetable` 데이터프레임을 `product\_code` 별로 그룹화한 후, 각 그룹에 대해 결측치를 해당 그룹의 평균값으로 채웁니다. 과정은 다음과 같습니다:

- `df\_basetable.groupby('product\_code')[X\_mean]`:
 - `product\_code` 별로 데이터를 그룹화합니다.
 - `measurement\_10` 부터 `measurement\_16` 까지의 열들만 선택합니다.
- `.apply(lambda x: pd.DataFrame(x.fillna(x.mean()), index=x.index, columns=x.columns))`:
 - `apply` 함수를 사용하여 각 그룹에 대해 결측치를 채우는 작업을 수행합니다.
 - `lambda` 함수 안에서 `x.fillna(x.mean())`를 사용하여 각 그룹의 결측치를 해당 그룹의 평균값으로 채웁니다.
 - `pd.DataFrame`을 사용하여 원래 인덱스와 열 이름을 유지하면서 결측치가 채워진 데이터를 데이터프레임으로 변환합니다.
- `df\_basetable[X\_mean] = ...`:
 - 채워진 값을 원래 데이터프레임인 `df\_basetable`의 `X\_mean` 열들에 할당합니다.

이 과정을 통해, `df\_basetable`의 `measurement\_10`부터 `measurement\_16`까지의 열들에 있는 결측치가 각 `product\_code` 그룹의 평균값으로 채워지게 됩니다.
결과적으로, `product\_code` 별로 데이터의 일관성을 유지하면서 결측치를 처리할 수 있습니다.

```
# 방법 1: groupby ~ apply
# apply 사용후 groupby에 의해 구분된 수준이 불필요(1:1 변환작업이라는 점에서) 하게
# index에 붙을 수가 있습니다.
# 이 때는 DataFrame에 index를 지정하여 반환시키면 없앨 수 있습니다.

X_mean = ['measurement_{}'.format(i) for i in range(10, 17)]
df_basetable[X_mean] = df_basetable.groupby('product_code')[X_mean].apply(
    lambda x: pd.DataFrame(x.fillna(x.mean()), index=x.index, columns=x.columns)
)
```

In [22]:

```
'''
아래 코드는 `df_basetable` 데이터프레임에서 `measurement_10`부터 `measurement_16`까지의 열들에 대해
각 `product_code` 그룹의 평균값으로 결측치를 채우는 작업을 수행합니다.
여기서는 `groupby`와 `transform`을 사용하여 각 그룹의 평균값으로 결측치를 채웁니다.

1. 특징 열 설정:
   이 줄에서는 `measurement_10`부터 `measurement_16`까지의 열 이름을 리스트 `X_mean`에 저장합니다.
   이 리스트는 결측치를 채울 열들을 나타냅니다.

2. 그룹별 결측치 채우기:
   이 부분에서는 `df_basetable` 데이터프레임을 `product_code` 별로 그룹화한 후,
```

각 그룹에 대해 결측치를 해당 그룹의 평균값으로 채웁니다. 과정은 다음과 같습니다:

- ``df_basetable.groupby('product_code')[X_mean]``:
 - ``product_code``별로 데이터를 그룹화합니다.
 - ``measurement_10``부터 ``measurement_16``까지의 열들만 선택합니다.
- ``.transform(lambda x: x.fillna(x.mean()))``:
 - ``transform`` 함수는 그룹화된 데이터에 대해 각 그룹별로 변환 작업을 수행하고, 원래의 인덱스를 유지한 채 변환된 값을 반환합니다.
 - ``lambda x: x.fillna(x.mean())``는 각 그룹에서 결측치를 해당 그룹의 평균값으로 채우는 작업을 수행합니다.
 - 이 변환은 그룹별로 이루어지며, 결과는 원래 데이터프레임의 인덱스와 일치합니다.
- ``df_basetable[X_mean] = ...``:
 - 채워진 값을 원래 데이터프레임인 ``df_basetable``의 ``X_mean`` 열들에 할당합니다.

이 과정을 통해, ``df_basetable``의 ``measurement_10``부터 ``measurement_16``까지의 열들에 있는 결측치가 각 ``product_code`` 그룹의 평균값으로 채워집니다.
이렇게 하면 각 그룹별로 데이터의 일관성을 유지하면서 결측치를 처리할 수 있습니다.

방법 2: groupby ~ transform

*# Transform을 통해 수준별 평균으로 1:1 변환된 값으로 구성된 DataFrame을 받아서,
각각의 요소별로 결측치이면 해당값으로 치환되게 구성합니다.*

```
X_mean = ['measurement_{}'.format(i) for i in range(10, 17)]
df_basetable[X_mean] = df_basetable.groupby('product_code')[X_mean].transform(
    lambda x: x.fillna(x.mean())
)
```

In [23]:

아래 코드는 ``df_basetable`` 데이터프레임에서 ``measurement_10``부터 ``measurement_16``까지의 열들에 대해 각 ``product_code`` 그룹의 평균값으로 결측치를 채운 후, 그룹화로 인해 생성된 인덱스를 제거합니다.
이를 통해 원래 데이터프레임의 형태를 유지하면서 결측치를 채우는 작업을 수행합니다.

1. 특징 열 설정:

이 줄에서는 ``measurement_10``부터 ``measurement_16``까지의 열 이름을 리스트 ``X_mean``에 저장합니다.
이 리스트는 결측치를 채울 열들을 나타냅니다.

2. 그룹별 결측치 채우기 및 인덱스 리셋:

이 부분에서는 ``df_basetable`` 데이터프레임을 ``product_code``별로 그룹화한 후,
각 그룹에 대해 결측치를 해당 그룹의 평균값으로 채운 후, 그룹화로 인해 생성된 인덱스를 제거합니다.

- ``df_basetable.groupby('product_code')[X_mean]``:
 - ``product_code``별로 데이터를 그룹화합니다.
 - ``measurement_10``부터 ``measurement_16``까지의 열들만 선택합니다.

- `.apply(lambda x: x.fillna(x.mean()))``:
 - `apply`` 함수의 `lambda x: x.fillna(x.mean())``는 각 그룹에서 결측치를 해당 그룹의 평균값으로 채우는 작업을 수행합니다.
- `.reset_index(level=0, drop=True)``:
 - 그룹화로 인해 생성된 `product_code`` 인덱스를 제거합니다.
 - `level=0``은 첫 번째(가장 바깥쪽) 인덱스 레벨을 리셋하라는 의미입니다.
 - `drop=True``는 인덱스를 열로 추가하지 않고 완전히 제거하라는 의미입니다.
- `df_basetable[X_mean] = ...``:
 - 채워진 값을 원래 데이터프레임인 `df_basetable``의 `X_mean`` 열들에 할당합니다.

이 과정을 통해 `df_basetable``의 `measurement_10``부터 `measurement_16``까지의 열들에 있는 결측치가 각 `product_code`` 그룹의 평균값으로 채워집니다.

또한, 그룹화로 인해 생성된 인덱스를 제거하여 원래 데이터프레임의 형태를 유지할 수 있습니다.

```
'''
```

방법 2-2:

0 수준에 생성된 `product_code`` 인덱스를 `reset_index``로 제거합니다.

```
df_basetable[X_mean] = df_basetable.groupby('product_code')[X_mean].apply(
    lambda x: x.fillna(x.mean())
).reset_index(level=0, drop=True)
```

In [24]: `Image('2-2.png')`

Out[24]: **Hint]** 전처리 단계에서 보간 결과를 확인해 보기 위한 각 변수의 평균과 표본표준편차.

	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
mean	17.796	11.736	17.131	17.506	11.719	19.022	11.434	16.034	19.194	11.734	15.666	16.033	15.051	16.398	701.768
std	0.997	0.994	0.994	0.992	0.993	1.005	0.997	1.278	1.579	1.433	1.149	1.461	1.478	1.671	119.180

열의 이름의 숫자는 `measurement_`` 번호, 값은 소수점 3째 자리까지 반올림

In [25]: `'''`

이 코드는 `df_basetable`` 데이터프레임의 `measurement_3``부터 `measurement_17``까지의 열들에 대해 평균(mean)과 표준편차(std)를 계산하고, 결과를 소수점 셋째 자리까지 반올림한 후, 열 이름을 간단하게 변경합니다.

1. 특정 열 선택:

이 부분에서는 `df_basetable`` 데이터프레임에서 `measurement_3``부터 `measurement_17``까지의 열들을 선택합니다.
`range(3, 18)``는 3부터 17까지의 숫자 범위를 생성하며, `['measurement_{}'.format(i) for i in range(3, 18)]``는 각 숫자에 대해 `measurement_`` 접두사를 붙여 열 이름의 리스트를 만듭니다.

2. 통계량 계산:

이 부분에서는 선택된 열들에 대해 평균(mean)과 표준편차(std)를 계산합니다.
 `agg` 함수는 여러 통계량을 한 번에 계산할 수 있게 합니다.
 여기서는 'mean'과 'std'를 인수로 주어 각각 평균과 표준편차를 계산합니다.

3. 반올림:

이 부분에서는 계산된 통계량을 소수점 셋째 자리까지 반올림합니다.

4. 열 이름 변경:

이 부분에서는 열 이름을 변경합니다. 각 열 이름에서 'measurement_' 부분을 제거하고 숫자 부분만 남깁니다.
 - `lambda x: x.split('\_')[-1]`는 각 열 이름 `x`에 대해 'measurement_'를 기준으로 문자열을 분할(split)하고, 마지막 부분(-1)을 가져옵니다. 예를 들어, 'measurement_3'는 분할 후 ['measurement', '3']이 되며, 마지막 부분인 '3'을 가져옴

5. 결과 데이터프레임:

- 최종적으로, `df\_basetable` 데이터프레임의 `measurement\_3`부터 `measurement\_17`까지의 열들에 대해 평균과 표준편차를 계산한 결과를 담은 새로운 데이터프레임이 생성됩니다.
- 이 데이터프레임은 각 통계량에 대해 소수점 셋째 자리까지 반올림된 값이 들어 있으며, 열 이름은 'measurement_' 접두사가 제거된 숫자로 단순화됩니다.

전체 코드 설명

- 열 선택: `df\_basetable`에서 `measurement\_3`부터 `measurement\_17`까지의 열들을 선택합니다.
- 통계량 계산: 선택된 열들에 대해 평균과 표준편차를 계산합니다.
- 반올림: 계산된 통계량을 소수점 셋째 자리까지 반올림합니다.
- 열 이름 변경: 열 이름을 'measurement_' 부분을 제거한 숫자로 변경합니다.

결과적으로, 이 코드는 `df\_basetable`의 특정 열들에 대한 요약 통계량(평균과 표준편차)을 간단한 형태로 제공하는 데이터프레임을 생성함
 ...

```
df_basetable[['measurement_{}'.format(i) for i in range(3, 18)]].agg(['mean', 'std']).round(3)\
    .rename(columns=lambda x: x.split('_')[-1])
```

Out[25]:

	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
mean	17.796	11.736	17.131	17.506	11.719	19.022	11.434	16.034	19.194	11.734	15.666	16.033	15.051	16.398	701.768
std	0.997	0.994	0.994	0.992	0.993	1.005	0.997	1.278	1.579	1.433	1.149	1.461	1.478	1.671	119.180

문제1

(basetable을 사용) measurement_3~16까지 결측 여부가 failure에 영향이 있는지를 파악하고,

failure를 분류하는 데 도움이 될 만한 것은 예측 모델의 입력 변수로 사용하고자 한다.

이를 위해 전처리 과정에서 뽑아낸 `isna_3~16`을 활용한다.

`n`이 3부터 16까지, 즉 `measurement_3~16`까지 다음의 검정을 수행한다.

$$H_0 : P(failure = True | measurement_n = Missing) = P(failure = True)$$

$$H_1 : P(failure = True | measurement_n = Missing) \neq P(failure = True)$$

모집단의 $P(failure = True) = 0.2114$

In []:

단계 1-1

우선, `measurement_3`으로 위 검정을 시행해보자.

$$H_0 : P(failure = True | isna_3 = True) = 0.2114$$

$$H_1 : P(failure = True | isna_3 = True) \neq 0.2114$$

으로 바꿀 수 있다.

$P(failure = True | isna_3 = True)$ 은 표본수가 충분하여 중심극한정리에 의해

정규분포를 따르는 것은 분석가 간에 이견이 없다고 한다.

위 검정의 p-value를 구하여 보고 힌트에 주어진 p-value와 비교하여

검정 방법에 문제가 없음을 확인하라.

함수 가이드

`scipy.stats` 에서 제공 기능 활용

문제 지시사항 외 Default 값 사용

Hint] p-value는 0.0037(소수점 다섯째 자리에서 반올림하여 넷째 자리까지 표시)

In [26]:

```
'''
이 코드는 모집단과 표본에 대한 비율 검정을 수행하여 `isna_3`가 결측인 데이터에서
`failure`가 `True`인 비율이 모평균과 얼마나 차이가 나는지 평가하는 과정을 설명합니다.

### 1. 모집단의 비율과 분산 계산
- `p_true`는 모집단에서 `failure`가 `True`인 비율을 나타냅니다. 여기서 `0.2114`로 설정되어 있습니다.
- `p_var`는 베르누이 분포의 분산을 계산합니다. 베르누이 분포의 분산은  $p * (1 - p)$ 로 계산됩니다.

### 2. 표본의 크기 계산
- `n_sam`는 `df_basetable` 데이터프레임에서 `isna_3`가 `True`인 (즉, 결측인) 개수입니다. 이는 표본의 크기를 나타냅니다.

### 3. 표본의 평균과 분산 계산
- `p_sam_true`는 표본의 평균으로, 모집단의 비율 `p_true`를 사용합니다.
- `p_sam_var`는 표본 평균의 분산으로, 이는 모집단 분산 `p_var`를 표본 크기 `n_sam`으로 나누어 계산합니다.

### 4. 표본에서 관측한 비율 계산
- `p_ob_true`는 `isna_3`가 `True`인 행들에서 `failure`가 `True`인 비율을 계산합니다. 이는 해당 부분집합의 평균을 의미합니다.

### 5. 검정 통계량 계산
- `Z`는 검정 통계량으로, 표본 비율 `p_ob_true`와 모집단 비율 `p_sam_true`의 차이를 표본 평균의 표준편차로 나눈 값입니다.
- 양측 검증을 위해 `Z` 값의 절대값에 마이너스를 취해 좌측 꼬리 영역을 검토합니다.

### 6. p-값 계산 및 결과 출력
- `norm.cdf(Z) * 2`를 통해 양측 검증의 p-값을 계산합니다. `norm.cdf`는 표준 정규 분포의 누적 분포 함수로,
  `Z` 값에 대한 누적 확률을 반환합니다. 이를 `2`배 하면 양측 검증의 p-값이 됩니다.
- 최종적으로 표본의 크기 `n_sam`, 표본 비율 `p_sam_true`, 표본 분산 `p_sam_var`,
  관측된 비율 `p_ob_true`, 검정 통계량 `Z`, 양측 검증의 p-값을 반환합니다.

이 코드는 베르누이 분포의 성질을 이용하여 표본에서 관측된 비율과 모집단의 비율이 통계적으로 유의미하게 다른지 평가합니다.
이는 통계적 가설 검정의 한 예로, 여기서 p-값을 통해 두 비율의 차이가 우연에 의한 것인지,
아니면 실제로 유의미한 차이가 있는지를 판단할 수 있습니다.
'''

from scipy.stats import norm

# 모집단의 failure가 True인 비율(True = 1로 매칭한다면, 평균)
```

```

# 베르누이 분포는 확률이 곧 평균입니다.
p_true = 0.2114
# 베르누이 분포의 성질을 이용하면, 모집단의 분산을 구할 수 있습니다. 분산 =  $p \times (1 - p)$ 
p_var = p_true * (1 - p_true)

# isna_3가 결측인 개수(표본수)
n_sam = df_basetable['isna_3'].sum()

# 표본평균 모평균 = 모평균
p_sam_true = p_true
# 표본의 평균의 분산 = 모분산 / 표본수
p_sam_var = p_var / n_sam

# 표본에서 관측한 비율(평균)을 구합니다.
p_ob_true = df_basetable.loc[df_basetable['isna_3'], 'failure'].mean()

# 검정통계량을 구합니다.
# 양측 검증이라 적용의 편의성을 위해
# 표준화 후 좌측 꼬리의 검정통계량을 구하기 위해, 절대값에 마이너스를 취합니다.
Z = (p_ob_true - p_sam_true) / (p_sam_var ** 0.5)
Z = -abs(Z)

# 누적분포함수를 이용하여 좌측 꼬리 영역을 구하고 x 2 를 해주어 양쪽 영역을 구합니다.
n_sam, p_sam_true, p_sam_var, p_ob_true, Z, norm.cdf(Z) * 2

```

```

Out[26]: (312,
          0.2114,
          0.0005343270512820513,
          0.14423076923076922,
          -2.905807189028007,
          0.0036630709140241546)

```

```

In [27]: '''
코드는 `df_basetable` 데이터프레임에서 특정 열(`isna_4`)의 결측치와 `failure` 비율에
대한 검정을 수행하고, 그 결과를 정규 분포 곡선 위에 시각화합니다.

### 1. 모집단 비율 및 분산 계산
- `P_true`는 모집단에서 `failure`가 `True`인 비율입니다. 여기서 `0.2114`로 설정되어 있습니다.
- `P_var`는 베르누이 분포의 분산으로, 이는 `P_true * (1 - P_true)`로 계산됩니다.

### 2. 표본 크기 및 표본 비율 계산
- `col`은 열 이름 `isna_4`를 동적으로 생성합니다.

```

```

- `n_samples`는 `df_basetable` 데이터프레임에서 `isna_4`가 `True`인 (즉, 결측인) 개수입니다. 이는 표본의 크기를 나타냅니다.
- `P_missing_true`는 `isna_4`가 `True`인 행들에서 `failure`가 `True`인 비율을 계산합니다. 이는 해당 부분집합의 평균을 의미합니다.

### 3. 표본 분산 및 표준 편차 계산
- `P_missing_var`는 표본 평균의 분산으로, 이는 모집단 분산 `P_var`를 표본 크기 `n_samples`으로 나누어 계산합니다.
- `P_missing_std`는 표본 분산의 제곱근으로, 표본 평균의 표준 편차를 의미합니다.

### 4. 검정 통계량 계산
- `stat`는 검정 통계량으로, 표본 비율 `P_missing_true`와 모집단 비율 `P_true`의 차이를 표본 평균의 표준 편차로 나눈 값입니다.
- 양측 검증을 위해 `stat` 값의 절대값에 마이너스를 취해 좌측 꼬리 영역을 검토합니다.

### 5. 정규 분포 곡선 시각화
- `np.linspace(-5, 5, 100)`은 -5에서 5까지 100개의 점을 생성합니다.
- `norm.pdf`는 표준 정규 분포의 확률 밀도 함수입니다.
- `plt.plot`는 이 확률 밀도 함수를 그래프로 그립니다.

### 전체 코드 설명
이 코드는 `df_basetable` 데이터프레임에서 특정 열(`isna_4`)의 결측치와 `failure` 비율에 대한 비율 검정을 수행하고, 이를 정규 분포 곡선 위에 시각화합니다. 이 과정에서:

1. 모집단의 비율과 분산을 계산하고,
2. 특정 열의 결측치 개수와 해당 열에서 `failure`가 `True`인 비율을 계산하며,
3. 표본 분산과 표준 편차를 계산한 후,
4. 검정 통계량을 구합니다.
5. 이 검정 통계량을 정규 분포 곡선 위에 표시하여 시각적으로 검정 결과를 보여줍니다.

이 시각화는 모집단 비율과 표본 비율의 차이가 통계적으로 유의미한지를 정규 분포 곡선과 검정 통계량을 통해 평가하는 데 유용합니다.
...

# 표준정규 분포를 이용한 pvalue 도출을 그래프로 설명해봅니다
from scipy.stats import norm
P_true = 0.2114
col = 'isna_{}'.format(4)
P_var = P_true * (1 - P_true)
n_samples = df_basetable[col].sum()
P_missing_true = df_basetable.loc[df_basetable[col], 'failure'].mean()
P_missing_var = P_var / n_samples
P_missing_std = P_missing_var ** 0.5

stat = -abs((P_missing_true - P_true) / P_missing_std)

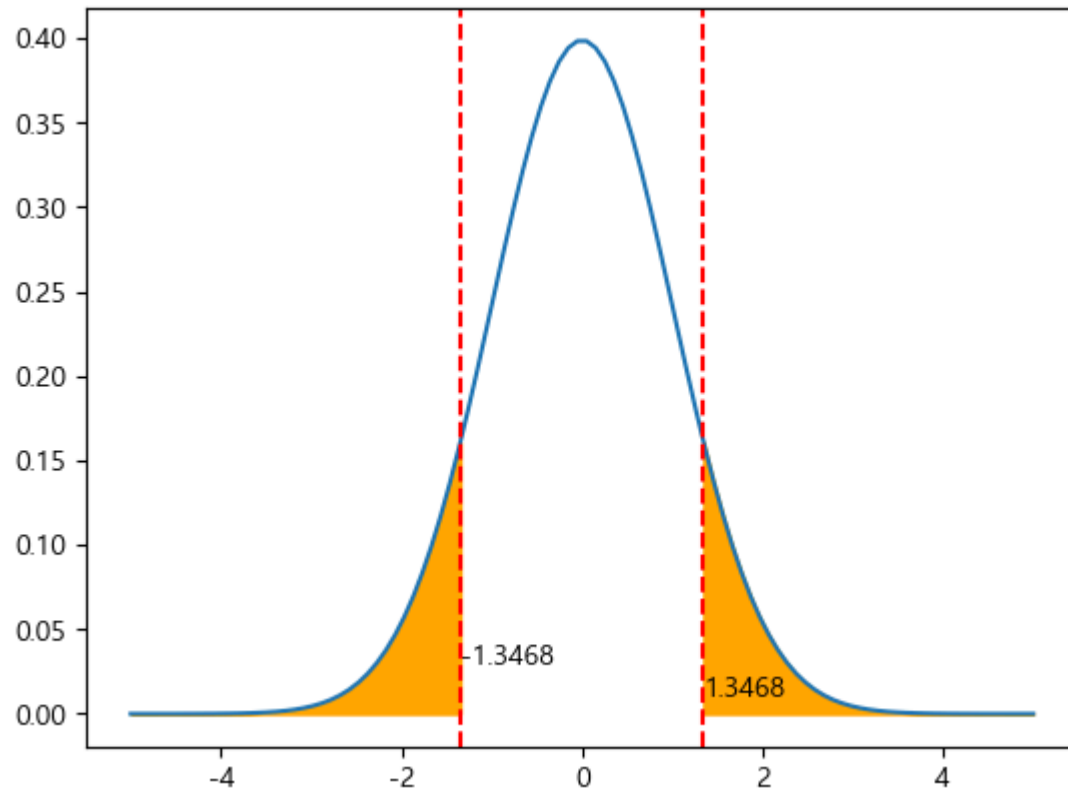
plt.plot(np.linspace(-5, 5, 100), norm.pdf(np.linspace(-5, 5, 100)))

```



```
plt.axvline(stat, color='red', linestyle='--')
plt.text(stat, 0.03, str(stat)[:7])
plt.axvline(-stat, color='red', linestyle='--')
plt.text(-stat, 0.01, str(-stat)[:6])
plt.fill_between(np.linspace(-5, stat, 100), np.zeros(shape=100),
                 norm.pdf(np.linspace(-5, stat, 100)), color='orange')
plt.fill_between(np.linspace(-stat, 5, 100), np.zeros(shape=100),
                 norm.pdf(np.linspace(-stat, 5, 100)), color='orange')

plt.show()
```



In [28]: `'''`

``proportions_ztest`` 함수는 ``statsmodels`` 라이브러리의 ``stats`` 모듈에 속한 함수로, 두 집단의 비율 차이를 검정하기 위해 사용됩니다. 이 함수는 두 집단의 비율 차이가 통계적으로 유의미한지 여부를 평가합니다.

함수의 인자

``proportions_ztest(count, nobs, value=None, alternative='two-sided', prop_var=False)``

```

1. count:
  - 유형: 배열 또는 리스트
  - 설명: 각 집단에서 성공 횟수를 나타내는 값입니다. 두 개의 값이 들어가야 하며, 예를 들어, [success1, success2] 형태로 입력합니다.

2. nobs:
  - 유형: 배열 또는 리스트
  - 설명: 각 집단의 관찰 수(표본 크기)를 나타내는 값입니다. 두 개의 값이 들어가야 하며, 예를 들어, [nobs1, nobs2] 형태로 입력합니다.

3. value:
  - 유형: 실수 (기본값은 0)
  - 설명: 검정하고자 하는 두 집단의 비율 차이입니다. 기본적으로는 두 집단의 비율이 같다고 가정하는 0을 사용합니다.

4. alternative:
  - 유형: 문자열 (기본값은 'two-sided')
  - 설명: 대립 가설의 형태를 나타냅니다. 선택 가능한 값은 'two-sided', 'larger', 'smaller'가 있습니다.
    - 'two-sided': 두 집단의 비율이 서로 다르다는 가설.
    - 'larger': 첫 번째 집단의 비율이 더 크다는 가설.
    - 'smaller': 첫 번째 집단의 비율이 더 작다는 가설.

5. prop_var:
  - 유형: bool 또는 배열 (기본값은 False)
  - 설명: 분산을 계산하는 방법을 지정합니다.
    - False인 경우, 합동 비율을 사용하여 분산을 계산합니다.
    - True인 경우, 두 집단의 비율 차이에 대한 가설을 사용하여 분산을 계산합니다.
    - 배열인 경우, 직접 분산 값을 지정할 수 있습니다.

### 리턴값
함수는 두 개의 값을 반환합니다:

1. z_stat:
  - 유형: 실수
  - 설명: Z-통계량 값으로, 이는 표준 정규 분포에서의 위치를 나타냅니다.

2. p_value:
  - 유형: 실수
  - 설명: p-값으로, 귀무 가설이 참일 때 현재 데이터가 관찰될 확률을 나타냅니다.
    일반적으로 p-값이 0.05보다 작으면 귀무 가설을 기각합니다.

### 요약
`proportions_ztest` 함수는 두 집단의 비율 차이에 대한 Z-검정을 수행하여,
두 집단의 비율이 통계적으로 유의미하게 다른지 평가하는데 유용합니다.
이를 위해 성공 횟수와 표본 크기를 입력받아 Z-통계량과 p-값을 반환합니다.
...

...

코드는 `proportions_ztest` 함수를 사용하여 `df_basetable` 데이터프레임에서
`isna_3`가 `True`인 경우의 `failure` 비율에 대한 비율 검정을 수행합니다.

```

`proportions\_ztest` 함수는 모집단 비율에 대한 가설 검정을 수행하는 함수입니다.

1. 비율 검정 함수 호출

2. 각 인자의 의미

1. positive count (`df\_basetable.loc[df\_basetable['isna\_3'], 'failure'].sum()`):

- `df\_basetable.loc[df\_basetable['isna\_3'], 'failure'].sum()`은
`df\_basetable` 데이터프레임에서 `isna\_3`가 `True`인 경우의 `failure`가 `True`인 빈도수(합계)를 계산합니다.
- 이는 표본에서 `failure`가 `True`인 사례의 수를 의미합니다.

2. sample size (`df\_basetable['isna\_3'].sum()`):

- `df\_basetable['isna\_3'].sum()`은 `df\_basetable` 데이터프레임에서 `isna\_3`가 `True`인 행의 수를 계산합니다.
- 이는 표본의 크기를 나타냅니다.

3. population proportion (`p\_true`):

- `p\_true`는 귀무가설에서 제시하는 모집단의 비율입니다. 여기서 `0.2114`로 설정되어 있습니다.

4. alternative hypothesis (`two-sided`):

- `two-sided`는 양측 검정을 의미합니다. 이는 표본 비율이 모집단 비율과 다르다고 가정하는 대립 가설을 테스트합니다.

5. population variance (`prop\_var=p\_true`):

- `prop\_var=p\_true`는 비율 검정에서 사용할 모집단의 분산을 설정합니다.
베르누이 분포의 분산은 $p * (1 - p)$ 로 계산되며, 여기서는 모집단의 비율과 동일하게 설정합니다.

4. 함수 실행 결과

- `proportions\_ztest` 함수는 두 가지 값을 반환합니다:

1. z-statistic: 검정 통계량
2. p-value: p-값

이 두 값은 표본 비율이 모집단 비율과 통계적으로 유의미하게 다른지 평가하는 데 사용됩니다.

코드의 전체 설명

- 이 코드는 `df\_basetable` 데이터프레임에서 `isna\_3`가 `True`인 경우의 `failure` 비율에 대해,
주어진 모집단 비율 `p\_true` (여기서는 `0.2114`)와 비교하여 비율 검정을 수행합니다.
- `proportions\_ztest` 함수는 표본의 `failure` 비율이 모집단 비율과 유의미하게 다른지를 평가하는 데 사용됩니다.
- 양측 검정을 통해 표본 비율이 모집단 비율보다 크거나 작은지 양쪽 방향으로 평가합니다.
- 이 검정의 결과로, `z-statistic`과 `p-value`가 반환됩니다.
`p-value`가 통계적 유의 수준(예: `0.05`)보다 작으면 귀무 가설을 기각하고, 표본 비율이 모집단 비율과 유의미하게 다르다고 결론 내림

예를 들어, 다음과 같이 호출하고 결과를 확인할 수 있습니다:

```
from statsmodels.stats.proportion import proportions_ztest
```

```

count = df_basetable.loc[df_basetable['isna_3'], 'failure'].sum()
nobs = df_basetable['isna_3'].sum()

z_stat, p_value = proportions_ztest(count, nobs, p_true,
                                    alternative='two-sided', prop_var=p_true)

print(f"Z-statistic: {z_stat}, p-value: {p_value}")

이 출력은 `Z-statistic`과 `p-value`를 통해 검정 결과를 제공합니다.
'''

from statsmodels.stats.proportion import proportions_ztest

proportions_ztest(
    # positive(failure=True)인 빈도수
    df_basetable.loc[df_basetable['isna_3'], 'failure'].sum(),
    df_basetable['isna_3'].sum(), # 표본의 수
    p_true, # 귀무가설에서 제시한 모집단의 비율
    'two-sided', # 검정의 종류: 양측 검정
    # 베르누이 분포의 분산을 계산할 때의 비율 (모집단의 비율과 동일하게 설정합니다.)
    prop_var = p_true
)

```

Out[28]: (-2.905807189028007, 0.0036630709140241546)

단계 1-2

measuremenet_3을 포함하여 measurement_4 ~ 16까지 위 검정을 반복하고

귀무가설을 기각할 수 있는 경우의 p-value의 합을 A라고 한다. (유의 수준은 5%로 한다.)

```

In [29]: '''
아래 코드는 `df_basetable` 데이터프레임의 특정 열(`isna_3`에서 `isna_16`까지)의 결측치와
`failure` 비율에 대한 비율 검정을 반복적으로 수행하고, 그 결과를 데이터프레임에 저장합니다.

### 1. 초기 설정 및 빈 리스트 생성
- 결과를 저장할 빈 리스트 `result`를 생성합니다.

### 2. 반복문을 통한 각 열에 대한 검정 수행
'''

```

```

- `for` 루프는 3부터 16까지의 숫자(i)에 대해 반복합니다.
- `na_col`은 동적으로 생성된 열 이름으로, 예를 들어 `i`가 3일 때 `na_col`은 `isna_3`입니다.
- `p_true`는 모집단에서 `failure`가 `True`인 비율입니다.
- `p_var`는 베르누이 분포의 분산으로,  $p\_true * (1 - p\_true)$ 로 계산됩니다.

### 3. 표본 크기 및 표본 분산 계산
- `n_sam`은 `df_basetable` 데이터프레임에서 `na_col`이 `True`인 (즉, 결측인) 개수입니다. 이는 표본의 크기를 나타냅니다.
- `p_sam_true`는 표본의 평균으로, 모집단의 비율 `p_true`를 사용합니다.
- `p_sam_var`는 표본 평균의 분산으로, 이는 모집단 분산 `p_var`를 표본 크기 `n_sam`으로 나누어 계산합니다.

### 4. 표본에서 관측한 비율 계산
- `p_ob_true`는 `na_col`이 `True`인 행들에서 `failure`가 `True`인 비율을 계산합니다. 이는 해당 부분집합의 평균을 의미합니다.

### 5. 검정 통계량 계산
- `Z`는 검정 통계량으로, 표본 비율 `p_ob_true`와 모집단 비율 `p_sam_true`의 차이를 표본 평균의 표준 편차로 나눈 값입니다.
- 양측 검증을 위해 `Z` 값의 절대값에 마이너스를 취해 좌측 꼬리 영역을 검토합니다.

### 6. 결과를 리스트에 추가
- 각 열(`na_col`)에 대한 결과를 `pd.Series`로 생성하여 리스트 `result`에 추가합니다.
- `pd.Series`는 `na_col`(열 이름), `n_sam`(표본 크기), `Z`(검정 통계량), `pvalue`(p-값)를 포함합니다.

### 7. 결과를 데이터프레임으로 변환 및 인덱스 설정
- 리스트 `result`를 `pd.DataFrame`으로 변환합니다.
- 열 `na_col`을 인덱스로 설정하여 최종 결과 데이터프레임 `df_result`를 생성합니다.

### 전체 코드 설명
이 코드는 `df_basetable` 데이터프레임에서 `isna_3`부터 `isna_16`까지의 각 열에 대해
결측치와 `failure` 비율에 대한 비율 검정을 수행합니다. 각 열에 대해:
1. 결측치의 수를 계산하고,
2. 표본에서 `failure`가 `True`인 비율을 구한 뒤,
3. 모집단 비율과 비교하여 검정 통계량 `Z`를 계산하며,
4. 이를 바탕으로 p-값을 계산합니다.

결과는 각 열에 대한 `na_col`, `n`, `Z`, `pvalue`를 포함하는 데이터프레임 `df_result`로 저장됩니다.
이 데이터프레임은 각 열의 결측치와 `failure` 비율에 대한 비율 검정 결과를 요약하여 제공합니다.
...

from scipy.stats import norm

result = list()
for i in range(3, 17):
    na_col = 'isna_{}'.format(i)

```

```
p_true = 0.2114
p_var = p_true * (1 - p_true)

n_sam = df_basetable[na_col].sum()
p_sam_true = p_true
p_sam_var = p_var / n_sam

p_ob_true = df_basetable.loc[df_basetable[na_col], 'failure'].mean()

Z = (p_ob_true - p_sam_true) / (p_sam_var ** 0.5)
Z = -abs(Z)

result.append(pd.Series([na_col, n_sam, Z, norm.cdf(Z) * 2], index=['na_col', 'n', 'Z', 'pvalue']))
df_result = pd.DataFrame(result).set_index('na_col')
df_result
```

Out[29]:

	n	Z	pvalue
na_col			
isna_3	312	-2.905807	0.003663
isna_4	442	-1.346824	0.178037
isna_5	565	-2.221383	0.026325
isna_6	640	-0.261780	0.793491
isna_7	766	-0.890904	0.372981
isna_8	853	-0.698051	0.485145
isna_9	989	-0.539344	0.589650
isna_10	1059	-0.762200	0.445940
isna_11	1180	-0.103525	0.917546
isna_12	1287	-0.609528	0.542174
isna_13	1395	-0.452658	0.650795
isna_14	1482	-0.681067	0.495829
isna_15	1603	-0.007695	0.993860
isna_16	1708	-0.478314	0.632427

In [30]:

```
'''
아래 코드는 이전에 수행된 비율 검정의 결과에서 p-값이 0.05보다 작은 항목들(즉, 통계적으로 유의미한 항목들)을 필터링하고,
이 항목들의 p-값 합계를 계산합니다.

### 1. 유의미한 결과 필터링
- `df_result` 데이터프레임에서 p-값이 0.05보다 작은 행들을 필터링합니다.
- `query` 메서드를 사용하여 조건에 맞는 행들을 선택합니다.
- `df_reject`는 p-값이 0.05보다 작은 행들로 구성된 데이터프레임입니다.

### 2. p-값 합계 계산
- `df_reject` 데이터프레임의 `pvalue` 열에 있는 모든 값의 합계를 계산합니다.
- `sum` 메서드를 사용하여 p-값의 합계를 구합니다.
- `A`는 p-값이 0.05보다 작은 항목들의 p-값 합계를 나타냅니다.
'''
```

전체 코드 설명

이 코드는 다음과 같은 순서로 동작합니다:

1. 유의미한 결과 필터링: `df_result` 데이터프레임에서 `p`-값이 `0.05`보다 작은 행들만을 선택하여 `df_reject` 데이터프레임을 생성합니다. 이는 비율 검정 결과 중에서 통계적으로 유의미한 결과만을 추출합니다.
2. `p`-값 합계 계산: `df_reject` 데이터프레임에서 `'pvalue'` 열의 값들을 모두 더하여 `A` 변수에 저장합니다. 이는 유의미한 검정 결과의 `p`-값 합계를 나타냅니다.

이 과정을 통해, 통계적으로 유의미한 검정 결과의 `p`-값 합계를 얻을 수 있습니다.

이를 통해 유의미한 결과가 얼마나 집중되어 있는지 혹은 얼마나 분포되어 있는지를 파악할 수 있습니다.

```
'''
```

```
df_reject = df_result.query('pvalue < 0.05')
A = df_reject['pvalue'].sum()
display(df_reject)
A
```

	n	Z	pvalue
na_col			
isna_3	312	-2.905807	0.003663
isna_5	565	-2.221383	0.026325

Out[30]: 0.029988079249523434

In [31]: '''

`chi2_contingency()` 함수는 Python의 `scipy.stats` 모듈에 포함된 함수로, 카이제곱 독립성 검정을 수행합니다. 이 검정은 두 개의 범주형 변수 사이의 독립성을 테스트하는 데 사용됩니다. 일반적으로 교차표(contingency table)를 입력으로 받아, 두 변수 간의 연관성을 평가합니다.

함수 정의

```
`chi2_contingency(observed, correction=True, lambda_=None)`
```

인자

1. observed:

- 2차원 배열(리스트)로 입력된 교차표(관측 빈도 테이블)입니다.
- 예를 들어, 각 범주의 관측 빈도를 나타내는 행렬입니다.

2. correction (기본값: `True`):

- Yates의 연속성 수정(Yates' continuity correction)을 적용할지 여부를 지정합니다.
- 2x2 교차표의 경우에만 적용됩니다.

3. lambda_ (기본값: `None`):

- 파라미터 `lambda\_`는 카이제곱 검정의 대체 형태를 지정합니다.
- 사용 가능한 값:
 - `None`: 표준 피어슨 카이제곱 검정
 - `'log'`: 로그 카이제곱 검정
 - `'sqrt'`: 제곱근 카이제곱 검정
 - 값 1: 피어슨 카이제곱 검정과 동일
 - 값 2: 로그라이크리히드 카이제곱 검정

리턴값

`chi2\_contingency()` 함수는 네 가지 값을 반환합니다:

1. Chi-square 통계량 (chi2):

- 관찰된 빈도와 기대 빈도의 차이를 측정하는 통계량입니다.

2. p-value:

- 귀무 가설(두 변수 간에 독립성이 있음)을 기각할지 여부를 결정하는 p-값입니다.
- 일반적으로 p-값이 0.05 이하이면 귀무 가설을 기각합니다.

3. 자유도 (dof):

- 카이제곱 분포의 자유도로, (행 개수 - 1) * (열 개수 - 1)로 계산됩니다.

4. 기대 빈도 (expected):

- 관찰된 빈도와 비교되는 기대 빈도 테이블입니다.
- 기대 빈도는 각 범주가 독립적이라고 가정했을 때의 빈도입니다.

속성 및 함수

`chi2\_contingency()` 함수의 주요 속성 및 함수는 입력된 교차표의 관찰 빈도와 기대 빈도를 계산하여, 두 변수 간의 독립성 여부를 평가하는 데 중점을 둡니다. 이 함수 자체는 간단하지만, 입력 데이터의 형식과 결과 해석에 있어서 다양한 활용이 가능합니다.

예제

다음은 `chi2\_contingency()` 함수를 사용하여 두 변수 간의 독립성을 테스트하는 예제입니다:

```
import numpy as np
from scipy.stats import chi2_contingency

# 예제 데이터: 교차표 생성 (관찰 빈도)
observed = np.array([[10, 10, 20],
                     [20, 20, 20]])
```

```
# 카이제곱 독립성 검정 수행
chi2, p, dof, expected = chi2_contingency(observed)

print(f"Chi-square Statistic: {chi2}")
print(f"p-value: {p}")
print(f"Degrees of Freedom: {dof}")
print("Expected Frequencies:")
print(expected)

# 결과 해석
if p < 0.05:
    print("두 변수는 독립이 아닙니다.")
else:
    print("두 변수는 독립입니다.")
```

위 예제에서, 두 변수의 관찰 빈도를 나타내는 2x3 교차표를 생성하고 `chi2_contingency()` 함수를 사용하여 카이제곱 독립성 검정을 수행합니다. Chi-square 통계량, p-값, 자유도, 기대 빈도를 출력하고, p-값을 기준으로 두 변수 간의 독립성을 해석합니다.

결론

`chi2_contingency()` 함수는 두 개의 범주형 변수 간의 독립성을 테스트하는 강력한 도구입니다. 교차표를 기반으로 관찰 빈도와 기대 빈도를 비교하여, 두 변수 간의 연관성을 평가하는 데 유용합니다. 이 함수는 사회과학, 생물학, 마케팅 등 다양한 분야에서 변수 간의 관계를 분석하는 데 널리 사용됩니다.

...

아래 코드는 두 가지 데이터프레임을 합치고, 그 과정에서 카이제곱 독립성 검정(Chi-squared test of independence)을 수행한 후의 p-값을 계산합니다.

1. 카이제곱 독립성 검정을 위한 열 선택 및 연산

- `df_basetable`에서 `isna_3`부터 `isna_16`까지의 열을 선택합니다.
- `apply` 함수를 사용하여 각 열에 대해 `lambda` 함수를 적용합니다.
- `lambda` 함수 내에서는 `pd.crosstab`을 사용하여 해당 열과 'failure' 열 간의 교차표를 생성하고, 이를 `chi2_contingency` 함수에 전달하여 카이제곱 독립성 검정을 수행합니다.
- `chi2_contingency` 함수의 두 번째 반환값은 검정의 p-값입니다.
- `rename` 메서드를 사용하여 Series의 이름을 `'chi2_contingency_pvalue'`로 변경합니다.

2. 데이터프레임 합치기

- `pd.concat` 함수를 사용하여 두 개의 데이터프레임을 합칩니다.
- 첫 번째 인자는 리스트로써, 합쳐질 데이터프레임들을 포함합니다.
- 두 번째 인자 `axis=1`은 열 방향으로 합침을 의미합니다.
- 첫 번째 데이터프레임은 이전에 생성된 `df_result`입니다.
- 두 번째 데이터프레임은 카이제곱 독립성 검정을 통해 얻은 p-값들을 포함합니다.

3. 결과

최종 결과 데이터프레임은 다음과 같이 두 가지 정보를 포함합니다:

- 각 열에 대한 비율 검정의 결과(`df\_result`)
- 각 열에 대한 카이제곱 독립성 검정의 p-값(`chi2\_contingency\_pvalue`)

이를 통해 각 열에 대한 두 가지 다른 방법으로 수행한 검정의 결과를 비교하고, 데이터프레임으로 나타내어 함께 살펴볼 수 있습니다.
...

```
from scipy.stats import chi2_contingency
```

```
pd.concat([  
    df_result,  
    df_basetable[['isna_{}'.format(i) for i in range(3, 17)]].apply(  
        lambda x: chi2_contingency(pd.crosstab(index=x, columns=df_basetable['failure']), correction=False)[1]  
    ).rename('chi2_contingency_pvalue')  
], axis=1)
```

Out[31]:

	n	Z	pvalue	chi2_contingency_pvalue
na_col				
isna_3	312	-2.905807	0.003663	0.003405
isna_4	442	-1.346824	0.178037	0.174166
isna_5	565	-2.221383	0.026325	0.024517
isna_6	640	-0.261780	0.793491	0.792186
isna_7	766	-0.890904	0.372981	0.365634
isna_8	853	-0.698051	0.485145	0.474635
isna_9	989	-0.539344	0.589650	0.582784
isna_10	1059	-0.762200	0.445940	0.436145
isna_11	1180	-0.103525	0.917546	0.912704
isna_12	1287	-0.609528	0.542174	0.531745
isna_13	1395	-0.452658	0.650795	0.637259
isna_14	1482	-0.681067	0.495829	0.482500
isna_15	1603	-0.007695	0.993860	0.996569
isna_16	1708	-0.478314	0.632427	0.615410

단계 1-3

검정 결과 귀무가설을 기각할 수 있는 경우는 총 두 건이다.

해당 파생 변수명의 뒷 자리 번호 순으로 na_1, na_2로 파생 변수를 만들어 prob1 데이터셋을 생성하라.

In [32]:

```
'''
코드는 `df_basetable` 데이터프레임에 새로운 열을 추가하는데,
이 열들은 기존의 열에 대한 변형값입니다. 각 단계를 설명하겠습니다.

### 1. `assign` 함수를 사용하여 열 추가
- `assign` 함수는 새로운 열을 추가하거나 기존 열을 변형하는데 사용됩니다.
```

- `df\_basetable` 데이터프레임에 `assign` 함수를 호출하여 새로운 데이터프레임 `df\_prob1`을 생성합니다.

2. 람다 함수를 사용하여 열 값 할당

- `assign` 함수의 각 인자는 새로운 열의 이름과 해당 열의 값에 대한 정보를 제공합니다.
- 여기서 사용된 값은 람다 함수입니다. 람다 함수는 각 행에 대해 계산되고 새로운 열에 할당됩니다.
- `lambda x: x['isna\_3']`는 기존의 'isna_3' 열을 `na\_1`이라는 새로운 열에 할당합니다.
- `lambda x: x['isna\_5']`는 기존의 'isna_5' 열을 `na\_2`라는 새로운 열에 할당합니다.

3. 결과

- `df\_prob1` 데이터프레임은 `df\_basetable`에 두 개의 새로운 열을 추가한 것입니다.
이 열들은 'isna_3'과 'isna_5' 열의 값을 그대로 복사한 것입니다.
- 따라서 `na\_1` 열은 'isna_3' 열의 값을, `na\_2` 열은 'isna_5' 열의 값을 갖습니다.
이것은 기존 열의 정보를 다른 이름으로 복사한 것으로, 데이터프레임에 새로운 관점을 추가하는 데 유용합니다.

```
df_prob1 = df_basetable.assign(
    na_1 = lambda x: x['isna_3'],
    na_2 = lambda x: x['isna_5'],
)
```

In [33]: df_prob1

Out[33]:

	product_code	loading	attribute_0	attribute_1	attribute_2	attribute_3	measurement_0	measurement_1	measurement_2	measurement_3	...	isr
id												
0	A	80.10	material_7	material_8	9	5	7	8	4	18.040	...	
1	A	84.89	material_7	material_8	9	5	14	3	3	18.213	...	
2	A	82.43	material_7	material_8	9	5	12	1	5	18.057	...	
3	A	101.07	material_7	material_8	9	5	13	2	6	17.295	...	
4	A	188.06	material_7	material_8	9	5	9	2	8	19.346	...	
...	...	...	...	...	...	...	...	...	...	...	...	...
26565	E	158.95	material_7	material_6	6	9	6	16	4	16.301	...	
26566	E	146.02	material_7	material_6	6	9	10	12	8	17.543	...	
26567	E	115.62	material_7	material_6	6	9	1	10	1	15.670	...	
26568	E	106.38	material_7	material_6	6	9	2	9	4	18.059	...	
26569	E	131.20	material_7	material_6	6	9	6	19	1	18.034	...	

21458 rows × 42 columns

In [34]:

A, round(A, 3)

Out[34]:

(0.029988079249523434, 0.03)

A의 값을 소수점 넷째 자리에서 반올림하여 셋째 자리까지 출력하시오.

0.030

문제 2

첫째는 스프링 개발 업체들은 실험이 제품 별로 공정하게 진행이 됐는지를 의문을 가지고 있다.

product_code에 따라 개발 업체가 다르다.

product_code에 대해서 스프링에 가한 부하(loading)를 동일하게 했는지 조사하라.

둘째는, attribute_0와 attribute_1은 스프링을 구성하는 주요 소재이다.

failure와는 관계가 없음이 이전에 검증되었다.

하지만, 이에 대한 재확인 요청을 받아 attribute_0와 attribute_1은 failure와 상관없음을 확인한다.

이를 위해 다음 단계를 수행하라.

In []:

단계 2-1

prob1에서 입력 변수 loading에 결측이 없는 행들을 뽑아 prob2 데이터프레임을 만든다.

Hint] prob2의 데이터 수는 21,257 이다.

In [35]:

```
df_prob2 = df_prob1.loc[df_prob1['loading'].notna()].copy()
print(len(df_prob2))
df_prob2
```

21257

Out[35]:

	product_code	loading	attribute_0	attribute_1	attribute_2	attribute_3	measurement_0	measurement_1	measurement_2	measurement_3	...	isr
id												
0	A	80.10	material_7	material_8	9	5	7	8	4	18.040	...	
1	A	84.89	material_7	material_8	9	5	14	3	3	18.213	...	
2	A	82.43	material_7	material_8	9	5	12	1	5	18.057	...	
3	A	101.07	material_7	material_8	9	5	13	2	6	17.295	...	
4	A	188.06	material_7	material_8	9	5	9	2	8	19.346	...	
...	...	...	...	...	...	...	...	...	...	...	...	
26565	E	158.95	material_7	material_6	6	9	6	16	4	16.301	...	
26566	E	146.02	material_7	material_6	6	9	10	12	8	17.543	...	
26567	E	115.62	material_7	material_6	6	9	1	10	1	15.670	...	
26568	E	106.38	material_7	material_6	6	9	2	9	4	18.059	...	
26569	E	131.20	material_7	material_6	6	9	6	19	1	18.034	...	

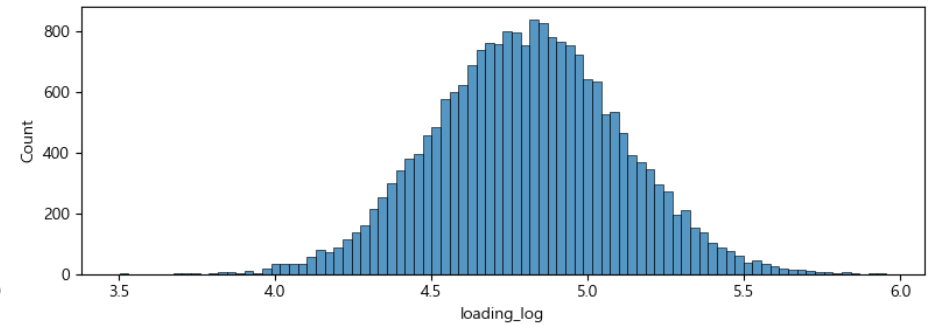
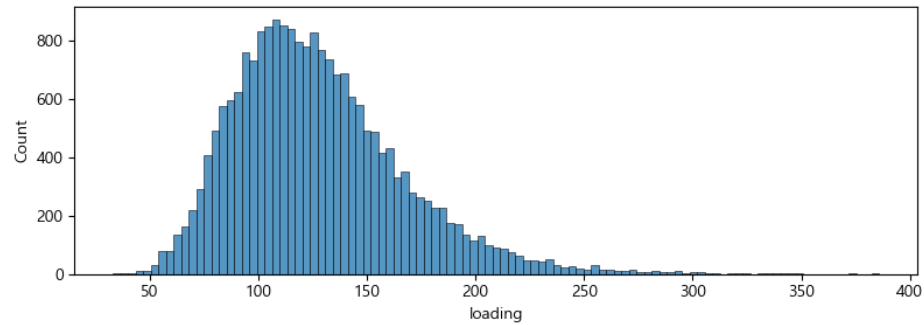
21257 rows x 13 columns

단계 2-2

prob2에 loading의 각 행들에 자연 로그 함수를 적용하여 파생 변수 loading_log를 만든다.

```
In [36]: df_prob2['loading_log'] = np.log(df_prob2['loading'])
```

```
In [37]: fig, axes = plt.subplots(1, 2, figsize=(16, 3))
sns.histplot(df_prob2['loading'], ax=axes[0])
sns.histplot(df_prob2['loading_log'], ax=axes[1])
plt.tight_layout()
plt.show()
```

단계 2-3

loading_log가 product_code 각각에 대해서 정규성을 지니고 있는지 확인하고자 한다.

이를 위해 product_code 별로 loading_log에 대한 Jarque-Bera로 검정하고,

검정 결과 정규성을 부정할 수 없는 경우의 product_code의 수를 B라고 한다.(유의 수준: 5%)

함수 가이드

scipy.stats 에서 제공 기능 활용

문제 지시사항 외 Default 값 사용

```
In [38]: '''
`jarque_bera()` 함수는 주로 통계학과 데이터 분석에서 사용되는 함수로,
Jarque-Bera 테스트를 수행하여 주어진 데이터의 정규성 여부를 검사합니다.
이 테스트는 데이터가 정규 분포를 따르는지 확인하기 위해 데이터의 왜도(skewness)와 첨도(kurtosis)를 사용합니다.

### 사용법
from scipy.stats import jarque_bera

# 데이터 준비
data = [sample_data]

# Jarque-Bera 테스트 수행
```

```
jb_stat, p_value = jarque_bera(data)
```

인자

- ``data``: 테스트할 데이터를 포함하는 배열 또는 리스트.
일반적으로 1차원 배열이어야 합니다.

리턴값

``jarque_bera()`` 함수는 두 가지 값을 반환합니다:

1. JB 통계량 (JB Statistic):
 - 테스트 통계량으로, 왜도와 첨도를 기반으로 계산됩니다.
 - 값이 작을수록 데이터가 정규 분포에 가까움을 나타냅니다.
2. p-value:
 - 관찰된 데이터가 정규 분포를 따른다는 귀무 가설을 검증하기 위한 p-값입니다.
 - p-값이 매우 작으면 (예: 0.05 이하) 귀무 가설을 기각할 수 있으며, 이는 데이터가 정규 분포를 따르지 않음을 시사합니다.

속성 및 함수

``jarque_bera()`` 함수는 다음과 같은 속성 및 함수를 포함하고 있습니다:

- 왜도 (Skewness):
 - 데이터의 비대칭 정도를 나타내는 통계량입니다.
 - 왜도가 0이면 데이터는 좌우 대칭임을 나타냅니다.
- 첨도 (Kurtosis):
 - 데이터 분포의 꼬리 두께를 나타내는 통계량입니다.
 - 첨도가 3이면 정규 분포와 유사한 꼬리 두께를 갖습니다.

예제

다음은 ``jarque_bera()`` 함수를 사용하여 데이터의 정규성을 테스트하는 예제입니다:

```
import numpy as np
from scipy.stats import jarque_bera

# 예제 데이터 생성
data = np.random.normal(0, 1, 1000) # 평균 0, 표준편차 1인 정규 분포 데이터 1000개 생성

# Jarque-Bera 테스트 수행
jb_stat, p_value = jarque_bera(data)

print(f"JB Statistic: {jb_stat}")
print(f"p-value: {p_value}")
```

```
# 결과 해석
if p_value > 0.05:
    print("데이터는 정규 분포를 따른다고 할 수 있습니다.")
else:
    print("데이터는 정규 분포를 따르지 않습니다.")
```

위 예제에서, 평균이 0이고 표준편차가 1인 정규 분포를 따르는 1000개의 데이터를 생성하고 Jarque-Bera 테스트를 수행합니다. 결과적으로 JB 통계량과 p-값을 출력하고, p-값을 기준으로 데이터가 정규 분포를 따르는지 여부를 해석합니다.

결론

`jarque\_bera()` 함수는 데이터의 정규성을 테스트하는 강력한 도구로, 주로 왜도와 첨도를 분석하여 데이터 분포의 특성을 파악하는 데 사용됩니다. 이 함수는 통계 분석과 데이터 과학에서 중요한 역할을 하며, 정규성 가정을 필요로 하는 다양한 분석 방법에 필수적입니다.

...

아래 코드는 `df\_prob2` 데이터프레임을 'product_code'로 그룹화한 후, 각 그룹에서 'loading_log' 열에 대해 Jarque-Bera 검정을 수행합니다. 그리고 각 그룹에 대해 검정 통계량과 p-값을 계산하여 결과를 데이터프레임으로 구성합니다. 이후, p-값이 0.05보다 큰 그룹의 수를 계산합니다.

1. Jarque-Bera 검정 수행 및 결과 데이터프레임 생성

- `groupby('product\_code')`는 'product_code' 열의 고유한 값들을 기준으로 데이터를 그룹화합니다.
- `['loading\_log']`는 각 그룹에서 분석할 열을 선택합니다.
- `apply(lambda x: pd.Series(jarque\_bera(x), index=['stat', 'pvalue']))`는 각 그룹에 대해 Jarque-Bera 검정을 수행합니다. `jarque\_bera` 함수는 주어진 데이터의 정규성을 검정하는데 사용됩니다. 그리고 검정 통계량(stat)과 p-값(pvalue)을 계산하여 이를 pandas Series로 반환합니다.
- `unstack(level=-1)`는 그룹 인덱스를 열로 변환하여 각 그룹에서 계산된 검정 통계량과 p-값을 포함하는 데이터프레임을 생성합니다.

2. 결과 출력 및 통계적 유의성 계산

- `display(df\_jarque)`는 Jarque-Bera 검정 결과를 출력합니다. 이는 각 'product_code' 그룹에 대해 계산된 검정 통계량(stat)과 p-값(pvalue)을 보여줍니다.
- `(df\_jarque['pvalue'] >= 0.05)`는 p-값이 0.05보다 큰지 여부를 나타내는 불리언 시리즈를 생성합니다. 이는 각 그룹의 p-값이 0.05보다 크면 True, 작으면 False가 됩니다.
- `.sum()`은 True인 값을 세어서 그룹의 수를 계산합니다. 이는 p-값이 0.05보다 큰 그룹의 수를 의미하며, 이를 변수 `B`에 할당합니다.

결과

- `df\_jarque`는 각 'product_code' 그룹에서 Jarque-Bera 검정을 수행한 결과를 담은 데이터프레임입니다.
- `B`는 p-값이 0.05보다 큰 그룹의 수를 나타내며, 이는 해당 그룹이 정규분포를 따른다고 할 수 있는 확률적인 근거가 있는 그룹의 수입니다.

```

from scipy.stats import jarque_bera

aa = df_prob2.groupby('product_code')['loading_log'].apply(
    lambda x: pd.Series(jarque_bera(x), index=['stat', 'pvalue'])
)

print(type(aa))
aa

```

```
<class 'pandas.core.series.Series'>
```

```

Out[38]: product_code
A          stat      1.467731
          pvalue    0.480050
B          stat      0.616884
          pvalue    0.734591
C          stat      7.536229
          pvalue    0.023096
E          stat      0.048395
          pvalue    0.976093
Name: loading_log, dtype: float64

```

```

In [39]: # unstack에서 정한 index의 수준을 column level에 위치하도록
# DataFrame의 형태를 변화 시켜줍니다.
df_jarque = df_prob2.groupby('product_code')['loading_log'].apply(
    lambda x: pd.Series(jarque_bera(x), index=['stat', 'pvalue'])
).unstack(level=-1)

print(type(df_jarque))

display(df_jarque)

```

```
<class 'pandas.core.frame.DataFrame'>
```

	stat	pvalue
product_code		
A	1.467731	0.480050
B	0.616884	0.734591
C	7.536229	0.023096
E	0.048395	0.976093

```
In [40]: B = (df_jarque['pvalue'] >= 0.05).sum()
B
```

```
Out[40]: 3
```

단계 2-4

loading_log 변수를 product_code로 구분했을 때,

등분산성을 보이는지 Bartlett 검정을 통해 확인한다.

검정 결과에서 p-value를 C라고 한다.

함수 가이드

scipy.stats 에서 제공 기능 활용

문제 지시사항 외 Default 값 사용

```
In [41]: '''
`Bartlett` 함수는 `scipy.stats` 모듈에 있는 함수로, Bartlett's test를 수행하여
두 개 이상의 표본이 동일한 분산을 가지는지 검정합니다.
이 테스트는 정규 분포를 따르는 데이터에 대해 분산의 동질성을 검정하는 데 사용됩니다.

### 함수의 인자
1. sample1, sample2, ...:
```

- 유형: 배열 또는 리스트
- 설명: 두 개 이상의 표본 데이터를 나타내는 배열 또는 리스트. 각 표본은 1차원 배열이어야 합니다. 함수는 두 개 이상의 인자를 받습니다.

리턴값

1. statistic:

- 유형: 실수
- 설명: Bartlett 검정 통계량. 이는 검정에 사용된 카이제곱 통계량 값을 나타냅니다.

2. p-value:

- 유형: 실수
- 설명: p-값. 귀무 가설이 참일 때, 현재 데이터가 관찰될 확률을 나타냅니다. 일반적으로 p-값이 0.05보다 작으면 귀무 가설을 기각하고, 그렇지 않으면 귀무 가설을 채택합니다.

예제

```
import numpy as np
from scipy.stats import bartlett
```

세 개의 표본 데이터 생성

```
sample1 = np.random.normal(0, 1, 100)
sample2 = np.random.normal(0, 1, 100)
sample3 = np.random.normal(0, 1, 100)
```

Bartlett's test 수행

```
statistic, p_value = bartlett(sample1, sample2, sample3)
```

```
print(f"Bartlett's test statistic: {statistic}")
```

```
print(f"Bartlett's test p-value: {p_value}")
```

이 예제에서는 세 개의 정규 분포를 따르는 표본 데이터를 생성하고, `bartlett` 함수를 사용하여 이들 표본의 분산이 동일한지 검정합니다.

요약

`scipy.stats.bartlett` 함수는 Bartlett's test를 수행하여 두 개 이상의 표본이 동일한 분산을 가지는지 평가합니다. 이 함수는 두 개 이상의 표본 데이터를 인자로 받아 검정 통계량과 p-값을 반환합니다. 일반적으로 p-값이 0.05보다 작으면 귀무 가설을 기각하여 표본들의 분산이 동일하지 않다고 결론 내릴 수 있습니다.

```
'''
```

```
print('')
```

In [42]: '''

아래 코드는 `df\_prob2` 데이터프레임의 'loading_log' 열에서 'product_code'가 각각 'A', 'B', 'C', 'E'인 그룹 간의 Bartlett 검정을 수

이를 통해 그룹 간의 등분산성을 검정하고, 그 결과의 p-값을 변수 `C`에 저장합니다.

1. Bartlett 검정 수행

- `bartlett` 함수는 Bartlett 검정을 수행하는 함수입니다. 이 함수는 주어진 데이터셋에서 각 그룹 간의 등분산성을 검정합니다.
- 각 그룹은 'product_code'가 'A', 'B', 'C', 'E'인 데이터프레임입니다. 각 그룹의 'loading_log' 열이 검정에 사용됩니다.
- 결과는 `result\_2\_4` 변수에 저장됩니다.

2. p-값 저장

- `result\_2\_4.pvalue`는 Bartlett 검정 결과의 p-값을 나타냅니다. 이를 변수 `C`에 할당합니다.

결과

- `C`는 'product_code'가 'A', 'B', 'C', 'E'인 그룹 간의 등분산성 검정의 결과인 p-값을 나타냅니다.
- p-값은 그룹 간의 등분산성 여부를 결정하는 데 사용되며, 일반적으로 유의수준 0.05 이하일 때 등분산성이 없다고 판단됩니다.
- ...

방법 1: 하드코딩

```
from scipy.stats import bartlett
result_2_4 = bartlett(
    df_prob2.loc[df_prob2['product_code'] == 'A', 'loading_log'],
    df_prob2.loc[df_prob2['product_code'] == 'B', 'loading_log'],
    df_prob2.loc[df_prob2['product_code'] == 'C', 'loading_log'],
    df_prob2.loc[df_prob2['product_code'] == 'E', 'loading_log'],
)
C = result_2_4.pvalue
result_2_4, C
```

```
Out[42]: (BartlettResult(statistic=1.928667047072185, pvalue=0.5873433093297675),
0.5873433093297675)
```

In [43]:

```
'''
`bartlett()` 함수는 Python의 `scipy.stats` 모듈에 포함된 함수로, Bartlett의 검정을 수행합니다.
이 검정은 주어진 두 개 이상의 샘플이 동일한 분산을 가지고 있는지 (동분산성, homogeneity of variances)를 테스트하는 데 사용됩니다.
Bartlett의 검정은 데이터가 정규 분포를 따른다는 가정 하에 수행됩니다.
```

함수 정의

```
`scipy.stats.bartlett(*args)`
```

인자

- ***args:**
 - 두 개 이상의 배열이나 리스트. 각 배열은 하나의 샘플을 나타내며, 모든 배열의 크기가 같을 필요는 없습니다.
 - 예를 들어, `bartlett(sample1, sample2, sample3)`와 같이 여러 샘플을 인자로 전달할 수 있습니다.

```

#### 리턴값
`bartlett()` 함수는 두 가지 값을 반환합니다:

1. 검정 통계량 (T-value):
  - Bartlett의 검정 통계량입니다.
  - 값이 클수록 샘플 간의 분산 차이가 크다는 것을 나타냅니다.

2. p-value:
  - 귀무 가설(모든 샘플이 동일한 분산을 가짐)을 기각할지 여부를 결정하는 p-값입니다.
  - 일반적으로 p-값이 0.05 이하이면 귀무 가설을 기각합니다.

#### 속성 및 함수
`bartlett()` 함수는 다음과 같은 속성과 기능을 가지고 있습니다:
- 정규성 가정:
  - Bartlett의 검정은 데이터가 정규 분포를 따른다고 가정합니다. 이 가정이 충족되지 않을 경우, 결과가 왜곡될 수 있습니다.
- 동분산성 테스트:
  - 여러 샘플이 동일한 분산을 가지는지 테스트하는 데 사용됩니다.

#### 예제
다음은 `bartlett()` 함수를 사용하여 여러 샘플 간의 분산의 동일성을 테스트하는 예제입니다:

import numpy as np
from scipy.stats import bartlett

# 예제 데이터 생성
np.random.seed(0)
sample1 = np.random.normal(0, 1, 100) # 평균 0, 표준편차 1인 정규 분포 샘플
sample2 = np.random.normal(0, 1, 100) # 평균 0, 표준편차 1인 정규 분포 샘플
sample3 = np.random.normal(0, 2, 100) # 평균 0, 표준편차 2인 정규 분포 샘플

# Bartlett 검정 수행
t_stat, p_value = bartlett(sample1, sample2, sample3)

print(f"Bartlett's Test Statistic: {t_stat}")
print(f"p-value: {p_value}")

# 결과 해석
if p_value < 0.05:
    print("샘플 간의 분산이 동일하지 않습니다.")
else:
    print("샘플 간의 분산이 동일합니다.")

```


위 예제에서, 세 개의 정규 분포 샘플을 생성하고 **Bartlett**의 검정을 수행합니다.
검정 통계량과 p-값을 출력하고, p-값을 기준으로 샘플 간의 분산이 동일한지 여부를 해석합니다.

결론

``bartlett()`` 함수는 여러 샘플의 분산이 동일한지 테스트하는 데 유용한 도구입니다.
이 검정은 데이터가 정규 분포를 따른다는 가정 하에 수행되며, p-값을 통해 귀무 가설을 기각할지 여부를 결정합니다.
통계 분석과 데이터 과학에서, 특히 분산 분석(ANOVA)와 같은 분석 기법에서 동분산성 가정을 확인하는 데 중요하게 사용됩니다.
...

...

아래 코드는 ``df_prob2`` 데이터프레임을 `'product_code'`로 그룹화한 후, 각 그룹에서 `'loading_log'` 열의 값을 리스트로 모은 뒤, 이를 **Bartlett** 검정에 적용하여 그룹 간의 등분산성을 검정합니다. 그 결과의 p-값을 변수 ``C``에 저장합니다.

여기서의 각 단계를 살펴보겠습니다:

1. 그룹화 후 리스트로 값 모으기

- ``groupby('product_code')['loading_log']``는 `'product_code'` 별로 `'loading_log'` 열을 그룹화합니다.
- ``agg(list)``는 각 그룹에서 `'loading_log'` 열의 값을 리스트로 모으는데, ``list`` 함수를 적용하여 리스트로 변환합니다.
- ``*``는 그룹화된 데이터프레임을 풀어내어 ``bartlett`` 함수에 전달할 수 있도록 합니다.
이렇게 함으로써, ``bartlett`` 함수는 각 그룹의 리스트를 별도의 인수로 처리할 수 있습니다.

2. Bartlett 검정 수행

- ``bartlett`` 함수는 **Bartlett** 검정을 수행하는 함수입니다.
이 함수는 주어진 데이터셋에서 각 그룹 간의 등분산성을 검정합니다.
- ``*``를 사용하여 각 그룹의 리스트를 별도의 인수로 전달합니다.

3. p-값 저장

- ``result_2_4.pvalue``는 **Bartlett** 검정 결과의 p-값을 나타냅니다.
이를 변수 ``C``에 할당합니다.

결과

- ``C``는 `'product_code'`별로 그룹화된 데이터에서 **Bartlett** 검정을 통해 얻은 등분산성 검정의 결과인 p-값을 나타냅니다.
- p-값은 그룹 간의 등분산성 여부를 결정하는 데 사용되며, 일반적으로 유의수준 **0.05** 이하일 때 등분산성이 없다고 판단됩니다.
...

방법 2: *groupby + list unpacking*

```
result_2_4 = bartlett(*df_prob2.groupby('product_code')['loading_log'].agg(list))
```

```
C = result_2_4.pvalue
result_2_4, C
```

```
Out[43]: (BartlettResult(statistic=1.928667047072185, pvalue=0.5873433093297675),
0.5873433093297675)
```

단계 2-5

product_code에 대한 분산분석(ANOVA)을 통해서 loading_log 평균에 차이가 있는지 검정한다.

그 결과 중 p-value를 D라고 한다.

함수 가이드

scipy.stats 제공 기능 활용

문제 지시사항 외 Default 값 사용

```
In [44]: '''
`f_oneway()` 함수는 Python의 `scipy.stats` 모듈에 포함된 함수로, 일원분산분석(ANOVA, Analysis of Variance)을 수행합니다.
이 검정은 두 개 이상의 그룹 간의 평균이 통계적으로 유의미하게 다른지 확인하는 데 사용됩니다.

### 함수 정의

scipy.stats.f_oneway(*args)

### 인자
- *args:
  - 두 개 이상의 배열이나 리스트. 각 배열은 하나의 샘플을 나타내며, 각 배열의 길이는 같을 필요가 없습니다.
  - 예를 들어, `f_oneway(sample1, sample2, sample3)`와 같이 여러 샘플을 인자로 전달할 수 있습니다.

### 리턴값

`f_oneway()` 함수는 두 가지 값을 반환합니다:

1. F-통계량 (F-statistic):
  - 그룹 간의 평균 차이를 나타내는 통계량입니다.
  - 값이 클수록 그룹 간의 평균 차이가 크다는 것을 나타냅니다.

2. p-value:
```

- 귀무 가설(모든 그룹의 평균이 같다)을 기각할지 여부를 결정하는 p-값입니다.
- 일반적으로 p-값이 0.05 이하이면 귀무 가설을 기각합니다.

속성 및 함수

`f\_oneway()` 함수는 다음과 같은 속성과 기능을 가지고 있습니다:

- 일원분산분석 (ANOVA):
 - 두 개 이상의 그룹 간의 평균 차이를 테스트하는 데 사용됩니다.
 - ANOVA는 각 그룹이 정규 분포를 따르고, 그룹 간의 분산이 동일하다는 가정을 합니다.
- F-통계량:
 - 그룹 간 변동(SSB, Sum of Squares Between)과 그룹 내 변동(SSW, Sum of Squares Within)을 사용하여 계산됩니다.
- p-value:
 - 그룹 간 평균 차이가 우연히 발생할 확률을 나타냅니다.

예제

다음은 `f\_oneway()` 함수를 사용하여 여러 그룹 간의 평균 차이를 테스트하는 예제입니다:

```
import numpy as np
from scipy.stats import f_oneway

# 예제 데이터 생성
np.random.seed(0)
group1 = np.random.normal(0, 1, 100) # 평균 0, 표준편차 1인 정규 분포 샘플
group2 = np.random.normal(0.5, 1, 100) # 평균 0.5, 표준편차 1인 정규 분포 샘플
group3 = np.random.normal(1, 1, 100) # 평균 1, 표준편차 1인 정규 분포 샘플

# 일원분산분석 (ANOVA) 수행
f_stat, p_value = f_oneway(group1, group2, group3)

print(f"F-statistic: {f_stat}")
print(f"p-value: {p_value}")

# 결과 해석
if p_value < 0.05:
    print("그룹 간 평균이 유의미하게 다릅니다.")
else:
    print("그룹 간 평균이 유의미하게 다르지 않습니다.")
```

위 예제에서, 세 개의 정규 분포 샘플을 생성하고 일원분산분석을 수행합니다.

F-통계량과 p-값을 출력하고, p-값을 기준으로 그룹 간의 평균이 유의미하게 다른지 여부를 해석합니다.

결론

`f\_oneway()` 함수는 여러 그룹 간의 평균 차이를 테스트하는 강력한 도구입니다.
이 함수는 데이터가 정규 분포를 따르고, 그룹 간의 분산이 동일하다는 가정을 하며,
p-값을 통해 그룹 간 평균 차이가 유의미한지 여부를 결정합니다.
통계 분석과 데이터 과학에서, 특히 실험 데이터의 분석에서 널리 사용됩니다.
...

...

코드는 `df\_prob2` 데이터프레임을 'product_code'로 그룹화한 후, 각 그룹에서 'loading_log' 열의 값을 리스트로 모은 뒤,
이를 일원분산분석(One-way ANOVA)에 적용하여 그룹 간의 평균값의 차이를 검정합니다.
이때, 그룹 간의 평균값이 모두 같다는 귀무가설을 검정하고, 이에 대한 결과의 p-값을 변수 `D`에 저장합니다.

1. 그룹화 후 리스트로 값 모으기

- `groupby('product\_code')['loading\_log']`는 'product_code' 별로 'loading_log' 열을 그룹화합니다.
- `agg(list)`는 각 그룹에서 'loading_log' 열의 값을 리스트로 모으는데, `list` 함수를 적용하여 리스트로 변환합니다.
- `\*`는 그룹화된 데이터프레임을 풀어내어 `f\_oneway` 함수에 전달할 수 있도록 합니다.
이렇게 함으로써, `f\_oneway` 함수는 각 그룹의 리스트를 별도의 인수로 처리할 수 있습니다.

2. 일원분산분석 수행

- `f\_oneway` 함수는 일원분산분석(One-way ANOVA)을 수행하는 함수입니다.
이 함수는 주어진 데이터셋에서 각 그룹 간의 평균값의 차이를 검정합니다.
- `\*`를 사용하여 각 그룹의 리스트를 별도의 인수로 전달합니다.

3. p-값 저장

- `result\_2\_5.pvalue`는 일원분산분석 결과의 p-값을 나타냅니다. 이를 변수 `D`에 할당합니다.

결과

- `D`는 'product_code'별로 그룹화된 데이터에서 일원분산분석을 통해 얻은 평균값의 차이 검정의 결과인 p-값을 나타냅니다.
- p-값은 그룹 간의 평균값의 차이를 평가하는 데 사용되며, 일반적으로 유의수준 0.05 이하일 때 차이가 있다고 판단됩니다.

...

```
from scipy.stats import f_oneway
```

```
result_2_5 = f_oneway(*df_prob2.groupby('product_code')['loading_log'].agg(list))
```

```
D = result_2_5.pvalue
result_2_5, D
```

```
Out[44]: (F_onewayResult(statistic=0.37177284134760413, pvalue=0.7733782072320899),
0.7733782072320899)
```

Tip! 일원분산분석을 직접 해봅니다.

GroupBy에 대한 활용법을 더 알아보고,

ANOVA에 대한 이해를 공고히 해보기 위해 예를 들어 보입니다.

In [45]:

```
'''
아래 코드는 일원분산분석(One-way ANOVA)의 각 구성 요소를 계산합니다.
이를 통해 F-통계량을 계산하고, 그룹 간의 평균값의 차이가 통계적으로 유의한지를 검정할 수 있습니다.

### 1. 자유도(dof) 계산
- `dof1`은 그룹 내 변동의 자유도입니다. 이는 그룹의 수에서 1을 뺀 값입니다.
- `dof2`는 오차(또는 그룹 간 변동)의 자유도입니다. 이는 전체 데이터의 수에서 그룹의 수를 뺀 값입니다.

### 2. 그룹 내 변동(SSR) 및 오차(또는 그룹 간 변동)(SSE) 계산
- `s_prd`는 각 그룹의 평균값을 갖는 새로운 Series입니다.
  이는 `groupby`를 사용하여 'product_code' 그룹별로 'loading_log'의 평균값을 계산한 후, `transform('mean')`을 사용하여
  각 행에 해당 그룹의 평균값을 할당합니다.
- `SSR`은 그룹 내 변동(SSR, Sum of Squares for Regression)을 계산합니다.
  이는 각 데이터의 그룹 평균값에서 전체 데이터의 평균값을 뺀 후 제곱하여 합한 값입니다.
- `SSE`는 오차(또는 그룹 간 변동, SSE, Sum of Squares for Error)을 계산합니다.
  이는 각 데이터의 개별 값에서 해당하는 그룹 평균값을 뺀 후 제곱하여 합한 값입니다.

### 3. 총 변동(SST) 및 평균 제곱(편차) 계산
- `SST`는 전체 변동(SST, Sum of Squares Total)을 계산합니다.
  이는 각 데이터의 개별 값에서 전체 데이터의 평균값을 뺀 후 제곱하여 합한 값입니다.
- `MSR`은 그룹 내 변동의 평균 제곱값(Mean Square Regression)을 계산합니다.
  SSR을 그룹 내 변동의 자유도(dof1)로 나눈 값입니다.
- `MSE`는 오차(또는 그룹 간 변동)의 평균 제곱값(Mean Square Error)을 계산합니다.
  SSE를 오차(또는 그룹 간 변동)의 자유도(dof2)로 나눈 값입니다.

### 4. F-통계량 계산
- F-통계량은 그룹 내 변동의 평균 제곱값과 오차(또는 그룹 간 변동)의 평균 제곱값의 비율입니다.
  이를 통해 그룹 간의 평균값의 차이가 통계적으로 유의한지를 검정할 수 있습니다.
'''

from scipy.stats import f

# F = MSR / MSE, dof1, dof2
# MSR = SSR / dof1, MSE = SSE / dof2
# SSR = 각 표본별 (product_code 별 loading_log 평균 - loading_log 평균) ** 2 의 합
```

```
# SSE = (loading_log - product_code 별 loading_log 평균) ** 2 의 합
# N: 데이터의 수
# dof1 = k(범주의수) - 1
# dof2 = N - k

dof1 = df_prob2['product_code'].nunique() - 1
dof2 = len(df_prob2) - df_prob2['product_code'].nunique()
s_prd = df_prob2.groupby('product_code')['loading_log'].transform('mean')
SSR = ((s_prd - df_prob2['loading_log'].mean()) ** 2).sum()
SSE = ((df_prob2['loading_log'] - s_prd) ** 2).sum()
# 구할 필요는 없습니다. 계산용으로 구해봅니다. SST = SSR + SSE
SST = ((df_prob2['loading_log'] - df_prob2['loading_log'].mean()) ** 2).sum()
MSR = SSR / dof1
MSE = SSE / dof2
F = MSR / MSE
dof1, dof2, F, 1 - f.cdf(F, dof1, dof2), SST, SSR + SSE
```

```
Out[45]: (3,
          21253,
          0.3717728413460874,
          0.7733782072333035,
          1879.1520065602526,
          1879.1520065602524)
```

단계 2-6

카이제곱검정을 통해 attribute_0, attribute_1의 결합값이 failure와 연관이 있는지 조사하라.

attribute_0, attribute_1의 결합값의 의미 attribute_0=material_7, attribute_1=material_8 이라면,

이 둘의 결합값 은material_7material_8를 의미한다.

(유의 수준 1%) 연관이 있다면 E값은 1 없으면 0으로 한다.

함수 가이드

scipy.stats.chi2_contingency, correction=False 문제 지시사항 외 Default 값 사용

```
In [46]: '''
`chi2_contingency()` 함수는 SciPy 라이브러리의 `scipy.stats` 모듈에 포함되어 있으며, 카이제곱 독립성 검정을 수행하는 데 사용됩니다.
이 함수는 관찰된 빈도표와 기대되는 빈도표 사이의 차이를 평가하여 두 변수 사이의 독립성을 검정합니다.

### 함수 정의
scipy.stats.chi2_contingency(observed, correction=True, lambda_=None)

### 인자
- `observed`: 배열 형태의 관찰된 빈도표. 일반적으로 2차원 배열(행렬)로 제공됩니다.
- `correction`: (기본값 `True`) 이어즈 연속성 수정을 적용할지 여부.
    이는 2x2 표에서 카이제곱 검정 통계량을 수정하여 보다 보수적인 검정을 수행하도록 합니다.
- `lambda_`: (기본값 `None`) Box-Cox 변환을 사용하여 빈도를 변환하기 위한 파라미터. `None`일 경우 변환을 적용하지 않습니다.

### 리턴값
`chi2_contingency()` 함수는 다음의 네 가지 값을 반환하는 튜플입니다:
1. `chi2`: 카이제곱 검정 통계량.
2. `p`: p-값, 검정 통계량의 누적 분포 함수의 값.
3. `dof`: 자유도 (degrees of freedom).
4. `expected`: 기대 빈도표.

### 속성
- `chi2`: 카이제곱 검정 통계량.
- `p`: p-값, 검정 통계량의 누적 분포 함수의 값.
- `dof`: 자유도.
- `expected`: 기대 빈도표.

### 함수 설명
1. 관찰된 빈도표(observed)
    - 실험이나 조사에서 관찰된 데이터의 빈도를 나타내는 2차원 배열입니다.
    - 예: [[10, 20], [20, 30]]

2. 이어즈 연속성 수정(correction)
    - 카이제곱 검정 통계량을 보수적으로 수정하여 작은 표본에 대해 좀 더 정확한 결과를 제공합니다.
    - 2x2 표에서만 적용됩니다.

3. 기대 빈도표(expected)
    - 독립 가정 하에서 예상되는 빈도를 계산합니다.
    - 각 셀의 기대 빈도는 (행 합계 * 열 합계) / 전체 합계로 계산됩니다.

4. 자유도(dof)
    - 자유도는 (행 개수 - 1) * (열 개수 - 1)로 계산됩니다.
```

- 예를 들어, 2x2 표의 경우 자유도는 1입니다.

5. 카이제곱 검정 통계량(chi2)

- 관찰된 빈도와 기대 빈도의 차이를 나타냅니다.
- 통계량은 각 셀에서 (관찰된 빈도 - 기대 빈도)² / 기대 빈도의 합으로 계산됩니다.

6. p-값(p)

- 검정 통계량에 대응하는 p-값입니다.
- p-값이 작은 경우(일반적으로 0.05 이하) 독립 가설을 기각하고 두 변수 사이의 연관성이 있다고 판단합니다.

아래 예제에서는 `observed` 배열을 사용하여 카이제곱 독립성 검정을 수행합니다.

결과로 검정 통계량, p-값, 자유도, 기대 빈도표를 출력합니다.

```
'''
```

```
import numpy as np
from scipy.stats import chi2_contingency

# 관찰된 빈도표
observed = np.array([[10, 20, 30], [6, 9, 17]])

# 카이제곱 독립성 검정 수행
chi2, p, dof, expected = chi2_contingency(observed)

print(f"Chi2: {chi2}")
print(f"P-value: {p}")
print(f"Degrees of Freedom: {dof}")
print("Expected Frequencies:")
print(expected)
```

```
Chi2: 0.27157465150403504
P-value: 0.873028283380073
Degrees of Freedom: 2
Expected Frequencies:
[[10.43478261 18.91304348 30.65217391]
 [ 5.56521739 10.08695652 16.34782609]]
```

In [47]: '''

아래 코드는 카이제곱 검정(Chi-square test)을 사용하여 두 개의 범주형 변수 간의 독립성을 검정합니다.

1. 크로스테이블 생성

- `pd.crosstab` 함수를 사용하여 'attribute_0'과 'attribute_1' 열을 합쳐서 인덱스로, 'failure' 열을 칼럼으로 하는 크로스테이블(교차표)을 생성합니다.

- 이 크로스테이블은 'attribute_0'과 'attribute_1'의 각 값들을 결합한 것에 따른 'failure' 열의 빈도를 보여줍니다.

2. 카이제곱 검정 수행

- `chi2_contingency` 함수를 사용하여 카이제곱 검정을 수행합니다. 이 함수는 크로스테이블을 인수로 받습니다.
- `correction=False`는 Yates' continuity correction을 적용하지 않도록 설정합니다.

3. p-값을 통한 결과 도출

- 검정의 결과로 반환된 p-값(`result_2_6[1]`)이 유의수준 0.01보다 크거나 같으면 `E`를 0으로 설정합니다.
이는 귀무가설을 기각하지 않음을 의미합니다.
- 그렇지 않은 경우, 즉 p-값이 0.01보다 작으면 `E`를 1로 설정합니다.
이는 귀무가설을 기각하고 대립가설을 채택함을 의미합니다.

결과

- `E` 변수는 귀무가설을 기각하는지 여부에 따라 0 또는 1의 값을 가집니다.
- 따라서 변수 `E`를 통해 두 범주형 변수 간의 연관성 여부를 확인할 수 있습니다.

...

코드는 `chi2_contingency` 함수를 사용하여 두 범주형 변수 간의 독립성을 검정합니다.

이를 통해 변수 간의 상관관계를 분석할 수 있습니다.

이 코드는 특히 `df_prob2` 데이터프레임의 두 개의 열(`attribute_0`과 `attribute_1`)의 합과 `failure` 열 간의 연관성을 검정합니다.

1. 데이터프레임 열의 합 계산:

- ```
index = df_prob2['attribute_0'] + df_prob2['attribute_1']
```
- `df_prob2` 데이터프레임의 `attribute_0` 열과 `attribute_1` 열을 더하여 새로운 시리즈를 생성합니다. 이 시리즈는 교차표(crosstab)의 행 인덱스로 사용됩니다.

#### 2. 교차표 생성:

- ```
pd.crosstab(
    index = df_prob2['attribute_0'] + df_prob2['attribute_1'],
    columns = df_prob2['failure']
)
```
- `pd.crosstab` 함수를 사용하여 교차표를 만듭니다.
이 교차표는 `attribute_0`과 `attribute_1`의 합과 `failure` 변수 간의 빈도수를 나타냅니다.
 - `index`는 `attribute_0`과 `attribute_1`의 합으로 설정되고, `columns`는 `failure` 변수로 설정됩니다.
결과는 두 변수 간의 빈도수를 보여주는 교차표가 됩니다.

3. 카이제곱 검정:

- ```
chi2_contingency(
 pd.crosstab(
 index = df_prob2['attribute_0'] + df_prob2['attribute_1'],
 columns = df_prob2['failure']
), correction=False
)
```

```

- `chi2_contingency` 함수를 사용하여 교차표에 대해 카이제곱 독립성 검정을 수행합니다.
- `correction=False`는 연속성 수정 (Yates' correction for continuity)을 사용하지 않도록 설정합니다.
 이 수정은 2x2 테이블에서만 사용됩니다.
- `chi2_contingency` 함수는 교차표를 인자로 받아 카이제곱 값, p-값, 자유도, 기대 빈도 수를 계산합니다.
4. 결과 저장:
result_2_6 = chi2_contingency(...)
- `chi2_contingency` 함수의 결과를 `result_2_6` 변수에 저장합니다. 이 변수는 튜플로, 다음과 같은 값을 포함합니다:
 - `chi2`: 카이제곱 통계량
 - `p`: p-값
 - `dof`: 자유도
 - `expected`: 기대 빈도 수의 배열

결과 해석
`result_2_6`에 저장된 값을 해석하는 방법은 다음과 같습니다:

1. 카이제곱 통계량 (chi2):
 - 관찰된 빈도와 기대 빈도의 차이를 측정한 값입니다. 값이 클수록 두 변수 간의 연관성이 높다는 것을 의미합니다.
2. p-값 (p):
 - 두 변수 간의 독립성 가설을 검정합니다.
 - 일반적으로 p-값이 0.05보다 작으면 귀무가설 (두 변수는 독립적이다)을 기각하고, 두 변수 간의 연관성이 있다고 판단합니다.
3. 자유도 (dof):
 - 독립적으로 변할 수 있는 데이터의 수를 나타냅니다. 교차표의 크기에서 영향을 받습니다.
4. 기대 빈도 수 (expected):
 - 귀무가설 하에서 기대되는 각 셀의 빈도입니다. 관찰된 빈도와 비교하여 카이제곱 값을 계산합니다.
...
print('')

```

```
In [48]: df_prob2['attribute_0'].value_counts()
```

```
Out[48]: material_7 16048
material_5 5209
Name: attribute_0, dtype: int64
```

```
In [49]: df_prob2['attribute_1'].value_counts()
```

```
Out[49]: material_8 10767
material_6 5281
material_5 5209
Name: attribute_1, dtype: int64
```

```
In [50]: aa = df_prob2['attribute_0'] + df_prob2['attribute_1']
print(type(aa))
aa
```

```
<class 'pandas.core.series.Series'>
```

```
Out[50]: id
0 material_7material_8
1 material_7material_8
2 material_7material_8
3 material_7material_8
4 material_7material_8
...
26565 material_7material_6
26566 material_7material_6
26567 material_7material_6
26568 material_7material_6
26569 material_7material_6
Length: 21257, dtype: object
```

```
In [51]: aa.value_counts()
```

```
Out[51]: material_7material_8 10767
material_7material_6 5281
material_5material_5 5209
dtype: int64
```

```
In [52]: pd.crosstab(
 index = df_prob2['attribute_0'] + df_prob2['attribute_1'],
 columns = df_prob2['failure']
)
```

```
Out[52]:
```

|                      | failure | 0    | 1 |
|----------------------|---------|------|---|
| row_0                |         |      |   |
| material_5material_5 | 4162    | 1047 |   |
| material_7material_6 | 4181    | 1100 |   |
| material_7material_8 | 8413    | 2354 |   |

```
In [53]: from scipy.stats import chi2_contingency

result_2_6 = chi2_contingency(
 pd.crosstab(
 index = df_prob2['attribute_0'] + df_prob2['attribute_1'],
 columns = df_prob2['failure']
), correction=False
)

result_2_6
```

```
Out[53]: (7.0400463075335615,
0.029598749837407304,
2,
array([[4106.03584702, 1102.96415298],
 [4162.79042198, 1118.20957802],
 [8487.17373101, 2279.82626899]]))
```

```
In [54]: # 귀무 0: 연관 X
귀무 X: 연관 0
E = 0 if result_2_6[1] >= 0.01 else 1
E
```

```
Out[54]: 0
```

**Tip!**  $\chi^2$  검정도 직접해봅니다.

$$\chi^2 = \sum_{i,j} \frac{(E_{ij} - O_{ij})^2}{E_{ij}}$$

```
In [55]: '''
`dot()` 함수는 Python에서 주로 NumPy 배열이나 pandas 객체 간의 행렬 곱셈(또는 점곱)을 수행하는 데 사용됩니다.
함수의 구체적인 동작은 사용되는 객체의 유형에 따라 약간 다릅니다.
아래에서는 NumPy와 pandas에서 `dot()` 함수의 동작과 사용법을 설명합니다.
```

```
1. NumPy에서 `dot()`
NumPy의 `dot()` 함수는 두 배열의 점곱을 계산합니다.
이는 두 벡터의 내적, 벡터와 행렬의 곱, 그리고 행렬 간의 곱을 수행하는 데 사용됩니다.
```

```
인자 (Parameters)
- a: 첫 번째 입력 배열.
```

- **b**: 두 번째 입력 배열.
- **out**: (선택 사항) 결과를 저장할 출력 배열.
- **where**: (선택 사항) 조건이 참인 위치에만 값을 저장하는 배열.

### ## 리턴값 (Returns)

- 두 입력 배열의 점곱 결과를 반환합니다.

### ## 예제

```
import numpy as np
```

```
두 벡터의 내적
```

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
result = np.dot(a, b)
```

```
print(result) # 32
```

```
행렬 곱셈
```

```
A = np.array([[1, 2], [3, 4]])
```

```
B = np.array([[5, 6], [7, 8]])
```

```
result = np.dot(A, B)
```

```
print(result)
```

```
[[19 22]
```

```
[43 50]]
```

```
2. pandas에서 `dot()`
```

pandas에서는 `Series`와 `DataFrame` 객체에 `dot()` 메서드가 있으며, 이 메서드는 벡터 간, 벡터와 행렬 간, 또는 행렬 간의 곱을 계산하는 데 사용됩니다.

```
pandas `Series.dot()`
```

`Series` 객체의 `dot()` 메서드는 두 `Series` 객체 간의 내적을 계산합니다.

```
인자 (Parameters)
```

- **other**: 곱할 다른 `Series` 객체 또는 배열과 같은 1차원 데이터 구조.

```
리턴값 (Returns)
```

- 스칼라 값으로 내적 결과를 반환합니다.

```
예제
```

```
import pandas as pd
```

```
s1 = pd.Series([1, 2, 3])
```

```
s2 = pd.Series([4, 5, 6])
```

```

result = s1.dot(s2)
print(result) # 32

3. pandas `DataFrame.dot()`
`DataFrame` 객체의 `dot()` 메서드는 데이터프레임과 `Series` 객체 간 또는 두 데이터프레임 간의 행렬 곱을 계산합니다.

인자 (Parameters)
- other: 곱할 다른 `DataFrame` 객체 또는 `Series` 객체.

리턴값 (Returns)
- 행렬 곱의 결과로 `DataFrame` 또는 `Series`를 반환합니다.

예제
import pandas as pd

df = pd.DataFrame([[1, 2], [3, 4]])
s = pd.Series([5, 6])
result = df.dot(s)
print(result)
0 17
1 39
dtype: int64

두 데이터프레임 간의 행렬 곱
df2 = pd.DataFrame([[5, 6], [7, 8]])
result = df.dot(df2)
print(result)
0 1
0 19 22
1 43 50

요약
- 1. NumPy `dot()` : 주로 벡터와 행렬 간의 곱셈을 수행합니다. 인자로 두 배열을 받고, 두 배열의 점곱 결과를 반환합니다.
- 2. pandas `Series.dot()` : 두 `Series` 객체 간의 내적을 계산합니다.
 인자로 다른 `Series` 객체를 받고, 내적 결과를 스칼라 값으로 반환합니다.
- 3. pandas `DataFrame.dot()` : `DataFrame`과 `Series` 객체 간 또는 두 `DataFrame` 객체 간의 행렬 곱을 계산합니다.
 인자로 다른 `DataFrame` 또는 `Series` 객체를 받고, 결과를 `DataFrame` 또는 `Series`로 반환합니다.

각 라이브러리에서 `dot()` 함수는 벡터와 행렬 간의 곱셈을 편리하게 수행할 수 있도록 도와줍니다.
'''
print('')

```

In [56]:

```
'''
코드는 두 개의 범주형 변수(`attribute_0`과 `attribute_1`)를 결합한 값과 `failure` 변수 간의 관계를 분석하는 과정에서,
관찰된 빈도와 예상 빈도를 계산하는 작업을 수행합니다.

1. 두 범주형 변수를 결합한 값 생성
- `attribute_0`과 `attribute_1` 열을 결합하여 `s_attr_01`라는 Series를 생성합니다.
이 Series는 각 행에서 `attribute_0`과 `attribute_1` 값을 결합한 문자열을 포함합니다.

2. 예상 빈도수 계산
- `s_attr_01.value_counts(normalize=True)`는 `s_attr_01`의 각 고유 값의 상대 빈도를 계산합니다.
 `normalize=True`는 상대 빈도를 계산하기 위한 옵션입니다.
- `.rename('val').to_frame()`는 이 상대 빈도를 DataFrame으로 변환하고, 열 이름을 'val'로 설정합니다.
- `df_prob2['failure'].value_counts().rename('val').to_frame().T`는 `failure` 열의 각 고유 값의 빈도를 계산하여
 DataFrame으로 변환하고, 전치(transpose)하여 행을 열로, 열을 행으로 변환합니다.
- `dot` 함수는 두 DataFrame 간의 행렬 곱을 수행합니다. 이 행렬 곱을 통해 예상 빈도수를 계산합니다.

3. 관찰된 빈도수 계산
- `pd.crosstab` 함수는 `attribute_0`과 `attribute_1`을 결합한 값과 `failure` 변수 간의 교차표(crosstab)를 생성합니다.
- 이 교차표는 각 결합된 값과 `failure` 값에 대한 관찰된 빈도수를 보여줍니다.

결과
- `df_expected`는 각 결합된 값(`s_attr_01`)과 `failure` 변수 간의 예상 빈도수를 나타내는 DataFrame입니다.
- `df_obs`는 관찰된 빈도수를 나타내는 교차표입니다.

이 과정은 카이제곱 검정에서 관찰된 빈도와 예상 빈도를 비교하여 두 범주형 변수 간의 독립성을 검정하는 데 유용합니다.
'''

$n * p_{ij} = n * p_i * p_j$

s_attr_01 = df_prob2['attribute_0'] + df_prob2['attribute_1']
print(s_attr_01)
print('-----')

print(s_attr_01.value_counts(normalize=True).rename('val').to_frame())
print('-----')

print(df_prob2['failure'].value_counts().rename('val').to_frame().T)
print('-----')

행렬곱을 이용하면 예상 빈도수를 비교적 간단히 구할 수 있습니다.
```

```
df_expected = s_attr_01.value_counts(normalize=True).rename('val').to_frame()\
 .dot(df_prob2['failure'].value_counts().rename('val').to_frame().T)
```

df\_expected

```
id
0 material_7material_8
1 material_7material_8
2 material_7material_8
3 material_7material_8
4 material_7material_8
```

...

```
26565 material_7material_6
26566 material_7material_6
26567 material_7material_6
26568 material_7material_6
26569 material_7material_6
```

Length: 21257, dtype: object

```

 val
material_7material_8 0.506516
material_7material_6 0.248436
material_5material_5 0.245049

```

```

 0 1
val 16756 4501

```

Out[56]:

```

 0 1
material_7material_8 8487.173731 2279.826269
material_7material_6 4162.790422 1118.209578
material_5material_5 4106.035847 1102.964153
```

In [57]:

```
df_obs = pd.crosstab(
 index = df_prob2['attribute_0'] + df_prob2['attribute_1'],
 columns = df_prob2['failure']
)
```

df\_obs



Out[57]:

|                      | failure | 0    | 1 |
|----------------------|---------|------|---|
| row_0                |         |      |   |
| material_5material_5 | 4162    | 1047 |   |
| material_7material_6 | 4181    | 1100 |   |
| material_7material_8 | 8413    | 2354 |   |

In [58]:

```
'''
이 코드는 카이제곱 검정(Chi-square test)의 통계량을 수동으로 계산하고, 자유도(degrees of freedom) 및 p-값을 구합니다.

1. 카이제곱 통계량(Chi) 계산
- `df_expected`와 `df_obs`의 차이를 제공합니다: `(df_expected - df_obs) ** 2`
- 이를 예상 빈도로 나눕니다: `((df_expected - df_obs) ** 2 / df_expected)`
- 위 결과를 합산하여 각 셀의 기여도를 모두 더합니다: `.sum().sum()`
- 최종적으로 카이제곱 통계량 `Chi`를 얻습니다.

2. 자유도(dof) 계산
- `s_attr_01.nunique()`는 결합된 `attribute_0`과 `attribute_1` 값의 고유한 수를 반환합니다.
- `df_prob2['failure'].nunique()`는 `failure` 변수의 고유한 수를 반환합니다.
- 자유도는 (행의 수 - 1) * (열의 수 - 1)으로 계산됩니다.

3. p-값 계산
- `chi2.cdf` 함수는 누적 분포 함수(Cumulative Distribution Function)로, 카이제곱 분포에서 주어진 카이제곱 통계량 `Chi`와
 자유도 `dof`를 사용하여 누적 확률을 계산합니다.
- `1 - chi2.cdf(Chi, dof)`는 카이제곱 검정의 p-값을 계산합니다.
 이는 카이제곱 통계량이 주어진 자유도에서 얼마나 극단적인지를 나타내는 확률입니다.

결과
- `Chi`는 카이제곱 통계량입니다.
- `dof`는 자유도입니다.
- `1 - chi2.cdf(Chi, dof)`는 p-값입니다.

이 코드의 전체적인 동작은 다음과 같습니다:
1. 관찰된 빈도와 예상 빈도 간의 차이를 기반으로 카이제곱 통계량 `Chi`를 계산합니다.
2. 결합된 `attribute_0`과 `attribute_1` 값 및 `failure` 값의 고유한 수를 사용하여 자유도 `dof`를 계산합니다.
3. 카이제곱 분포에서의 p-값을 계산하여, 귀무가설(두 변수는 독립이다)을 기각할 수 있는지를 평가합니다.

결과적으로, 반환된 값 `Chi`, `dof`, `p-값`을 통해 두 변수 간의 독립성을 검정할 수 있습니다.
p-값이 작을수록(일반적으로 0.05보다 작을 때) 두 변수 간의 독립성을 기각하고, 연관성이 있다고 판단합니다.
'''
```

```
'''

from scipy.stats import chi2

Chi = ((df_expected - df_obs) ** 2 / df_expected).sum().sum()
dof = (s_attr_01.nunique() - 1) * (df_prob2['failure'].nunique() - 1)
Chi, dof, 1 - chi2.cdf(Chi, dof)
```

Out[58]: (7.040046307533586, 2, 0.029598749837406957)

In [59]: B, C, D, E, B + C + D + E, round(B + C + D + E, 2)

Out[59]: (3, 0.5873433093297675, 0.7733782072320899, 0, 4.360721516561858, 4.36)

B + C + D + E의 값을 소수점 셋째 자리에서 반올림하여 둘째 자리까지 출력하시오.

## 문제 3

로지스틱 회귀모델로 수치형 변수 measurement\_0 ~ 17,

loading과 이진형인 na\_1, na\_2 중에서 최적의 성능을 보이는 입력 변수들을 찾고자 한다.

In [ ]:

## 단계 3-1

prob1을 복사하여 prob3을 만든다. loading의 결측치는 loading의 평균으로 대체한다.

In [60]: '''

이 코드는 `df\_prob1` 데이터프레임의 `loading` 열에 결측값이 있는 경우, 해당 결측값을 `loading` 열의 평균값으로 대체하는 작업을 수행  
이를 통해 결측값을 처리한 새로운 데이터프레임 `df\_prob3`을 생성합니다.

### 1. `assign` 메서드 사용

- `assign` 메서드는 기존 데이터프레임에 새로운 열을 추가하거나 기존 열을 수정하여 새로운 데이터프레임을 반환합니다.

- `lambda x: x['loading'].fillna(x['loading'].mean())`는 `loading` 열의 결측값을 해당 열의 평균값으로 대체하는 람다 함수입니다.

### 2. `fillna` 메서드 사용

- `x['loading']`는 `df\_prob1` 데이터프레임의 `loading` 열을 선택합니다.
- `fillna(x['loading'].mean())`는 `loading` 열에서 결측값을 `loading` 열의 평균값으로 대체합니다.

### 동작 과정

1. `df\_prob1` 데이터프레임에서 `loading` 열을 선택합니다.
2. `loading` 열에서 결측값을 `loading` 열의 평균값으로 대체합니다.
3. `assign` 메서드를 사용하여 수정된 `loading` 열을 `df\_prob3` 데이터프레임에 추가합니다.
4. 결과적으로, `loading` 열의 결측값이 모두 평균값으로 대체된 `df\_prob3` 데이터프레임이 생성됩니다.

이 코드는 결측값을 평균값으로 대체하여 데이터의 완전성을 유지하고, 후속 분석에서 결측값으로 인한 문제를 방지하는 데 유용합니다.

...

코드는 `df\_prob1` 데이터프레임에 새로운 열을 추가하거나 기존 열을 수정하는 과정에서 사용됩니다. 이 과정에서 `lambda` 함수가 사용되며 구체적으로, `lambda` 함수에 전달되는 `x`는 `df\_prob1` 데이터프레임입니다.

### 코드 설명

1. `assign` 메서드:

- `assign` 메서드는 데이터프레임에 새로운 열을 추가하거나 기존 열을 수정합니다.
- 이 메서드는 키워드 인자로 열 이름과 값을 지정합니다.
- 값을 지정할 때는 직접 값을 주거나, 함수를 사용하여 열을 생성할 수 있습니다.

2. `lambda` 함수:

- `lambda x: x['loading'].fillna(x['loading'].mean())` 부분은 익명 함수 (lambda 함수)를 정의합니다.
- `x`는 `df\_prob1` 데이터프레임을 나타냅니다.

3. 동작 과정:

- `assign` 메서드가 호출될 때, `df\_prob1` 데이터프레임이 `lambda` 함수의 인자인 `x`로 전달됩니다.
- `lambda` 함수는 `df\_prob1`의 `loading` 열에서 결측값을 해당 열의 평균값으로 채웁니다.
- `fillna` 메서드는 결측값을 채우는 데 사용되며, `mean` 메서드는 `loading` 열의 평균을 계산합니다.

### `lambda` 함수의 인자 `x`

`lambda x: x['loading'].fillna(x['loading'].mean())`에서 `x`는 `df\_prob1` 데이터프레임입니다.

이 함수는 다음과 같은 방식으로 동작합니다:

1. `x['loading']`: `df\_prob1` 데이터프레임의 `loading` 열을 선택합니다.
2. `x['loading'].mean()`: `loading` 열의 평균값을 계산합니다.
3. `x['loading'].fillna(x['loading'].mean())`: `loading` 열의 결측값을 해당 열의 평균값으로 채웁니다.

### 전체 코드의 결과

이 코드는 `df\_prob1` 데이터프레임에서 `loading` 열의 결측값을 해당 열의 평균값으로 채운 후, 이를 포함하는 새로운 데이터프레임 `df\_prob3`을 생성합니다. `df\_prob1` 자체는 변경되지 않으며, 변경된 내용이 반영된 새로운 데이터프레임 `df\_prob3`가 생성됩니다.

```
...

print(df_prob1['loading'].mean())

df_prob1[df_prob1['loading'].isna() == True]
```

127.68089711624407

Out[60]:

|       | product_code | loading | attribute_0 | attribute_1 | attribute_2 | attribute_3 | measurement_0 | measurement_1 | measurement_2 | measurement_3 | ... | isr |
|-------|--------------|---------|-------------|-------------|-------------|-------------|---------------|---------------|---------------|---------------|-----|-----|
| id    |              |         |             |             |             |             |               |               |               |               |     |     |
| 156   | A            | NaN     | material_7  | material_8  | 9           | 5           | 14            | 4             | 6             | 18.550000     | ... |     |
| 159   | A            | NaN     | material_7  | material_8  | 9           | 5           | 12            | 5             | 3             | 17.301000     | ... |     |
| 187   | A            | NaN     | material_7  | material_8  | 9           | 5           | 10            | 4             | 7             | 16.030000     | ... |     |
| 220   | A            | NaN     | material_7  | material_8  | 9           | 5           | 9             | 7             | 6             | 18.159000     | ... |     |
| 271   | A            | NaN     | material_7  | material_8  | 9           | 5           | 14            | 1             | 3             | 17.816052     | ... |     |
| ...   | ...          | ...     | ...         | ...         | ...         | ...         | ...           | ...           | ...           | ...           | ... | ... |
| 26071 | E            | NaN     | material_7  | material_6  | 6           | 9           | 4             | 12            | 4             | 17.784127     | ... |     |
| 26100 | E            | NaN     | material_7  | material_6  | 6           | 9           | 1             | 10            | 11            | 17.325000     | ... |     |
| 26108 | E            | NaN     | material_7  | material_6  | 6           | 9           | 5             | 14            | 7             | 17.270000     | ... |     |
| 26138 | E            | NaN     | material_7  | material_6  | 6           | 9           | 7             | 7             | 4             | 17.087000     | ... |     |
| 26391 | E            | NaN     | material_7  | material_6  | 6           | 9           | 7             | 13            | 4             | 16.730000     | ... |     |

201 rows × 42 columns

```
In [61]: df_prob3 = df_prob1.assign(
 loading = lambda x: x['loading'].fillna(x['loading'].mean())
)

print(df_prob3.loc[156, 'loading'])
print(df_prob3.loc[157, 'loading'])
print(df_prob3.loc[159, 'loading'])
```

```
127.68089711624407
167.58
127.68089711624407
```

```
In [62]: df_prob3.columns
```

```
Out[62]: Index(['product_code', 'loading', 'attribute_0', 'attribute_1', 'attribute_2',
 'attribute_3', 'measurement_0', 'measurement_1', 'measurement_2',
 'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6',
 'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10',
 'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14',
 'measurement_15', 'measurement_16', 'measurement_17', 'failure',
 'isna_3', 'isna_4', 'isna_5', 'isna_6', 'isna_7', 'isna_8', 'isna_9',
 'isna_10', 'isna_11', 'isna_12', 'isna_13', 'isna_14', 'isna_15',
 'isna_16', 'isna_17', 'na_1', 'na_2'],
 dtype='object')
```

## 단계 3-2:

prob3를 80%는 학습데이터 prob3\_train으로 20%는 테스트데이터 prob3\_test로 나눈다.

prob3\_train의 failure가 1인 비율과 prob3\_test의 failure가 1의 비율을 동일하게 한다.

### 함수 가이드

sklearn.model\_selection.train\_test\_split, random\_state=123,

train과 test의 failure의 비율은 stratify 매개 변수를 이용하여 맞춘다.

문제 지시사항 외 Default 값 사용

```
In [63]: from sklearn.model_selection import train_test_split
```

```
train과 test의 failure의 비율은 stratify 매개 변수를 이용하여 맞춥니다.
```

```
df_prob3_train, df_prob3_test = train_test_split(df_prob3, train_size=0.8, random_state=123, stratify=df_prob3['failure'])
```

```
df_prob3_train, df_prob3_test = df_prob3_train.copy(), df_prob3_test.copy()
```

```
In [64]: '''
이 코드는 `df_prob3_train`과 `df_prob3_test` 데이터프레임에서 `failure` 열의 클래스 분포를 계산하여,
이를 하나의 데이터프레임으로 결합하는 작업을 수행합니다.
이로써 두 데이터프레임 간의 `failure` 클래스 비율을 비교할 수 있습니다.

1. `value_counts`를 사용하여 클래스 분포 계산
- `df_prob3_train['failure'].value_counts(normalize=True)`는 `df_prob3_train` 데이터프레임의 `failure` 열에서
 각 고유 값의 빈도수를 계산하고, 이를 전체 빈도로 나눠서 비율로 변환합니다.
- `rename("train")`는 결과 Series의 이름을 "train"으로 변경합니다.

- 동일한 방법으로 `df_prob3_test` 데이터프레임의 `failure` 열에서
 각 고유 값의 빈도수를 비율로 변환하고, Series의 이름을 "test"로 변경합니다.

2. `pd.concat`를 사용하여 데이터프레임 결합
- `pd.concat` 함수는 여러 데이터프레임(또는 Series)을 결합합니다.
- `[df_prob3_train['failure'].value_counts(normalize=True).rename("train"),
 df_prob3_test['failure'].value_counts(normalize=True).rename("test")]`는 결합할 두 Series의 리스트입니다.
- `axis=1`은 열 단위로 결합할 것을 지정합니다. 즉, 두 Series를 열로 결합하여 하나의 데이터프레임을 만듭니다.

동작 과정
1. `df_prob3_train`과 `df_prob3_test` 데이터프레임의 `failure` 열에서 각 클래스의 비율을 계산합니다.
2. 이 비율을 각각 "train"과 "test"라는 이름의 Series로 변환합니다.
3. `pd.concat`를 사용하여 이 두 Series를 열 단위로 결합하여 하나의 데이터프레임을 만듭니다.

결과
- 반환된 데이터프레임은 `failure` 클래스의 각 값에 대한 "train"과 "test" 데이터프레임의 비율을 열로 포함합니다.
- 이를 통해 `df_prob3_train`과 `df_prob3_test` 데이터프레임에서 `failure` 클래스의 분포를 비교할 수 있습니다.
'''

pd.concat([
 df_prob3_train['failure'].value_counts(normalize=True).rename("train"),
 df_prob3_test['failure'].value_counts(normalize=True).rename("test")
], axis=1)
```

```
Out[64]:
```

|   | train    | test     |
|---|----------|----------|
| 0 | 0.788535 | 0.788677 |
| 1 | 0.211465 | 0.211323 |

## 단계 3-3

prob3\_train의 수치형 입력 변수 loading, measurement\_0 ~ 17을 표준화한다.

prob3\_train의 표준화 설정으로 prob3\_test의 loading, measurement\_0 ~ 17에도 적용한다.

표준화 처리한 prob3\_train과 prob3\_test는 문제 4와 문제 5에서 사용한다.

### 함수 가이드

sklearn.preprocessing 제공 기능 활용,

문제 지시사항 외 Default 값 사용

In [65]:

```
'''
이 코드는 `StandardScaler`를 사용하여 `df_prob3_train`과 `df_prob3_test`
데이터프레임의 특정 열들을 표준화(정규화)하는 작업을 수행합니다.
표준화는 데이터의 각 특징(열) 값을 평균이 0, 분산이 1이 되도록 변환하는 과정입니다.
이를 통해 데이터의 스케일을 맞춰 머신러닝 모델의 성능을 향상시킬 수 있습니다.

1. 필요한 열 리스트 생성
- 표준화할 열의 이름을 리스트 `X_std`에 저장합니다.
- `'loading'` 열과 `'measurement_0'`부터 `'measurement_17'`까지의 열 이름을 포함합니다.

2. `StandardScaler` 객체 생성
- `StandardScaler` 객체를 생성합니다. 이 객체는 데이터를 표준화하는 데 사용됩니다.

3. `fit_transform` 메서드로 학습 데이터 표준화
- `fit_transform` 메서드를 사용하여 `df_prob3_train` 데이터프레임의 `X_std` 열들을 표준화합니다.
 - `fit` 메서드는 데이터를 기반으로 각 열의 평균과 표준편차를 계산합니다.
 - `transform` 메서드는 각 열의 값을 (값 - 평균) / 표준편차로 변환합니다.
- 변환된 값을 `df_prob3_train`의 동일한 열에 저장합니다.

4. `transform` 메서드로 테스트 데이터 표준화
- `transform` 메서드를 사용하여 `df_prob3_test` 데이터프레임의 `X_std` 열들을 표준화합니다.
 - `fit_transform`과 달리 `transform`은 이미 계산된 평균과 표준편차를 사용하여 데이터를 변환합니다.
 이는 학습 데이터의 통계를 사용하여 테스트 데이터를 변환하는 것을 의미합니다.
```

- 변환된 값을 `df\_prob3\_test`의 동일한 열에 저장합니다.

### 동작 과정

1. `X\_std` 리스트에 표준화할 열의 이름을 저장합니다.

2. `StandardScaler` 객체를 생성합니다.

3. `fit\_transform` 메서드를 사용하여 `df\_prob3\_train` 데이터프레임의 열들을 표준화하고, 변환된 값을 저장합니다.

4. `transform` 메서드를 사용하여 `df\_prob3\_test` 데이터프레임의 열들을 동일한 방법으로 표준화하고, 변환된 값을 저장합니다.

### 결과

- `df\_prob3\_train`과 `df\_prob3\_test`의 `X\_std` 열들이 표준화되어, 각 열의 값들이 평균이 0, 분산이 1이 되도록 변환됩니다.

- 이를 통해 모델 학습 시 데이터 스케일 차이로 인한 영향을 줄이고, 모델 성능을 향상시킬 수 있습니다.

...

```
from sklearn.preprocessing import StandardScaler
```

```
std_scaler = StandardScaler()
```

```
X_std = ['loading'] + ['measurement_{}'.format(i) for i in range(18)]
```

```
df_prob3_train[X_std] = std_scaler.fit_transform(df_prob3_train[X_std])
```

```
df_prob3_test[X_std] = std_scaler.transform(df_prob3_test[X_std])
```

## 단계 3-4

로지스틱 회귀모델을 사용하여 loading, measurement\_0~17과 na\_1, na\_2를 입력 변수로 하여

prob3\_train을 학습한다.

로지스틱 회귀모델을 prob3\_test로 성능을 측정한 값을 A라고 한다.

입력 변수: loading, measurement\_0~17, na\_1, na\_2

대상 변수: failure

성능 지표: AUC(area under of ROC curve)

---

### 함수 가이드



`sklearn.linear_model.LogisticRegression`, `solver='lbfgs'`, 문제 지시사항 외 Default 값 사용

`sklearn.metrics.roc_auc_score`

In [66]:

```
'''
`roc_auc_score()` 함수는 ROC 곡선(Receiver Operating Characteristic Curve)의 AUC(Area Under the Curve)를 계산하는 데 사용됩니다.
AUC는 분류 모델의 성능을 평가하는 데 중요한 지표로, 모델이 양성 샘플과 음성 샘플을 얼마나 잘 구분하는지를 나타냅니다.

함수 정의
sklearn.metrics.roc_auc_score(y_true, y_score, *, average='macro', sample_weight=None,
 max_fpr=None, multi_class='raise', labels=None)

인자
1. y_true:
 - 실제 클래스 레이블을 나타내는 1차원 배열이나 리스트. 이진 또는 다중 클래스 레이블을 가질 수 있습니다.

2. y_score:
 - 예측 점수나 확률을 나타내는 배열. 이진 분류의 경우 1차원 배열이고,
 다중 클래스 분류의 경우 클래스별 점수를 나타내는 2차원 배열입니다.

3. average (기본값: `'macro'`):
 - 다중 클래스 분류에서 사용되는 평균 방법을 지정합니다.
 - `'macro'`: 클래스별 AUC의 단순 평균.
 - `'weighted'`: 클래스별 AUC의 가중 평균.
 - `'micro'`: 전체 예측을 하나로 합쳐서 AUC 계산.
 - `'samples'`: 각 샘플에 대해 AUC 계산 후 평균.

4. sample_weight (기본값: `None`):
 - 각 샘플의 가중치를 나타내는 배열. None이면 모든 샘플의 가중치는 동일하게 간주됩니다.

5. max_fpr (기본값: `None`):
 - 최대 허용 거짓 양성 비율(FPR)입니다. 이 값을 설정하면 해당 FPR까지의 AUC를 계산합니다.

6. multi_class (기본값: `'raise'`):
 - 다중 클래스 분류의 경우 사용되는 접근 방식을 지정합니다.
 - `'raise'`: 오류를 발생시킵니다.
 - `'ovr'`: One-vs-Rest 접근 방식.
 - `'ovo'`: One-vs-One 접근 방식.

7. labels (기본값: `None`):
```

- 다중 클래스 분류의 경우 클래스 레이블을 지정합니다. `None`이면 ``y_true``에서 고유 클래스 레이블이 자동으로 결정됩니다.

### 리턴값

- `roc_auc`:

- ROC AUC 점수를 나타내는 단일 값. 이 값은 모델의 분류 성능을 나타내며, **0.5**는 무작위 추측을 의미하고, **1.0**은 완벽한 분류를 의미함

### 예제

다음은 ``roc_auc_score()`` 함수를 사용하여 ROC AUC 점수를 계산하는 예제입니다:

```
from sklearn.metrics import roc_auc_score
```

# 예제 데이터 생성

```
y_true = [0, 0, 1, 1] # 실제 클래스 레이블
```

```
y_scores = [0.1, 0.4, 0.35, 0.8] # 예측 확률
```

# ROC AUC 점수 계산

```
auc = roc_auc_score(y_true, y_scores)
```

```
print(f"ROC AUC Score: {auc}")
```

위 예제에서, 실제 클래스 레이블과 예측 확률을 입력으로 받아

ROC AUC 점수를 계산합니다. 결과적으로 모델의 분류 성능을 평가할 수 있습니다.

### 다중 클래스 예제

다음은 다중 클래스 분류에서 ``roc_auc_score()`` 함수를 사용하는 예제입니다:

```
from sklearn.metrics import roc_auc_score
```

```
import numpy as np
```

# 예제 데이터 생성

```
y_true = np.array([0, 1, 2, 2, 1]) # 실제 클래스 레이블
```

```
y_scores = np.array([
 [0.9, 0.05, 0.05], # 클래스별 예측 확률
 [0.1, 0.8, 0.1],
 [0.2, 0.2, 0.6],
 [0.3, 0.4, 0.3],
 [0.05, 0.9, 0.05]
])
```

# 다중 클래스 ROC AUC 점수 계산 (One-vs-Rest 방식)

```
auc = roc_auc_score(y_true, y_scores, multi_class='ovr')
```

```
print(f"Multi-class ROC AUC Score: {auc}")
```

위 예제에서, 실제 클래스 레이블과 클래스별 예측 확률을 입력으로 받아 다중 클래스 분류의 ROC AUC 점수를 계산합니다. ``multi_class='ovr'``를 사용하여 One-vs-Rest 방식을 지정합니다.

### 결론

``roc_auc_score()`` 함수는 분류 모델의 성능을 평가하는 중요한 지표인 ROC AUC 점수를 계산하는 데 사용됩니다. 이 함수는 이진 분류와 다중 클래스 분류 모두에 사용할 수 있으며, 다양한 평균 방식과 가중치 옵션을 제공합니다. 모델의 예측 성능을 평가하고 비교하는 데 매우 유용한 도구입니다.

...

코드는 ``LogisticRegression`` 모델을 사용하여 ``df_prob3_train`` 데이터셋을 학습시키고, 테스트 및 학습 데이터셋에서의 성능을 ROC AUC 점수로 평가합니다.

### 1. 학습에 사용할 피처 리스트 생성

- ``X_lr`` 리스트에 로지스틱 회귀 모델에서 사용할 피처(특징) 열의 이름을 저장합니다.
- ``'loading'`` 열, ``'measurement_0'``부터 ``'measurement_17'``까지의 열, 그리고 ``'na_1'``, ``'na_2'`` 열을 포함합니다.

### 2. 로지스틱 회귀 모델 객체 생성

- ``LogisticRegression`` 객체를 생성합니다.
- ``solver='lbfgs'``는 최적화 알고리즘으로 ``lbfgs``를 사용합니다. 이는 작은 데이터셋과 다수의 클래스에 적합한 알고리즘입니다.

### 3. 모델 학습

- ``fit`` 메서드를 사용하여 ``df_prob3_train`` 데이터프레임의 ``X_lr`` 열들을 피처로, ``'failure'`` 열을 타겟으로 설정하여 로지스틱 회귀 모델을 학습시킵니다.

### 4. ROC AUC 점수 계산

- ``roc_auc_score`` 함수는 실제 값과 예측 확률을 사용하여 ROC AUC 점수를 계산합니다.
- ``clf_lr.predict_proba(df_prob3_test[X_lr])[:, 1]``는 테스트 데이터셋에서 각 샘플이 Positive 클래스(1일 확률)를 예측합니다. ``[:, 1]``은 Positive 클래스의 확률을 선택하는 것을 의미합니다.
- ``df_prob3_test['failure']``는 테스트 데이터셋의 실제 ``failure`` 값입니다.
- 결과적으로 ``A``에는 테스트 데이터셋의 ROC AUC 점수가 저장됩니다.

### 5. 학습 데이터셋의 ROC AUC 점수 계산

- ``roc_auc_score(df_prob3_train['failure'], clf_lr.predict_proba(df_prob3_train[X_lr])[:, 1])``는 학습 데이터셋에서의 ROC AUC 점수를 계산합니다.
- ``A``와 함께 튜플로 반환하여 학습 데이터셋과 테스트 데이터셋의 ROC AUC 점수를 비교할 수 있습니다.

### 동작 과정

1. ``X_lr`` 리스트에 사용할 피처 열의 이름을 저장합니다.
2. ``LogisticRegression`` 객체를 생성하고, ``solver``로 ``lbfgs``를 지정합니다.

3. `df\_prob3\_train` 데이터프레임의 `X\_lr` 피처와 ``failure`` 타겟을 사용하여 모델을 학습시킵니다.
4. 테스트 데이터셋에서 각 샘플이 **Positive** 클래스일 확률을 예측하고, 이를 사용하여 ROC AUC 점수를 계산합니다.
5. 학습 데이터셋에서의 ROC AUC 점수도 계산하여 튜플로 반환합니다.

### 결과

- `A`: 테스트 데이터셋의 ROC AUC 점수
- 학습 데이터셋의 ROC AUC 점수

이 결과를 통해 모델의 성능을 평가할 수 있으며, 특히 테스트 데이터셋에서의 ROC AUC 점수를 통해 모델이 새로운 데이터에 대해 얼마나 잘 일반화되는지 확인할 수 있습니다.

```
'''
```

```
from sklearn.linear_model import LogisticRegression
```

```
roc_auc는 순위기반의 지표입니다. positive의 정도를 나타내는 수치(예측 결과)를 넘겨줍니다.
```

```
from sklearn.metrics import roc_auc_score
```

```
X_lr = ['loading'] + ['measurement_{}'.format(i) for i in range(18)] + ['na_1', 'na_2']
```

```
print('X_lr = ', X_lr)
```

```
clf_lr = LogisticRegression(solver='lbfgs')
```

```
clf_lr.fit(df_prob3_train[X_lr], df_prob3_train['failure'])
```

```
Positive 정도를 나타내는 수치로 Positive Class일 확률을 넘깁니다.
```

```
A = roc_auc_score(df_prob3_test['failure'],
 clf_lr.predict_proba(df_prob3_test[X_lr])[:, 1])
```

```
(
```

```
 A,
```

```
 roc_auc_score(df_prob3_train['failure'],
 clf_lr.predict_proba(df_prob3_train[X_lr])[:, 1])
```

```
)
```

```
X_lr = ['loading', 'measurement_0', 'measurement_1', 'measurement_2', 'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6', 'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10', 'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14', 'measurement_15', 'measurement_16', 'measurement_17', 'na_1', 'na_2']
```

```
Out[66]: (0.5792951262053387, 0.5952982435574312)
```

```
In [67]: '''
```

```
`clf_lr.decision_function(df_prob3_train[X_lr])` 코드는
```

```
`LogisticRegression` 모델의 `decision_function` 메서드를 사용하여 학습 데이터셋에 대한 결정 함수 값을 계산합니다.
```

```
결정 함수 값은 각 샘플에 대해 모델이 계산한 선형 결합 값으로,
```

로지스틱 회귀 모델의 경우 이는 로지스틱 함수에 전달되기 전에 계산되는 값입니다.

### `decision\_function` 메서드의 역할

`decision\_function` 메서드는 각 샘플에 대해 로지스틱 회귀 모델의 선형 결정 함수 값을 반환합니다. 이 값은 클래스 결정의 경계와 얼마나 떨어져 있는지를 나타냅니다. 선형 회귀 모델의 예측 값처럼, 이 값은 모델의 예측 확률을 결정하는 데 사용됩니다.

### 동작 과정

1. 선형 결합 계산: 로지스틱 회귀 모델은 입력 피처의 가중치(**weight**)와 편향(**bias**)을 학습합니다. 각 샘플에 대해 다음과 같이 선형 결합을 계산합니다:  

$$\text{linear combination} = \mathbf{XW} + \mathbf{b}$$
여기서  $\mathbf{X}$ 는 피처 벡터,  $\mathbf{W}$ 는 가중치 벡터,  $\mathbf{b}$ 는 편향입니다.
2. 결정 함수 값 반환: `decision\_function` 메서드는 각 샘플에 대해 계산된 선형 결합 값을 반환합니다. 이 값은 나중에 로지스틱 함수(시그모이드 함수)에 전달되어 확률 값으로 변환됩니다.

### 예제

```
clf_lr = LogisticRegression(solver='lbfgs')
clf_lr.fit(df_prob3_train[X_lr], df_prob3_train['failure'])
decision_values = clf_lr.decision_function(df_prob3_train[X_lr])
print(decision_values)
```

- `decision\_values`에는 각 샘플에 대해 계산된 결정 함수 값이 저장됩니다.

### 결정 함수 값의 해석

- 양수 값: 샘플이 **Positive** 클래스(1)로 분류될 가능성이 높음을 나타냅니다.
- 음수 값: 샘플이 **Negative** 클래스(0)로 분류될 가능성이 높음을 나타냅니다.
- 값이 클수록 해당 클래스에 대한 확신이 높습니다.

### 전체 예제 코드

```
from sklearn.linear_model import LogisticRegression

학습 데이터셋에 대해 모델을 학습시킵니다.
clf_lr = LogisticRegression(solver='lbfgs')
clf_lr.fit(df_prob3_train[X_lr], df_prob3_train['failure'])

학습 데이터셋의 결정 함수 값을 계산합니다.
decision_values = clf_lr.decision_function(df_prob3_train[X_lr])

결정 함수 값을 출력합니다.
print(decision_values)
```

이 코드는 `LogisticRegression` 모델을 학습시키고, 학습 데이터셋에 대해 각 샘플의 결정 함수 값을 계산하여 출력합니다. 결정 함수 값은 모델의 예측 확률을 계산하기 위한 중간 단계로, 이 값이 클수록 해당 클래스에 대한 모델의 확신이 높음을 나타냅니다.

```
'''
Logistic Regression에서의 decision_function은 Logit을 반환합니다.
이를 이용해도 Positive 확률과 똑같은 roc_auc를 얻을 수 있습니다.
decision function을 출력해봅니다.
'''
```

```
clf_lr.decision_function(df_prob3_train[X_lr])
```

```
Out[67]: array([-1.37637411, -0.7573929 , -1.11812489, ..., -0.98811264,
 -1.51840754, -1.61391617])
```

## 로짓 변환이란?

로짓 변환은 확률 (p)를 로짓 값으로 변환하는 함수로 정의됩니다:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

여기서:

- (p)는 Positive 클래스(1)에 대한 예측 확률입니다.
- (1 - p)는 Negative 클래스(0)에 대한 예측 확률입니다.

In [68]:

```
'''
코드는 `LogisticRegression` 모델의 예측 확률을 사용하여 로짓 변환(logit transformation)을 수행합니다.
로짓 변환은 확률을 로짓 값으로 변환하는 과정으로, 로지스틱 회귀 모델에서 결정 함수 값과 동일한 결과를 제공합니다.

로짓 변환은 확률을 무한한 범위의 실수 값으로 변환하여 선형 회귀 모델에서 사용하는 것과 같은 선형 모델을 사용할 수 있게 합니다.

코드 설명
- `clf_lr.predict_proba(df_prob3_train[X_lr]):`
 학습 데이터에 대한 각 클래스의 예측 확률을 반환합니다. 반환된 배열의 각 행은 `[p0, p1]` 형태로,
 `p0`는 Negative 클래스(0)의 확률, `p1`은 Positive 클래스(1)의 확률입니다.
- `clf_lr.predict_proba(df_prob3_train[X_lr])[:, 1]:`
 Positive 클래스(1)에 대한 예측 확률만 선택합니다.
- `1 - clf_lr.predict_proba(df_prob3_train[X_lr])[:, 1]:`
 Negative 클래스(0)에 대한 예측 확률을 계산합니다.
- `clf_lr.predict_proba(df_prob3_train[X_lr])[:, 1]`
'''
```

```

 / (1 - clf_lr.predict_proba(df_prob3_train[X_lr]))[:, 1])`:
 Positive 클래스(1)에 대한 예측 확률을 Negative 클래스(0)에 대한 예측 확률로 나눕니다.
- `np.log(...)` : 이 비율에 대한 자연 로그를 계산합니다.

예제
from sklearn.linear_model import LogisticRegression
import numpy as np

학습 데이터셋에 대해 모델을 학습시킵니다.
clf_lr = LogisticRegression(solver='lbfgs')
clf_lr.fit(df_prob3_train[X_lr], df_prob3_train['failure'])

Positive 클래스(1)에 대한 예측 확률을 구합니다.
probs = clf_lr.predict_proba(df_prob3_train[X_lr]))[:, 1]

로짓 변환을 수행합니다.
logit_values = np.log(probs / (1 - probs))

로짓 값을 출력합니다.
print(logit_values)

결정 함수 값과 로짓 값의 관계
로지스틱 회귀 모델에서 `decision_function` 메서드가 반환하는 값은 바로 이 로짓 값입니다.
따라서 위의 코드에서 계산된 `logit_values`는 `clf_lr.decision_function(df_prob3_train[X_lr])`와 동일합니다.
이를 확인하기 위해 아래와 같이 코드를 작성할 수 있습니다:

decision_function 값을 구합니다.
decision_values = clf_lr.decision_function(df_prob3_train[X_lr])

로짓 변환 값을 계산합니다.
logit_values = np.log(probs / (1 - probs))

두 값이 동일한지 확인합니다.
np.allclose(decision_values, logit_values)

이 코드는 `decision_values`와 `logit_values`가 동일한지 확인합니다.
`np.allclose`는 두 배열의 모든 요소가 거의 같은지 확인하는 함수입니다.
이 확인 작업을 통해 로짓 변환 값이 로지스틱 회귀의 결정 함수 값과 동일하다는 것을 알 수 있습니다.
...

실제 로짓을 구해봅니다.
np.log(

```

```
clf_lr.predict_proba(df_prob3_train[X_lr][:, 1] / (1 - clf_lr.predict_proba(df_prob3_train[X_lr][:, 1]))
)
```

```
Out[68]: array([-1.37637411, -0.7573929 , -1.11812489, ..., -0.98811264,
 -1.51840754, -1.61391617])
```

## 로지스틱 함수 (시그모이드 함수)

로지스틱 함수는 다음과 같이 정의됩니다:

$$\text{logit}(z) = \frac{1}{1+e^{-z}}$$

여기서 (z)는 결정 함수 값입니다.

이 함수는 모든 실수 값을 (0, 1) 범위의 확률로 변환합니다.

```
In [69]: '''
`expit(clf_lr.decision_function(df_prob3_train[X_lr]))` 코드는
`LogisticRegression` 모델의 결정 함수 값을 사용하여 각 샘플에 대해 예측 확률을 계산합니다.
`expit` 함수는 로지스틱 함수(시그모이드 함수)로, 결정 함수 값을 확률 값으로 변환합니다.

코드 설명
1. `clf_lr.decision_function(df_prob3_train[X_lr])`:
 - 로지스틱 회귀 모델의 결정 함수 값을 계산합니다. 각 샘플에 대해 선형 결합 값(z)를 반환합니다.
2. `expit(...)` :
 - 결정 함수 값 (z)를 로지스틱 함수에 전달하여 확률 값으로 변환합니다.

동작 과정
1. 결정 함수 값 계산:
 - `clf_lr.decision_function(df_prob3_train[X_lr])`는 각 샘플에 대해 선형 결합 값을 반환합니다. 이 값은 $XW + b$ 로 계산됩니다.
2. 로지스틱 함수 적용:
 - `expit` 함수는 결정 함수 값을 확률 값으로 변환합니다. 이는 각 샘플에 대해 Positive 클래스(1)에 속할 확률을 나타냅니다.

예제 코드
from sklearn.linear_model import LogisticRegression
from scipy.special import expit

학습 데이터셋에 대해 모델을 학습시킵니다.
clf_lr = LogisticRegression(solver='lbfgs')
clf_lr.fit(df_prob3_train[X_lr], df_prob3_train['failure'])
```



```

결정 함수 값을 구합니다.
decision_values = clf_lr.decision_function(df_prob3_train[X_lr])

로지스틱 함수를 적용하여 확률 값으로 변환합니다.
probabilities = expit(decision_values)

확률 값을 출력합니다.
print(probabilities)

동작 원리
1. 모델 학습:
 - `clf_lr.fit(df_prob3_train[X_lr], df_prob3_train['failure'])`는 로지스틱 회귀 모델을 학습 데이터로 학습시킵니다.
2. 결정 함수 값 계산:
 - `decision_values = clf_lr.decision_function(df_prob3_train[X_lr])`는 학습 데이터의 각 샘플에 대해 결정 함수 값을 계산합니다.
3. 확률 값 변환:
 - `probabilities = expit(decision_values)`는 결정 함수 값을 로지스틱 함수에 전달하여 각 샘플이 Positive 클래스(1)일 확률을 계산

검증
위의 `probabilities` 값은 `clf_lr.predict_proba(df_prob3_train[X_lr])[:, 1]`와 동일해야 합니다.
이를 확인하기 위해 아래와 같은 코드를 작성할 수 있습니다:

predict_proba를 사용하여 확률 값을 구합니다.
predicted_probabilities = clf_lr.predict_proba(df_prob3_train[X_lr])[:, 1]

두 값이 동일한지 확인합니다.
np.allclose(probabilities, predicted_probabilities)

이 코드는 `probabilities`와 `predicted_probabilities`가 거의 동일한지 확인합니다.
`np.allclose`는 두 배열의 모든 요소가 거의 같은지 확인하는 함수입니다.
이를 통해 로지스틱 함수가 제대로 동작했는지 검증할 수 있습니다.
...

Logit의 역함수인 로지스틱 함수에 넣으면 확률이 나옵니다.
from scipy.special import expit

expit(clf_lr.decision_function(df_prob3_train[X_lr]))

```

```

Out[69]: array([0.20159197, 0.31921256, 0.24635926, ..., 0.27128503, 0.17969614,
 0.16604561])

```

```
In [70]: '''
`roc_auc_score(df_prob3_test['failure'], clf_lr.decision_function(df_prob3_test[X_lr]))`
코드는 테스트 데이터에 대한 ROC AUC 점수를 계산합니다.
여기서 `decision_function`을 사용하여 결정 함수 값을 입력으로 사용합니다.

ROC AUC 점수란?
ROC AUC는 분류 모델의 성능을 평가하는 지표 중 하나로, 모델의 분류 능력을 전체적으로 나타냅니다.
AUC 값은 0.5에서 1 사이의 값을 가지며, 1에 가까울수록 더 나은 모델 성능을 의미합니다.

동작 과정
1. 결정 함수 값 계산:
 - `clf_lr.decision_function(df_prob3_test[X_lr])`는 테스트 데이터에 대한 결정 함수 값을 계산합니다.
2. ROC AUC 점수 계산:
 - `roc_auc_score(df_prob3_test['failure'], ...)`는 실제 라벨(`failure`)과 결정 함수 값을 사용하여 ROC AUC 점수를 계산합니다.

코드 설명
1. 모델 학습:
 - 로지스틱 회귀 모델을 학습 데이터로 학습시킵니다.
2. 결정 함수 값 계산:
 - `clf_lr.decision_function(df_prob3_test[X_lr])`는 테스트 데이터의 각 샘플에 대한 결정 함수 값을 반환합니다.
3. ROC AUC 점수 계산:
 - `roc_auc_score` 함수는 실제 라벨과 결정 함수 값을 사용하여 ROC AUC 점수를 계산합니다.

예제 코드
from sklearn.metrics import roc_auc_score
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

데이터 전처리: 학습 및 테스트 데이터셋 표준화
std_scaler = StandardScaler()
X_std = ['loading'] + ['measurement_{}'.format(i) for i in range(18)]
df_prob3_train[X_std] = std_scaler.fit_transform(df_prob3_train[X_std])
df_prob3_test[X_std] = std_scaler.transform(df_prob3_test[X_std])

로지스틱 회귀 모델 학습
X_lr = ['loading'] + ['measurement_{}'.format(i) for i in range(18)]
 + ['na_1', 'na_2']
clf_lr = LogisticRegression(solver='lbfgs')
clf_lr.fit(df_prob3_train[X_lr], df_prob3_train['failure'])

테스트 데이터에 대한 결정 함수 값 계산
```

```

decision_values_test = clf_lr.decision_function(df_prob3_test[X_lr])

테스트 데이터에 대한 ROC AUC 점수 계산
AUC_test = roc_auc_score(df_prob3_test['failure'], decision_values_test)

ROC AUC 점수 출력
print(AUC_test)

동작 원리
1. 데이터 전처리:
 - `StandardScaler`를 사용하여 학습 데이터와 테스트 데이터의 특성을 표준화합니다.
2. 모델 학습:
 - `clf_lr.fit(df_prob3_train[X_lr], df_prob3_train['failure'])`를 통해 로지스틱 회귀 모델을 학습시킵니다.
3. 결정 함수 값 계산:
 - `decision_values_test = clf_lr.decision_function(df_prob3_test[X_lr])`는 테스트 데이터의 각 샘플에 대한 결정 함수 값을 반환함
4. ROC AUC 점수 계산:
 - `AUC_test = roc_auc_score(df_prob3_test['failure'], decision_values_test)`는
 테스트 데이터에 대한 실제 라벨과 결정 함수 값을 사용하여 ROC AUC 점수를 계산합니다.

검증
이 점수는 모델이 테스트 데이터에서 얼마나 잘 예측하는지를 나타냅니다.
ROC AUC 점수는 모델의 성능을 평가하는 데 중요한 지표로 사용됩니다.
AUC 값이 0.5보다 크면 모델이 무작위 예측보다 나은 성능을 보이며, 1에 가까울수록 완벽한 분류 성능을 의미합니다.
...

Logit을 가지고 roc_auc_score를 구할 수 있습니다.
roc_auc_score(df_prob3_test['failure'], clf_lr.decision_function(df_prob3_test[X_lr]))

```

Out[70]: 0.5792951262053387

```

In [71]: '''
`roc_auc_score(df_prob3_test['failure'], df_prob3_test['loading'])` 코드는
테스트 데이터에 대해 ROC AUC 점수를 계산합니다. 여기서는 예측에 사용되는 특성이 `loading`이며,
이 특성에 대한 예측이 실제 라벨인 `failure`과 얼마나 잘 일치하는지를 측정합니다.

ROC AUC 점수란?
ROC AUC는 분류 모델의 성능을 평가하는 지표 중 하나입니다. 모델의 분류 능력을 전반적으로 나타내며,
AUC 값은 0.5에서 1 사이의 값을 가지며, 1에 가까울수록 더 나은 모델 성능을 나타냅니다.

동작 과정
1. 예측값 준비:
 - `df_prob3_test['loading']`는 `loading` 특성에 대한 예측값을 나타냅니다.

```

```

- `df_prob3_test['failure']`는 실제 라벨을 나타냅니다.
2. ROC AUC 점수 계산:
- `roc_auc_score` 함수는 실제 라벨과 예측값을 사용하여 ROC AUC 점수를 계산합니다.

코드 설명
1. 예측값 준비:
- `df_prob3_test['loading']`는 로지스틱 회귀 모델이 예측한 특성 `loading`의 값입니다.
- `df_prob3_test['failure']`는 테스트 데이터의 실제 라벨인 `failure`입니다.
2. ROC AUC 점수 계산:
- `roc_auc_score(df_prob3_test['failure'], df_prob3_test['loading'])`는
 실제 라벨과 예측값을 사용하여 ROC AUC 점수를 계산합니다.

예제 코드
from sklearn.metrics import roc_auc_score

테스트 데이터에 대한 ROC AUC 점수 계산
AUC_test = roc_auc_score(df_prob3_test['failure'], df_prob3_test['loading'])

ROC AUC 점수 출력
print(AUC_test)

동작 원리
1. 데이터 준비:
- `df_prob3_test['loading']`는 테스트 데이터의 특성 `loading`에 대한 예측값입니다.
- `df_prob3_test['failure']`는 테스트 데이터의 실제 라벨인 `failure`입니다.
2. ROC AUC 점수 계산:
- `roc_auc_score(df_prob3_test['failure'], df_prob3_test['loading'])`는
 실제 라벨과 예측값을 사용하여 ROC AUC 점수를 계산합니다.

검증
이 점수는 모델이 `loading` 특성을 사용하여 테스트 데이터에서 얼마나 잘 예측하는지를 나타냅니다.
ROC AUC 점수가 0.5보다 크면 모델이 일반적인 기준보다 더 나은 성능을 보이며, 1에 가까울수록 완벽한 예측을 나타냅니다.
'''

roc_auc_score(df_prob3_test['failure'], df_prob3_test['loading'])

```

Out[71]: 0.5778277926972065

In [72]: '''  
 이 코드는 ROC 곡선을 그리는 과정을 나타냅니다. ROC 곡선은 분류 모델의 성능을 시각화하는 방법 중 하나로,  
 민감도(TPR, True Positive Rate)와 특이도(FPR, False Positive Rate) 사이의 관계를 보여줍니다.

## ### 동작 과정

## 1. 데이터 준비:

- `df_prob3_test[['failure']]`는 테스트 데이터의 실제 라벨을 나타내는 **DataFrame**입니다.
- `clf_lr.decision_function(df_prob3_test[X_lr])`는 로지스틱 회귀 모델의 결정 함수 값을 계산합니다. 이 값은 각 샘플의 예측 점수를 나타냅니다.
- `assign` 함수를 사용하여 각 샘플의 예측 점수를 새로운 열인 `'score'`로 추가합니다.
- 결과를 내림차순으로 정렬합니다.

## 2. TPR 및 FPR 계산:

- 누적 합산을 통해 각 샘플에 대한 TPR(진양성율, True Positive Rate) 및 FPR(가양성율, False Positive Rate)을 계산합니다.
- TPR은 True Positive의 누적 비율로, 양성 샘플 중에서 모델이 올바르게 예측한 비율입니다.
- FPR은 False Positive의 누적 비율로, 음성 샘플 중에서 모델이 잘못 예측한 비율입니다.

## 3. ROC 곡선 그리기:

- 계산된 FPR 및 TPR 값을 사용하여 ROC 곡선을 그립니다.
- 직선 `y = x`를 대각선으로 추가하여 무작위 분류를 나타냅니다.

## 4. 축 레이블 추가:

- x축과 y축에 각각 FPR과 TPR을 나타내는 레이블을 추가합니다.

## 5. 그래프 출력:

- `plt.show()` 함수를 사용하여 그래프를 출력합니다.

## ### 코드 설명

- `plt.figure(figsize=(6, 5))`: 그래프의 크기를 지정합니다.
- `plt.plot([0, 1], [0, 1], color='orange', linestyle='--')`: ROC 곡선의 기준이 되는 대각선을 그립니다.
- `df_prob3_test[['failure']].assign(...)`: 테스트 데이터에서 실패 여부를 기반으로 점수를 계산하고, 그에 따라 TPR 및 FPR을 계산합니다.
- `.pipe(lambda x: plt.plot(x['fpr'], x['tpr']))`: 계산된 TPR 및 FPR을 사용하여 ROC 곡선을 그립니다.
- `plt.gca().set_xlabel('False Postive Rate')`: x축에 FPR 레이블을 추가합니다.
- `plt.gca().set_ylabel('True Postive Rate')`: y축에 TPR 레이블을 추가합니다.
- `plt.show()`: 그래프를 출력합니다.

## ### 그래프 해석

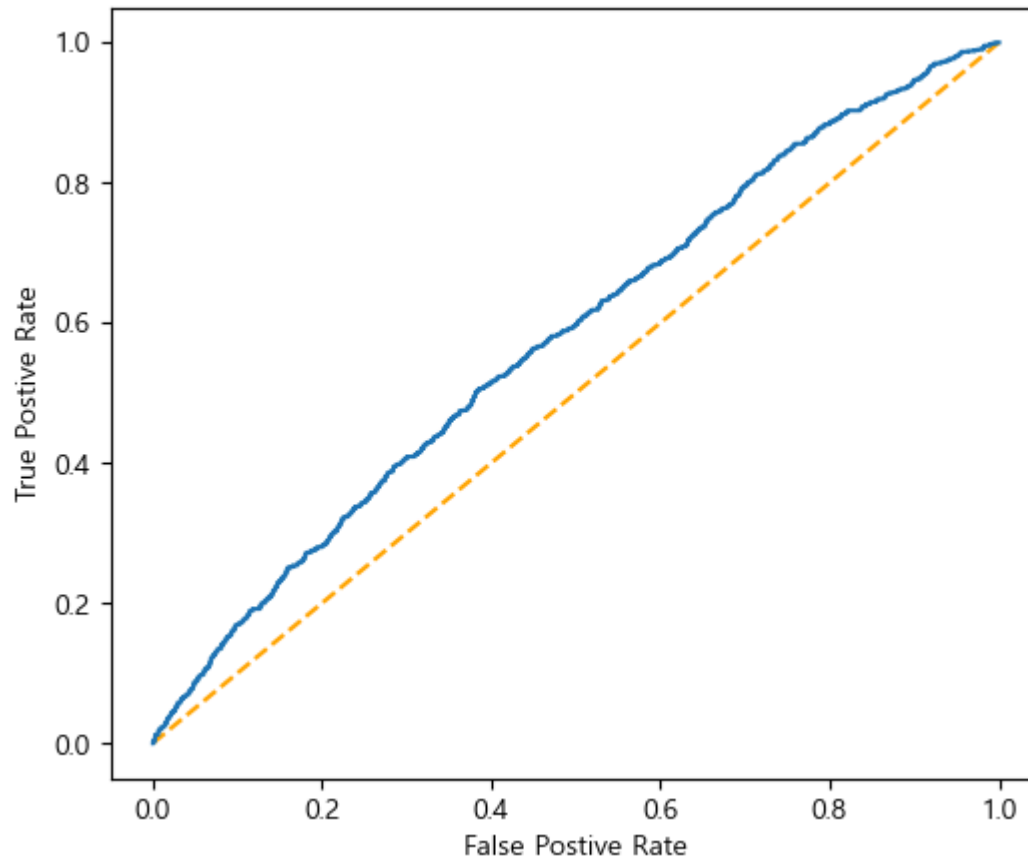
- 그래프의 각 점은 결정 임계값을 변화시켰을 때, 해당 임계값에서의 TPR과 FPR을 나타냅니다.
- ROC 곡선은 왼쪽 상단에 가까울수록 모델의 성능이 좋을 나타냅니다.
- 기준선인 대각선은 무작위 예측을 나타내며, 곡선이 이 대각선 위에 위치하면 모델의 성능이 무작위 수준보다 나은 것입니다.

# ROC AUC를 그려봅니다.

# 이 때 *Logit*으로 구해 보겠습니다.

```
plt.figure(figsize=(6, 5))
```

```
plt.plot([0, 1], [0, 1], color='orange', linestyle='--')
df_prob3_test[['failure']].assign(
 score = clf_lr.decision_function(df_prob3_test[X_lr])
).sort_values('score', ascending=False).assign(
 fpr=lambda x: (1-x['failure']).cumsum() / (1 - x['failure']).sum(),
 tpr=lambda x: (x['failure']).cumsum() / (x['failure']).sum(),
).pipe(
 lambda x: plt.plot(x['fpr'], x['tpr'])
)
plt.gca().set_xlabel('False Postive Rate')
plt.gca().set_ylabel('True Postive Rate')
plt.show()
```



In [73]: ...

이 코드는 ROC 곡선 아래 영역의 넓이를 계산하여 ROC AUC의 실제 값을 구합니다.

### 동작 과정

1. 예측값 준비:

- `df_prob3_test[['failure']]`  
`.assign(score=clf_lr.decision_function(df_prob3_test[X_lr]))`는 테스트 데이터의 각 샘플에 대한 결정 함수 값을 계산합니다.
- `assign` 함수를 사용하여 각 샘플의 예측 점수를 새로운 열인 `'score'`로 추가합니다.
- 결과를 내림차순으로 정렬합니다.

2. TPR 및 FPR 계산:

- 누적 합산을 통해 각 샘플에 대한 TPR(진양성율, True Positive Rate) 및 FPR(가양성율, False Positive Rate)을 계산합니다.

3. 이전 FPR 값 계산:

- `fpr_1 = lambda x: x['fpr'].shift(1).fillna(0)`를 사용하여 각 샘플의 이전 FPR 값을 계산합니다.
- 이전 FPR 값을 사용하여 사다리꼴의 넓이를 계산합니다.

4. ROC 곡선 아래 영역 계산:

- `sq = lambda x: (x['fpr'] - x['fpr_1']) * x['tpr']`를 사용하여 각 샘플에서의 사다리꼴의 넓이를 계산합니다.
- 계산된 넓이를 모두 합산하여 ROC 곡선 아래 영역의 값을 구합니다.

### 코드 설명

- `['sq'].sum()`: ROC 곡선 아래 영역의 넓이를 모두 합산하여 ROC AUC 값을 반환합니다.

이 방법은 ROC 곡선 아래 영역의 넓이를 계산하여 ROC AUC 값을 구하는 방법 중 하나입니다.

ROC AUC 값은 분류 모델의 성능을 평가하는 데 사용되는 중요한 지표 중 하나이며, 값이 1에 가까울수록 모델의 성능이 우수함을 의미합니다  
 ...

# ROC AUC의 실제 값을 구해봅니다.

```
df_prob3_test[['failure']].assign(
 score = clf_lr.decision_function(df_prob3_test[X_lr])
).sort_values('score', ascending=False).assign(
 fpr=lambda x: (1 - x['failure']).cumsum() / (1 - x['failure']).sum(),
 tpr=lambda x: (x['failure']).cumsum() / (x['failure']).sum(),
).assign(
 fpr_1 = lambda x: x['fpr'].shift(1).fillna(0)
).assign(
 sq = lambda x: (x['fpr'] - x['fpr_1']) * x['tpr']
)['sq'].sum()
```

Out[73]: 0.5792951262053387

## 단계 3-5

loading, measurement\_0 ~ 17, na\_1, na\_2를 후보 입력 변수로 한다.

전진 선택법을 사용하여 이 후보 입력 변수 중에서 최적의 성능을 보이는 입력 변수의 조합을 찾는다.

전진 선택법의 선택 기준은 prob3\_train을 대상으로 5겹 층화교차검증(5-Fold stratified cross validation)을 하고

검외(OOF, Out-Of Fold) 성능의 평균값으로 한다. 전진 선택 과정에서 선택했던 변수를 제외하지 않는다.

입력 변수: 본 단계 요건 참고

대상 변수: failure

성능 지표: AUC(area under of ROC curve)

## 함수가이드

mlxtend.feature\_selection.SequentialFeatureSelector

sklearn.linear\_model.LogisticRegression, solver='lbfgs'

sklearn.metrics.roc\_auc\_score

sklearn.model\_selection.StratifiedKFold, random\_state=123, shuffle=True

문제 지시사항 외 Default 값 사용

In [74]:

```
'''
`SequentialFeatureSelector`는 `mlxtend` 라이브러리에서 제공하는 함수로,
특성 선택(feature selection)을 순차적으로 수행하는 방법을 제공합니다.
이 함수는 모델의 성능을 최적화하기 위해 특성의 부분 집합을 선택하는 데 사용됩니다.
Sequential Feature Selection(SFS)은 Forward Selection과 Backward Elimination 방법을 포함합니다.

함수 정의
from mlxtend.feature_selection import SequentialFeatureSelector

인자
`SequentialFeatureSelector` 클래스는 다양한 인자를 받아 특성 선택 과정을 제어합니다.

- estimator:
```



- 특성 선택에 사용할 모델(추정기)입니다. 예를 들어, `LogisticRegression()`, `RandomForestClassifier()` 등이 가능합니다.
- `k_features`:
  - 선택할 특성의 개수입니다. 정수로 지정하면 고정된 개수의 특성을 선택하고, `(min_features, max_features)` 튜플로 지정하면 지정된 범위 내에서 최적의 특성 개수를 선택합니다.
- `forward` (기본값: `True`):
  - `True`이면 Forward Selection(순방향 선택)을 수행하고, `False`이면 Backward Elimination(역방향 제거)을 수행합니다.
- `floating` (기본값: `False`):
  - `True`이면 Floating 알고리즘을 사용하여 선택된 특성 집합을 동적으로 조정합니다.
- `verbose` (기본값: `0`):
  - 선택 과정의 진행 상황을 출력하는 정도를 지정합니다. `0`이면 출력 없음, `1`이면 최소한의 출력, `2`이면 상세 출력.
- `scoring` (기본값: `None`):
  - 모델 성능을 평가하는 데 사용할 스코어링 방법입니다. 예를 들어, `'accuracy'`, `'f1'`, `'roc_auc'` 등이 있습니다. `None`이면 `estimator`의 기본 평가 방법을 사용합니다.
- `cv` (기본값: `5`):
  - 교차 검증을 수행할 폴드 수입니다. `None`이면 교차 검증을 수행하지 않습니다.
- `n_jobs` (기본값: `1`):
  - 사용할 CPU 코어 수입니다. `-1`이면 모든 코어를 사용합니다.
- `pre_dispatch` (기본값: `'2*n_jobs'`):
  - 병렬 실행 시 작업을 미리 디스패치할 수를 지정합니다.

### 리턴값

`SequentialFeatureSelector` 클래스는 특성 선택을 수행하는 객체를 반환합니다. 주요 속성과 메서드는 다음과 같습니다:

- `k_feature_idx_`:
  - 선택된 특성의 인덱스입니다.
- `k_feature_names_`:
  - 선택된 특성의 이름입니다. 데이터프레임 입력 시 유용합니다.
- `k_score_`:
  - 선택된 특성 집합의 성능 점수입니다.

### 주요 메서드

- `fit(X, y)`:
  - 모델을 학습시키고 최적의 특성 집합을 선택합니다.
- `transform(X)`:
  - 선택된 특성 집합을 사용하여 데이터를 변환합니다.
- `fit_transform(X, y)`:
  - 모델을 학습시키고 데이터를 변환합니다.

### 예제

다음은 `SequentialFeatureSelector`를 사용하여 특성 선택을 수행하는 예제입니다:

```
import pandas as pd
```

```

from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from mlxtend.feature_selection import SequentialFeatureSelector as SFS

데이터 로드
iris = load_iris()
X, y = iris.data, iris.target
df = pd.DataFrame(X, columns=iris.feature_names)

모델 정의
knn = KNeighborsClassifier(n_neighbors=3)

Sequential Feature Selector 설정
sfs = SFS(knn,
 k_features=3,
 forward=True,
 floating=False,
 scoring='accuracy',
 cv=5)

특성 선택 수행
sfs = sfs.fit(X, y)

선택된 특성 확인
print(f"Selected features: {sfs.k_feature_idx_}")
print(f"Selected feature names: {sfs.k_feature_names_}")
print(f"Cross-validation score: {sfs.k_score_}")

선택된 특성을 사용하여 데이터 변환
X_selected = sfs.transform(X)
print(f"Shape of X_selected: {X_selected.shape}")

결론
`SequentialFeatureSelector`는 모델의 성능을 최적화하기 위해 특성을 선택하는 데 유용한 도구입니다.
Forward Selection과 Backward Elimination 방법을 모두 지원하며, 교차 검증을 통해 선택 과정의 신뢰성을 높일 수 있습니다.
'''

print('')

```

In [75]: '''

```

아래 코드는 특성 선택을 위해 Sequential Feature Selector를 사용하는 것으로 보입니다.
'''

```

여기에서는 Logistic Regression 모델을 사용하여 최적의 특성 조합을 찾습니다.

### 동작 과정:

1. Sequential Feature Selector 초기화:

- `SequentialFeatureSelector` 클래스를 사용하여 Sequential Feature Selector 객체를 초기화합니다.
- `LogisticRegression(solver='lbfgs')`를 전달하여 사용할 모델을 지정합니다.  
여기서는 로지스틱 회귀 모델을 사용합니다.
- `k\_features = "best"`: 최적의 성능을 보이는 입력 변수의 조합을 찾습니다.
- `forward=True`: 전진 선택법을 사용하여 특성을 선택합니다.
- `cv=StratifiedKFold(5, random\_state=123, shuffle=True)`:  
5-Fold 층화 교차 검증을 수행합니다. 이를 통해 데이터를 5개의 폴드로 나누고,  
각 반복에서 한 폴드를 테스트 세트로 사용하여 모델을 평가합니다.
- `floating=False`: 전진 선택 과정에서 선택했던 변수를 제외하지 않습니다.
- `scoring='roc\_auc'`: 모델 성능을 평가할 지표로 ROC AUC를 사용합니다.

2. 특성 선택 실행:

- `sfs.fit(df\_prob3\_train[X\_sfs], df\_prob3\_train['failure'])`:  
훈련 데이터의 특성과 라벨을 사용하여 Sequential Feature Selector를 실행합니다.  
여기서 `df\_prob3\_train[X\_sfs]`는 특성들을, `df\_prob3\_train['failure']`는 라벨을 나타냅니다.

코드는 주어진 특성 집합에서 로지스틱 회귀 모델을 사용하여 최적의 특성 조합을 찾는 과정을 수행합니다.  
최적의 특성 조합은 ROC AUC 점수를 최대화하는 것을 목표로 합니다.

...

...

- `X\_sfs`라는 변수를 정의합니다. 이 변수는 특성들의 리스트를 나타냅니다.
- 'loading'은 첫 번째 특성이고, 'measurement\_1'부터 'measurement\_17'까지의 숫자를 갖는 특성들을 추가합니다.
- 마지막으로, 'na\_1'과 'na\_2' 두 개의 추가적인 특성을 리스트에 추가합니다.
- `SequentialFeatureSelector` 클래스의 인스턴스를 생성합니다.
- 이때, 모델로 로지스틱 회귀(Logistic Regression)를 사용합니다. 로지스틱 회귀의 solver로 'lbfgs'를 지정합니다.
- `k\_features = "best"`는 최적의 성능을 보이는 입력 변수의 조합을 찾는 옵션입니다.
- `forward=True`는 전진 선택법을 사용하여 특성을 선택하는 옵션입니다.
- `cv=StratifiedKFold(5, random\_state=123, shuffle=True)`은 5-Fold 층화 교차 검증을 수행하는 옵션입니다.  
이를 통해 데이터를 5개의 폴드로 나누고, 각 반복에서 한 폴드를 테스트 세트로 사용하여 모델을 평가합니다.
- `floating=False`는 전진 선택 과정에서 선택했던 변수를 제외하지 않는 옵션입니다.
- `scoring='roc\_auc'`는 모델 성능을 평가할 지표로 ROC AUC를 사용하는 옵션입니다.
- 이제 `SequentialFeatureSelector` 객체를 사용하여 특성 선택을 수행합니다.
- `df\_prob3\_train[X\_sfs]`는 훈련 데이터의 특성을, `df\_prob3\_train['failure']`는 해당 특성들에 대한 라벨(실패 여부)을 나타냅니다.
- 선택된 최적의 특성 조합은 `sfs` 객체 내에 저장됩니다.

이 코드는 로지스틱 회귀 모델을 사용하여 최적의 특성 조합을 찾기 위해 Sequential Feature Selector를 구성하고 실행합니다. 최적의 특성 조합을 찾는 데에는 ROC AUC를 평가 지표로 사용하며, 5-Fold 총화 교차 검증을 통해 모델의 일반화 성능을 평가합니다.

```
from mlxtend.feature_selection import SequentialFeatureSelector
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold

X_sfs = ['loading'] + ['measurement_{}'.format(i) for i in range(18)] + ['na_1', 'na_2']

sfs = SequentialFeatureSelector(
 LogisticRegression(solver='lbfgs'),
 # 이 후보 입력 변수 중에서 최적의 성능을 보이는 입력 변수의 조합을 찾는다.
 k_features = "best",
 forward=True, # 전진 선택법을 사용하여
 # 5겹 총화교차검증(5-Fold stratified cross validation)을 하고
 cv=StratifiedKFold(5, random_state=123, shuffle=True),
 floating=False, # 전진 선택 과정에서 선택했던 변수를 제외하지 않는다.
 # _score로 끝나면 뺀다: roc_auc_score -> roc_auc,
 # _error 끝나면: neg_ neg_mean_squared_error
 scoring='roc_auc',
)
sfs.fit(df_prob3_train[X_sfs], df_prob3_train['failure'])
```

```
Out[75]: SequentialFeatureSelector(clone_estimator=True,
 cv=StratifiedKFold(n_splits=5, random_state=123, shuffle=True),
 estimator=LogisticRegression(C=1.0, class_weight=None,
 dual=False,
 fit_intercept=True,
 intercept_scaling=1,
 l1_ratio=None,
 max_iter=100,
 multi_class='warn',
 n_jobs=None,
 penalty='l2',
 random_state=None,
 solver='lbfgs',
 tol=0.0001, verbose=0,
 warm_start=False),
 floating=False, forward=True, k_features='best',
 n_jobs=1, pre_dispatch='2*n_jobs', scoring='roc_auc',
 verbose=0)
```

```
In [76]: X_sfs_best = list(sfs.k_feature_names_)
X_sfs_best
```

```
Out[76]: ['loading',
 'measurement_1',
 'measurement_4',
 'measurement_14',
 'measurement_17',
 'na_1']
```

## 단계 3-6

단계 3-5에서 찾은 최적의 입력 변수 조합으로 로지스틱 회귀모델을 사용하여 prob3\_train을 학습하고

prob3\_test로 성능을 측정한 값을 B라고 한다.

입력 변수: **단계 3-5**에서 도출한 최적의 입력 변수 조합

대상 변수: failure

성능 지표: AUC(area under of ROC curve)

---

### 함수 가이드

sklearn.linear\_model.LogisticRegression, solver='lbfgs'

sklearn.metrics.roc\_auc\_score

문제 지시사항 외 Default 값 사용

---

```
In [77]: '''
코드는 Sequential Feature Selector를 통해 선택된 최적의 특성을 사용하여 로지스틱 회귀 모델을 훈련하고,
이 모델의 성능을 ROC AUC로 평가합니다.

코드 설명:
```

- `X\_lr`은 최적의 특성 조합인 `X\_sfs\_best`를 나타냅니다.  
이전 단계에서 **Sequential Feature Selector**를 통해 선택된 최적의 특성들이 여기에 저장되어 있습니다.
- 로지스틱 회귀 모델을 초기화합니다. 이 모델은 'lbfgs'라는 최적화 알고리즘을 사용하여 최적화됩니다.
- 로지스틱 회귀 모델을 훈련합니다.
- `df\_prob3\_train[X\_lr]`은 훈련 데이터에서 선택된 최적의 특성을 나타냅니다.
- `df\_prob3\_train['failure']`은 각 샘플의 실패 여부(타겟 변수)를 나타냅니다.
- 테스트 데이터를 사용하여 모델의 성능을 평가합니다.
- `clf\_lr.predict\_proba(df\_prob3\_test[X\_lr])[:, 1]`를 사용하여 모델이 예측한 클래스 1(양성 클래스)에 대한 확률을 얻습니다.
- 이 확률 값을 사용하여 ROC AUC 점수를 계산합니다.
- 이 점수는 변수 `B`에 저장됩니다.
- 마찬가지로 훈련 데이터를 사용하여 모델의 성능을 평가합니다.
- 훈련 데이터에 대한 ROC AUC 점수와 테스트 데이터에 대한 ROC AUC 점수를 함께 반환합니다.

코드는 선택된 최적의 특성을 사용하여 훈련된 로지스틱 회귀 모델의 성능을 평가합니다.

모델은 테스트 데이터와 훈련 데이터 모두에 대해 ROC AUC 점수를 계산합니다.

ROC AUC는 모델의 분류 성능을 평가하는 데 사용되는 중요한 지표 중 하나입니다.

...

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

X_lr = X_sfs_best
clf_lr = LogisticRegression(solver='lbfgs')
clf_lr.fit(df_prob3_train[X_lr], df_prob3_train['failure'])
B = roc_auc_score(df_prob3_test['failure'],
 clf_lr.predict_proba(df_prob3_test[X_lr])[:, 1])

(
 B,
 roc_auc_score(df_prob3_train['failure'],
 clf_lr.predict_proba(df_prob3_train[X_lr])[:, 1])
)
```

Out[77]: (0.5838326230092876, 0.5936682060775388)

In [78]: A, B, A - B, round(A - B, 3)

Out[78]: (0.5792951262053387, 0.5838326230092876, -0.004537496803948882, -0.005)

A-B값을 소수점 넷째 자리에서 반올림하여 셋째 자리까지 출력하시오

**-0.005**

## 문제 4

차원 축소 기법을 통한 데이터의 특성과 failure 분류 성능을 높힐 만한 요소를 살펴 본다.

첫째로, loading을 제외하고, measurement\_0 ~ 17을 입력으로 failure를 대상 변수로

Linear Discriminant Analysis(LDA) 모델을 만든다.

True/False 이진 변수인 failure를 분류한다는 점에서 LDA 모델은 measurement\_0 ~ 17를

한 개의 경계점으로 failure를 최대한 정확하게 구분하도록 하나의 연속형 변수로 변환한다.

스프링의 내구력이 높을 수록 failure 확률이 낮아진다면,

측정값 measurement\_0 ~ 17의 LDA 변환값은 스프링의 내구력을 나타낸다고 할 수 있다.

실험에서 스프링에 가한 부하(loading)와 LDA 변환값의 상관도를 측정하여,

측정값 measurement\_0~17 에서 예상되는 내구력이 스프링에 따라 부하(loading)의 반영 정도를 가늠한다.

둘째로, PCA를 사용하여 차원 감소로 failure 분류 성능에 얼마나 효과가 있을지 살펴본다.

문제3에서 사용했던, 전처리(loading 결측치 처리와 표준화 과정을 거친) 과정을 거친

prob3\_train과 prob3\_test를 사용한다.

In [ ]:

### 단계 4-1

prob3\_train에서 measurement\_0 ~ 17을 입력으로 failure를 대상 변수로 하여

LDA(Linear Discriminant Analysis) 모델을 학습한다.

measurement\_0 ~ 17에 대한 prob3\_train에서의 LDA의 변환값과 loading과

스피어만 상관도 (spearman correlation)의 p-value를 구하여 A라고 한다.

입력 변수] measurement\_0 ~ 17 (순서에 유의 하시오)

대상 변수] failure

## 함수가이드

sklearn.discriminant\_analysis 제공 기능 활용

scipy.stats.spearmanr

문제 지시사항 외 Default 값 사용

In [79]:

```
'''
`LinearDiscriminantAnalysis` (LDA) 클래스는 `scikit-learn` 라이브러리의 `discriminant_analysis` 모듈에
있는 지도 학습 방법 중 하나로, 분류 문제와 차원 축소에 사용됩니다.
LDA는 클래스 간 분산을 최대화하면서 클래스 내 분산을 최소화하는 방식으로 데이터의 선형 결합을 찾습니다.

클래스의 인자
`LinearDiscriminantAnalysis` 클래스의 생성자는 다음과 같은 인자를 가집니다:
1. solver:
 - 유형: 문자열 (기본값은 'svd')
 - 설명: LDA를 풀기 위한 방법을 지정합니다. 사용할 수 있는 값은 'svd', 'lsqr', 'eigen'입니다.
 - 'svd': 특이값 분해(SVD)를 사용한 방법.
 - 'lsqr': 최소제곱법을 사용한 방법.
 - 'eigen': 고유값 분해를 사용한 방법.
2. shrinkage:
 - 유형: 문자열, 'auto', 실수 또는 None (기본값은 None)
 - 설명: 공분산 행렬에 대한 수축 값을 지정합니다. 'lsqr' 또는 'eigen' solver와 함께 사용됩니다.
 'auto'로 설정하면 Ledoit-Wolf lemma를 사용하여 최적의 수축 값을 계산합니다.
3. priors:
 - 유형: 배열 또는 None (기본값은 None)
 - 설명: 각 클래스의 사전 확률을 지정합니다. None인 경우 클래스 비율에 따라 사전 확률이 자동으로 설정됩니다.
```



```

4. n_components:
 - 유형: 정수 또는 None (기본값은 None)
 - 설명: 투영 후 차원 수를 지정합니다. None인 경우, 클래스 수보다 하나 적은 차원 수를 사용합니다.
5. store_covariance:
 - 유형: 부울 (기본값은 False)
 - 설명: True로 설정하면 공분산 행렬을 저장합니다.
6. tol:
 - 유형: 실수 (기본값은 1e-4)
 - 설명: SVD solver에서 사용할 작은 특이값의 절단 값입니다.

주요 메서드
1. fit(X, y):
 - 설명: LDA 모델을 데이터에 적합시킵니다.
 - 인자:
 - `X`: 훈련 데이터 배열 (샘플 수 x 특성 수).
 - `y`: 타겟 레이블 배열.
 - 리턴값: 적합된 `LinearDiscriminantAnalysis` 인스턴스.
2. transform(X):
 - 설명: LDA를 사용하여 차원 축소를 수행합니다.
 - 인자:
 - `X`: 입력 데이터 배열.
 - 리턴값: 변환된 데이터 배열.
3. fit_transform(X, y):
 - 설명: 모델을 적합시키고 차원 축소를 수행합니다.
 - 인자:
 - `X`: 훈련 데이터 배열.
 - `y`: 타겟 레이블 배열.
 - 리턴값: 변환된 데이터 배열.
4. predict(X):
 - 설명: LDA 모델을 사용하여 클래스 레이블을 예측합니다.
 - 인자:
 - `X`: 입력 데이터 배열.
 - 리턴값: 예측된 클래스 레이블 배열.
5. predict_proba(X):
 - 설명: 각 클래스에 대한 확률을 예측합니다.
 - 인자:
 - `X`: 입력 데이터 배열.
 - 리턴값: 각 클래스에 대한 예측 확률 배열.
6. decision_function(X):
 - 설명: 각 클래스에 대한 결정 함수 값을 계산합니다.
 - 인자:
 - `X`: 입력 데이터 배열.

```

- 리턴값: 각 클래스에 대한 결정 함수 값 배열.

### 주요 속성

1. `coef_`:
  - 설명: 선형 판별 함수의 계수 (각 클래스에 대한 선형 결합의 계수).
2. `intercept_`:
  - 설명: 선형 판별 함수의 절편.
3. `explained_variance_ratio_`:
  - 설명: 각 판별 축이 설명하는 분산의 비율.
4. `means_`:
  - 설명: 각 클래스의 평균 벡터.
5. `priors_`:
  - 설명: 각 클래스의 사전 확률.
6. `covariance_`:
  - 설명: 공분산 행렬 (`store_covariance=True`로 설정된 경우).
7. `scalings_`:
  - 설명: 판별 축을 형성하는 선형 판별 함수의 계수 (`solver`가 'svd'인 경우).
8. `xbar_`:
  - 설명: 전체 평균 벡터.

### 예제

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.datasets import load_iris
import numpy as np
```

# 데이터셋 로드

```
iris = load_iris()
X = iris.data
y = iris.target
```

# LDA 모델 생성 및 적합

```
lda = LinearDiscriminantAnalysis()
lda.fit(X, y)
```

# 차원 축소 수행

```
X_r2 = lda.transform(X)
```

# 예측 수행

```
y_pred = lda.predict(X)
```

# 결과 출력

```
print("예측된 클래스 레이블:\n", y_pred)
```

```
print("LDA 차원 축소된 데이터:\n", X_r2)
print("선형 판별 함수의 계수:\n", lda.coef_)
```

이 예제에서는 Iris 데이터셋을 사용하여 LDA 모델을 적합시키고, 차원 축소와 클래스 예측을 수행합니다. `coef\_` 속성을 통해 선형 판별 함수의 계수를 출력합니다.

### 요약

`LinearDiscriminantAnalysis` 클래스는 지도 학습 방식으로 분류 문제를 해결하고 차원 축소를 수행합니다. 다양한 인자들을 통해 모델의 동작 방식을 설정할 수 있으며, 주요 메서드와 속성을 사용하여 모델을 적합시키고 데이터를 변환하거나 예측할 수 있습니다.

```
'''
```

```
print('')
```

In [80]:

```
'''
코드는 선형 판별 분석(Linear Discriminant Analysis, LDA) 모델을 사용하여 특성 데이터를 클래스에 대해 분류하는 모델을 훈련합니다.
```

### 코드 설명:

- `LinearDiscriminantAnalysis` 클래스를 사용하여 선형판별분석(LDA) 객체를 초기화합니다.
- 분류 모델을 훈련하기 위한 특성 데이터를 지정합니다.  
여기서는 'measurement\_0'부터 'measurement\_17'까지의 특성들을 사용합니다.
- LDA 모델을 훈련합니다.
- `df\_prob3\_train[X\_lda]`은 훈련 데이터에서 선택된 특성들을 나타냅니다.
- `df\_prob3\_train['failure']`은 각 샘플의 실패 여부(타겟 변수)를 나타냅니다.
- LDA 모델은 주어진 특성 데이터와 타겟 변수에 대해 훈련되며, 클래스 간의 분리를 최대화하는 경계를 찾습니다.

이렇게 훈련된 LDA 모델은 주어진 특성 데이터를 기반으로 각 클래스에 속할 확률을 예측할 수 있게 됩니다.

```
'''
```

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
lda = LinearDiscriminantAnalysis()
```

```
X_lda = ['measurement_{}'.format(i) for i in range(18)]
lda.fit(df_prob3_train[X_lda], df_prob3_train['failure'])
```

```
Out[80]: LinearDiscriminantAnalysis(n_components=None, priors=None, shrinkage=None,
 solver='svd', store_covariance=False, tol=0.0001)
```

```
In [81]: '''
코드는 훈련 데이터의 'loading' 특성과 LDA를 사용하여 변환된 특성 간의 스피어만 상관관계를 계산하고, 해당 상관관계의 유의성을 검정함

코드 설명:

- `spearmanr` 함수를 사용하여 'loading' 특성과 LDA를 사용하여 변환된 특성 간의 스피어만 상관관계를 계산합니다.
- `df_prob3_train['loading']`은 훈련 데이터의 'loading' 특성을 나타냅니다.
- `lda.transform(df_prob3_train[X_lda])`은 LDA를 사용하여 변환된 특성을 나타냅니다.

- `result_4_1.pvalue`는 스피어만 상관관계의 유의성을 나타내는 p-value를 나타냅니다.
- p-value는 두 변수 간의 관계가 우연히 발생했을 확률을 나타내며, 작을수록 두 변수 간의 관계가 통계적으로 유의미하다는 것을 의미함니
'''

from scipy.stats import spearmanr

result_4_1 = spearmanr(df_prob3_train['loading'],
 lda.transform(df_prob3_train[X_lda]))
A = result_4_1.pvalue
result_4_1, A
```

```
Out[81]: (SpearmanrResult(correlation=0.0029462997112993335, pvalue=0.6995009550811424),
 0.6995009550811424)
```

```
In [82]: '''
코드는 훈련 데이터를 LDA를 사용하여 변환한 후, 첫 번째 변환된 특성에 대한 히스토그램을 그리고 각 클래스에 대해 색을 다르게 표시함니

코드 설명:

- 새로운 그림(figure)을 생성하고, 이 그림의 크기를 지정합니다.
 이 경우에는 가로 길이가 8이고, 세로 길이가 3인 플롯을 생성합니다.

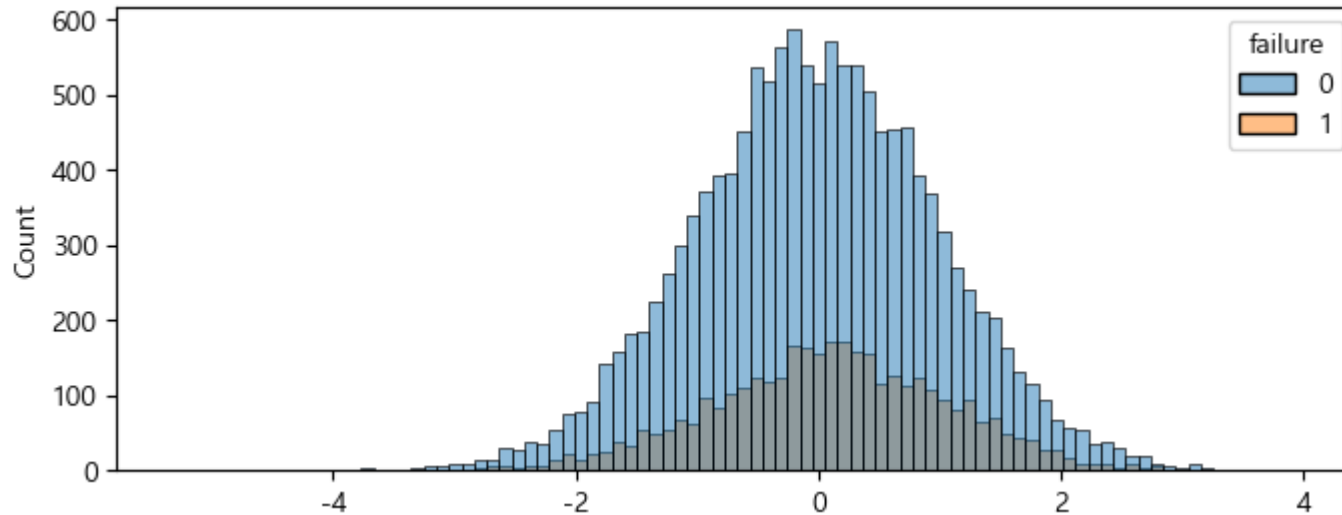
- `sns.histplot()` 함수를 사용하여 히스토그램을 그립니다.
- `x` 인자에는 LDA를 사용하여 변환된 특성 중 첫 번째 특성을 전달합니다.
 이를 위해 `lda.transform(df_prob3_train[X_lda])[ :, 0]`를 사용합니다.
- `hue` 인자에는 클래스를 지정하여 각 클래스에 대한 히스토그램을 다르게 색상으로 표시합니다.
 여기서는 실패 여부를 나타내는 'failure' 열을 전달합니다.

- 플롯을 화면에 출력합니다.
```

이 코드는 훈련 데이터를 LDA를 사용하여 변환한 후, 변환된 첫 번째 특성에 대한 히스토그램을 그리고 각 클래스에 대해 색을 다르게 표시합니다. 히스토그램은 각 클래스의 데이터 분포를 시각화하여 클래스 간의 차이를 살펴볼 수 있게 합니다.

```
...

failure별 LDA의 히스토그램을 그려봅니다.
plt.figure(figsize=(8, 3))
sns.histplot(x=lda.transform(df_prob3_train[X_lda])[:, 0],
 hue=df_prob3_train['failure'])
plt.show()
```



In [83]:

코드는 LDA를 사용하여 변환된 특성 중 첫 번째 성분에 대한 두 클래스(여기서는 실패와 비실패)의 분포를 비교하는 작업을 수행합니다.

### 코드 설명:

- `lda.transform()` 함수를 사용하여 LDA를 사용하여 변환된 특성 데이터를 가져옵니다. 이때 `[:, 0]`를 사용하여 변환된 특성 중 첫 번째 성분만 선택합니다.
- 이렇게 선택된 첫 번째 성분은 `lda_comp1`에 저장됩니다.

```
lda_comp1_0 = lda_comp1[df_prob3_train['failure'] == 0]
```

```
lda_comp1_1 = lda_comp1[df_prob3_train['failure'] == 1]
```

- 실패(`'failure'` 값이 0) 및 비실패(`'failure'` 값이 1) 클래스에 대한 첫 번째 변환된 특성 성분을 각각 `lda_comp1_0` 및 `lda_comp1_1`에 저장합니다.

- `bartlett` 함수를 사용하여 두 클래스 간의 분산이 동일한지 여부를 검정합니다.

- 귀무 가설은 "두 클래스의 분산이 동일하다"입니다.
- `result\_bart`에는 검정 결과가 저장됩니다.
- `ttest\_ind` 함수를 사용하여 두 클래스 간의 평균 차이가 통계적으로 유의한지 검정합니다.
- `equal\_var` 매개변수에는 두 클래스의 분산이 동일한지 여부를 지정합니다.  
이 값은 이전에 계산한 Bartlett 테스트의 결과에 따라 설정됩니다.
- `result\_ttest`에는 검정 결과가 저장됩니다.

이렇게 하여 두 클래스 간의 평균 차이가 통계적으로 유의한지 확인하고, 분산이 동일한지 여부를 고려하여 결과를 도출합니다.

```
시각화를 통해서는 LDA를 통해나온 성분의 뚜렷한 분리력은 확인이 어렵습니다.
t-검정으로 가능해보겠습니다.
from scipy.stats import ttest_ind
from scipy.stats import bartlett

lda_comp1 = lda.transform(df_prob3_train[X_lda][:, 0])

lda_comp1_0 = lda_comp1[df_prob3_train['failure'] == 0]
lda_comp1_1 = lda_comp1[df_prob3_train['failure'] == 1]

정규분포를 따른다는 가정을 하겠습니다.
result_bart = bartlett(lda_comp1_0, lda_comp1_1)
result_ttest = ttest_ind(lda_comp1_0, lda_comp1_1,
 equal_var=result_bart.pvalue >= 0.05)

result_bart, result_ttest
```

```
Out[83]: (BartlettResult(statistic=3.2076454769152565, pvalue=0.07329488644384342),
Ttest_indResult(statistic=-5.927378456131374, pvalue=3.137024373995401e-09))
```

## 단계 4-2

prob3\_train에서 measurement\_0 ~ 17을 대상으로

주성분분석(Principal Component Analysis, PCA) 모델을 학습한다.

분산 설명율이 높은 순으로 주성분을 변수명을 pca\_0 ~ 17하여

prob3\_train에 추가하여 prob4\_train을 만든다.

prob3\_test에 prob3\_train를 학습했던 PCA 모델로 동일한 방법으로

pca\_0 ~17 파생 변수를 추가하여 prob4\_test를 만든다.

입력 변수] measurement\_0 ~ 17 (순서에 유의 하시오)

## 함수가이드

sklearn.decomposition.PCA, random\_state=123

문제 지시사항 외 Default 값 사용

```
In [84]: '''
`PCA` (Principal Component Analysis) 클래스는 `scikit-learn` 라이브러리의 `decomposition` 모듈에 있는 클래스입니다.
PCA는 데이터의 차원을 축소하면서 데이터의 분산을 최대한 보존하는 새로운 직교 축들을 찾는 방법입니다.
데이터의 패턴을 이해하고 시각화하는 데 유용하며, 차원 축소를 통해 데이터의 복잡성을 줄이고 계산 효율성을 높이는 데 사용됩니다.

클래스의 인자
1. n_components:
 - 유형: 정수, 실수, None 또는 문자열 (기본값은 None)
 - 설명: 주성분의 수를 지정합니다.
 - 정수인 경우, 원하는 주성분의 수.
 - 실수 (0에서 1 사이)인 경우, 원하는 분산의 비율.
 - 'mle'인 경우, 자동으로 차원을 결정 (MLE, 최대 우도 추정).
 - None인 경우, 모든 성분을 반환.
2. copy:
 - 유형: 부울 (기본값은 True)
 - 설명: 입력 데이터를 복사할지 여부를 지정합니다. False인 경우, 입력 데이터 자체가 변경될 수 있습니다.
3. whiten:
 - 유형: 부울 (기본값은 False)
 - 설명: True로 설정하면, 출력 성분들이 단위 분산을 갖도록 합니다.
 이는 데이터가 불균형하거나 다양한 스케일을 가질 때 유용할 수 있습니다.
4. svd_solver:
 - 유형: 문자열 (기본값은 'auto')
 - 설명: SVD 분해를 수행할 방법을 지정합니다.
 - 'auto': 적절한 방법을 자동으로 선택.
 - 'full': 완전 SVD 방식.
 - 'arpack': 랜덤화된 SVD 방식.
```

- 'randomized': 빠른 근사치 SVD 방식.
- 5. tol:
  - 유형: 실수 (기본값은 0.0)
  - 설명: svd\_solver가 'arpack'인 경우, 수렴 허용 오차입니다.
- 6. iterated\_power:
  - 유형: 정수 또는 문자열 (기본값은 'auto')
  - 설명: svd\_solver가 'randomized'인 경우, 거듭제곱 횟수를 지정합니다.
- 7. random\_state:
  - 유형: 정수, RandomState 인스턴스 또는 None (기본값은 None)
  - 설명: 난수 생성 시드를 설정합니다. 재현 가능한 결과를 위해 사용합니다.

### 주요 메서드

1. fit(X, y=None):
  - 설명: PCA 모델을 데이터에 적합시킵니다.
  - 인자:
    - `X`: 훈련 데이터 배열 (샘플 수 x 특성 수).
    - `y`: 타겟 레이블 배열 (사용되지 않음).
  - 리턴값: 적합된 `PCA` 인스턴스.
2. fit\_transform(X, y=None):
  - 설명: 모델을 적합시키고 데이터를 변환합니다.
  - 인자:
    - `X`: 입력 데이터 배열.
    - `y`: 타겟 레이블 배열 (사용되지 않음).
  - 리턴값: 변환된 데이터 배열.
3. transform(X):
  - 설명: 학습된 PCA 모델을 사용하여 데이터를 변환합니다.
  - 인자:
    - `X`: 입력 데이터 배열.
  - 리턴값: 변환된 데이터 배열.
4. inverse\_transform(X):
  - 설명: 주성분 공간에서 원래 특성 공간으로 변환합니다.
  - 인자:
    - `X`: 주성분 공간의 데이터 배열.
  - 리턴값: 원래 특성 공간으로 변환된 데이터 배열.
5. fit\_transform(X, y=None):
  - 설명: 모델을 적합시키고 차원 축소를 수행합니다.
  - 인자:
    - `X`: 입력 데이터 배열.
    - `y`: 타겟 레이블 배열 (사용되지 않음).
  - 리턴값: 변환된 데이터 배열.

### 주요 속성



1. `components_`:  
- 설명: 주성분 축을 나타내는 배열 (주성분 수 x 특성 수). 각 행은 하나의 주성분을 나타냅니다.
2. `explained_variance_`:  
- 설명: 각 주성분의 분산값. 고유값에 해당합니다.
3. `explained_variance_ratio_`:  
- 설명: 각 주성분의 분산 비율. 전체 분산에서 해당 주성분이 차지하는 비율을 나타냅니다.
4. `singular_values_`:  
- 설명: 각 주성분의 특이값.
5. `mean_`:  
- 설명: 각 특성의 평균값.
6. `n_components_`:  
- 설명: 선택된 주성분의 수.
7. `n_features_`:  
- 설명: 입력 데이터의 특성 수.
8. `n_samples_`:  
- 설명: 입력 데이터의 샘플 수.
9. `noise_variance_`:  
- 설명: 노이즈 분산 추정값. 이는 데이터가 모델링된 PCA 차원에 맞지 않을 때 사용됩니다.

### 예제

```
from sklearn.decomposition import PCA
import numpy as np
import matplotlib.pyplot as plt
```

# 데이터 생성

```
np.random.seed(0)
X = np.random.randn(100, 5)
```

# PCA 모델 생성 및 적합

```
pca = PCA(n_components=2)
X_r = pca.fit_transform(X)
```

# 결과 출력

```
print("주성분:\n", pca.components_)
print("설명된 분산 비율:\n", pca.explained_variance_ratio_)
```

# 데이터 시각화

```
plt.scatter(X_r[:, 0], X_r[:, 1])
plt.xlabel('PC 1')
plt.ylabel('PC 2')
plt.title('PCA 결과')
plt.show()
```

이 예제에서는 5차원 데이터셋을 생성하고, 이를 2차원으로 축소하는 PCA 모델을 적합시킵니다.  
주성분과 설명된 분산 비율을 출력하고, 변환된 데이터를 시각화합니다.

### 요약

`PCA` 클래스는 데이터의 차원을 축소하고, 데이터를 이해하고 시각화하는 데 유용한 도구입니다.  
다양한 인자들을 통해 모델의 동작 방식을 설정할 수 있으며, 주요 메서드와 속성을 사용하여 모델을 적합시키고 데이터를 변환하거나 복원할 수 있습니다.  
...

```
print('')
```

In [85]:

```
'''
코드는 주성분 분석(PCA)을 사용하여 주어진 특성 데이터를 변환하는 작업을 수행합니다.

코드 설명:

- PCA 객체를 생성합니다.

- PCA를 적용할 특성의 열 이름을 지정합니다.
 여기서는 'measurement_0'부터 'measurement_17'까지의 열을 대상으로 합니다.

- PCA 모델을 훈련 데이터에 적합시킵니다. `fit` 메서드를 사용하여 주어진 특성 데이터를 분석하고, 주성분을 추출합니다.
- 여기서 `df_prob3_train[X_pca]`은 PCA를 적용할 특성 데이터를 의미합니다.
 이 데이터를 기반으로 PCA 모델이 학습됩니다.

PCA 모델을 적합시킴으로써, 주어진 특성 데이터의 차원을 줄이고, 주성분을 추출하는 과정이 진행됩니다.
이후에는 추출된 주성분을 사용하여 데이터를 변환할 수 있습니다.

코드는 PCA를 사용하여 주어진 특성 데이터를 변환하는 작업을 수행합니다.
'''

from sklearn.decomposition import PCA

pca = PCA(random_state=123)
X_pca = ['measurement_{}'.format(i) for i in range(18)]
pca.fit(df_prob3_train[X_pca])
```

Out[85]: PCA(copy=True, iterated\_power='auto', n\_components=None, random\_state=123,  
svd\_solver='auto', tol=0.0, whiten=False)

```
In [86]: '''
`pca.explained_variance_ratio_`는 PCA를 통해 추출된 각 주성분이 전체 분산 중 설명하는 비율을 나타내는 속성입니다.
이 비율은 각 주성분이 전체 데이터의 변동성을 얼마나 잘 설명하는지를 나타냅니다.

주성분은 분산이 큰 순서대로 정렬되므로, 첫 번째 주성분이 가장 많은 분산을 설명하고,
그 다음으로 두 번째 주성분이 그 다음으로 많은 분산을 설명합니다.
이 속성은 주성분의 설명력을 이해하고 데이터의 차원을 축소하기 위해 어떤 주성분을 선택해야 하는지 결정하는 데 도움이 됩니다.

`pca.explained_variance_ratio_`의 각 요소는 주성분별로 설명하는 분산의 비율을 나타내며, 배열의 형태로 반환됩니다.
'''

pca.explained_variance_ratio_
```

```
Out[86]: array([0.10311277, 0.07882566, 0.06974445, 0.06462319, 0.0592398 ,
 0.05833005, 0.05666113, 0.05606465, 0.05591772, 0.05511547,
 0.05474842, 0.05422689, 0.05398036, 0.04862636, 0.04521927,
 0.03931509, 0.03775836, 0.00849036])
```

```
In [87]: '''
`pca.explained_variance_`는 PCA를 통해 추출된 각 주성분이 설명하는 분산의 크기를 나타내는 속성입니다.
이 값은 각 주성분이 데이터의 분산을 얼마나 잘 설명하는지를 나타냅니다.

주성분은 분산이 큰 순서대로 정렬되므로, 첫 번째 주성분이 가장 많은 분산을 설명하고,
그 다음으로 두 번째 주성분이 그 다음으로 많은 분산을 설명합니다.
이 속성은 각 주성분의 중요도를 파악하고 데이터의 차원을 축소할 때 어떤 주성분을 선택해야 하는지 판단하는 데 도움이 됩니다.

`pca.explained_variance_`의 각 요소는 주성분별로 설명하는 분산의 크기를 나타내며, 배열의 형태로 반환됩니다.
'''

explained_variance_ = cov. matrix eigen values 입니다.
pca.explained_variance_
```

```
Out[87]: array([1.85613793, 1.41894459, 1.25547315, 1.1632852 , 1.06637856,
 1.05000212, 1.01995967, 1.00922255, 1.00657757, 0.99213623,
 0.98552903, 0.97614082, 0.97170312, 0.87532548, 0.81399432,
 0.70771288, 0.67969009, 0.15283534])
```

```
In [88]: '''
`np.linalg.eigh` 함수는 `NumPy` 라이브러리의 선형대수 모듈(`numpy.linalg`)에 포함되어 있으며,
대칭 또는 에르미트 행렬(복소수 행렬)의 고유값과 고유벡터를 계산하는 데 사용됩니다.
```

```

함수의 인자
1. a:
 - 유형: (N, N) 배열_like
 - 설명: 고유값과 고유벡터를 구하려는 실수 또는 복소수 대칭 행렬(에르미트 행렬).
2. UPLO:
 - 유형: 문자열 {'L', 'U'} (기본값은 'L')
 - 설명: 행렬의 어느 삼각 부분을 사용할지 지정합니다.
 - 'L': 하삼각 부분을 사용합니다.
 - 'U': 상삼각 부분을 사용합니다.

리턴값
1. w:
 - 유형: (N,) ndarray
 - 설명: 행렬 `a`의 고유값을 오름차순으로 정렬한 배열.
2. v:
 - 유형: (N, N) ndarray
 - 설명: 행렬 `a`의 고유벡터를 열 벡터로 갖는 배열. 즉, `a @ v[:, i] = w[i] * v[:, i]`가 성립합니다.

주요 속성 및 메서드
`np.linalg.eigh` 함수는 함수 자체이므로 별도의 속성이나 메서드는 없습니다.
대신, 고유값 및 고유벡터를 포함하는 배열을 반환하며, 이러한 배열은 `NumPy` 배열의 속성과 메서드를 사용합니다.

요약
`np.linalg.eigh` 함수는 대칭 또는 에르미트 행렬의 고유값과 고유벡터를 계산하는 데 사용됩니다.
입력 인자로 행렬과 행렬의 어느 삼각 부분을 사용할지를 지정할 수 있으며, 반환값으로 고유값과 고유벡터를 반환합니다.
고유값은 오름차순으로 정렬되며, 고유벡터는 각 열에 해당하는 고유벡터를 포함합니다.
이를 통해 행렬의 스펙트럼 분석과 같은 다양한 선형대수 문제를 해결할 수 있습니다.
'''

print('')

```

```

In [89]: '''
아래 예제에서는 3x3 대칭 행렬 `A`의 고유값과 고유벡터를 계산합니다.
결과를 출력하고, 고유벡터와 고유값을 검증하여 결과가 올바른지 확인합니다.
'''

import numpy as np

대칭 행렬 (실수)
A = np.array([[1, 2, 3],
 [2, 4, 5],

```

```
[3, 5, 6]])

대칭 행렬의 고유값과 고유벡터 계산
w, v = np.linalg.eigh(A)

고유값 출력
print("고유값:")
print(w)
print('-----')

고유벡터 출력
print("고유벡터:")
print(v)
print('-----')

고유벡터와 고유값을 검증
for i in range(len(w)):
 eig_val = w[i]
 eig_vec = v[:, i]
 print(f"\n고유값 {eig_val}에 대한 고유벡터 검증:")
 print(np.allclose(np.dot(A, eig_vec), eig_val * eig_vec)) # True이면 검증 성공
 print('-----')
```

고유값:

```
[-0.51572947 0.17091519 11.34481428]
```

고유벡터:

```
[[-0.73697623 0.59100905 -0.32798528]
 [-0.32798528 -0.73697623 -0.59100905]
 [0.59100905 0.32798528 -0.73697623]]
```

고유값 -0.5157294715892581에 대한 고유벡터 검증:

True

고유값 0.17091518882717524에 대한 고유벡터 검증:

True

고유값 11.344814282762076에 대한 고유벡터 검증:

True

In [90]:

```
'''
`np.argsort` 함수는 `NumPy` 라이브러리에서 제공하는 함수로, 배열을 정렬하는 인덱스를 반환합니다.
이를 통해 원래 배열을 정렬된 순서로 재배열할 수 있습니다.

함수의 인자
1. a:
 - 유형: array_like
 - 설명: 정렬할 입력 배열.
2. axis:
 - 유형: 정수 또는 None (기본값은 -1)
 - 설명: 정렬할 축을 지정합니다.
 - -1: 마지막 축을 기준으로 정렬합니다.
 - None: 배열을 평탄화한 후 정렬합니다.
3. kind:
 - 유형: {'quicksort', 'mergesort', 'heapsort', 'stable'} (기본값은 'quicksort')
 - 설명: 정렬에 사용할 알고리즘을 지정합니다.
 - 'quicksort': 빠르고 평균적으로 좋은 성능을 보이지만 최악의 경우 느릴 수 있습니다.
 - 'mergesort': 안정적인 정렬로, 데이터의 순서를 보존합니다.
 - 'heapsort': 느리지만 메모리 사용량이 적습니다.
 - 'stable': 안정적인 정렬로, 데이터의 순서를 보존합니다.
4. order:
```

- 유형: 문자열 또는 문자열의 리스트 (기본값은 `None`)
- 설명: 필드 이름을 지정하여 구조화된 배열의 특정 필드를 정렬합니다.

### 리턴값

- `indices`:

- 유형: `ndarray`
- 설명: 입력 배열 ``a``를 정렬된 순서로 만들기 위한 인덱스 배열. 이 인덱스를 사용하여 원래 배열을 정렬할 수 있습니다.

### 주요 속성 및 메서드

``np.argsort`` 함수 자체에는 별도의 속성이나 메서드는 없지만, 반환된 인덱스 배열에는 ``NumPy`` 배열의 속성과 메서드를 사용할 수 있습니다.

### 요약

``np.argsort`` 함수는 배열을 정렬하는 데 필요한 인덱스를 반환하는 함수입니다. 이를 통해 원래 배열을 정렬된 순서로 재배열할 수 있으며, 다양한 정렬 알고리즘을 선택할 수 있습니다. 이 함수는 다차원 배열에 대해서도 사용할 수 있으며, 특정 축을 기준으로 정렬할 수 있습니다. 반환된 인덱스 배열은 원래 배열의 순서를 재정렬하는 데 사용됩니다.

...

...

아래 예제에서는 1차원 배열과 2차원 배열의 정렬 인덱스를 구하고, 해당 인덱스를 사용하여 배열을 정렬합니다. ``np.take_along_axis``를 사용하면 다차원 배열의 특정 축을 기준으로 정렬할 수 있습니다.

...

```
import numpy as np
```

```
1차원 배열 생성
```

```
arr = np.array([3, 1, 2])
```

```
배열을 정렬하는 인덱스 반환
```

```
indices = np.argsort(arr)
```

```
print("정렬 인덱스:", indices)
```

```
print('-----')
```

```
인덱스를 사용하여 원래 배열 정렬
```

```
sorted_arr = arr[indices]
```

```
print("정렬된 배열:", sorted_arr)
```

```
print('-----')
```

```
다차원 배열 예제
```

```
arr_2d = np.array([[3, 1, 2], [6, 4, 5]])
```

```
마지막 축(axis=-1)을 기준으로 정렬하는 인덱스 반환
indices_2d = np.argsort(arr_2d, axis=-1)

print("다차원 배열의 정렬 인덱스:\n", indices_2d)
print('-----')

인덱스를 사용하여 다차원 배열 정렬
sorted_arr_2d = np.take_along_axis(arr_2d, indices_2d, axis=-1)
print("정렬된 다차원 배열:\n", sorted_arr_2d)
```

정렬 인덱스: [1 2 0]

-----

정렬된 배열: [1 2 3]

-----

다차원 배열의 정렬 인덱스:

```
[[1 2 0]
 [1 2 0]]
```

-----

정렬된 다차원 배열:

```
[[1 2 3]
 [4 5 6]]
```

In [91]:

```
'''
이 코드는 주어진 입력 디자인 행렬의 공분산 행렬에 대해 고유값 분해를 수행합니다.
- `np.linalg.eigh()` 함수를 사용하여 주어진 공분산 행렬에 대한 고유값 분해를 수행합니다.
 입력으로는 입력 디자인 행렬의 공분산 행렬이 들어갑니다.
- 입력 행렬은 데이터의 특성을 나타내는 열로 구성되며, 각 특성의 평균이 0이 되도록 평균을 빼준 후, 공분산을 계산합니다.
 이렇게 함으로써 입력 행렬의 공분산 행렬을 구합니다.
- 공분산 행렬의 고유값과 고유벡터를 반환합니다. `ev`는 고유값을, `emat`은 고유벡터로 구성된 행렬을 반환합니다.

- 고유값을 내림차순으로 정렬하여 출력합니다. `np.argsort(-ev)`를 사용하여 고유값을 내림차순으로 정렬한 후,
 해당 인덱스로 고유값을 선택합니다. 이를 통해 고유값의 크기와 중요도를 확인할 수 있습니다.
'''

입력 디자인 행렬의 공분산행렬에서 고윳값 분해를 해보도록 하겠습니다.
ev, emat = np.linalg.eigh(
 (df_prob3_train[X_pca] - df_prob3_train[X_pca].mean()).pipe(lambda x: x.T.dot(x)) / (len(df_prob3_train) - 1)
)

emat은 고윳값에 해당하는 eigen vector(column vector 형태)로
구성된 행렬: 고유행렬입니다.
```



```
고윳값만 출력해봅니다.
ev[np.argsort(-ev)]
```

```
Out[91]: array([1.85613793, 1.41894459, 1.25547315, 1.1632852 , 1.06637856,
 1.05000212, 1.01995967, 1.00922255, 1.00657757, 0.99213623,
 0.98552903, 0.97614082, 0.97170312, 0.87532548, 0.81399432,
 0.70771288, 0.67969009, 0.15283534])
```

```
In [92]: '''
코드는 주성분 분석(PCA)을 통해 추출된 주성분(eigenvector)들을 데이터프레임으로 정리하는 작업을 수행합니다.
- `pca.components_` 속성은 PCA 모델에서 추출된 주성분(eigenvector)을 나타냅니다.
 각 행은 하나의 주성분을 나타내며, 각 열은 특성(변수)을 나타냅니다.
 이러한 구조는 각 주성분이 각 특성에 대한 가중치를 나타내는 형태입니다.
- `pd.DataFrame()` 함수를 사용하여 주성분(eigenvector)을 데이터프레임으로 변환합니다.
- `pca.components_`를 행렬의 행으로 사용하여 데이터프레임을 생성합니다.
 각 주성분을 행으로 나타내므로, 각 주성분은 데이터프레임의 각 행에 대응됩니다.
- `index` 매개변수를 사용하여 각 주성분에 대해 'ev_0', 'ev_1', ..., 'ev_17'과 같은 이름을 지정합니다.
 이는 주성분을 식별하기 위한 인덱스입니다.
- `columns` 매개변수를 사용하여 주성분에 해당하는 특성(변수) 이름을 지정합니다.
 여기서는 주성분이 사용된 원래 특성(변수)의 이름을 사용합니다.
```

결과적으로, `df\_eigen` 데이터프레임은 각 주성분의 구성 요소를 특성별로 보여줍니다.  
 각 주성분이 어떤 특성에 더 큰 가중치를 부여하는지를 확인할 수 있습니다.

```
'''
```

```
PCA에는 고유행렬의 Transpose가 pca.components_ 에 있습니다.
(* emat과는 요소의 부호가 반전돼 있을 수 있습니다.)
행은 ev_i i번째 주성분의 eigen vector입니다.
그리고 열은 각 eigen vector에 대응되는 변수입니다.
행렬의 값의 절대값 통해 각 주성분에서 지니는 변수의 비중을 파악할 수 있습니다.

df_eigen = pd.DataFrame(
 pca.components_, index = ['ev_{}'.format(i) for i in range(18)], columns=X_pca
)

df_eigen
```

Out[92]:

|       | measurement_0 | measurement_1 | measurement_2 | measurement_3 | measurement_4 | measurement_5 | measurement_6 | measurement_7 | measurement_8 |
|-------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| ev_0  | 0.022883      | -0.019582     | -0.015097     | 0.050394      | 0.204060      | 0.371117      | 0.190529      | 0.268949      | 0.427         |
| ev_1  | 0.526517      | -0.562470     | -0.103690     | 0.016855      | -0.042447     | 0.036740      | 0.000815      | 0.021352      | -0.059        |
| ev_2  | -0.186391     | 0.074810      | 0.142814      | 0.003889      | 0.052836      | 0.007751      | 0.029356      | -0.016607     | 0.031         |
| ev_3  | -0.352613     | 0.330151      | -0.080426     | 0.003231      | 0.040330      | 0.037848      | -0.021599     | -0.027172     | -0.029        |
| ev_4  | 0.029019      | -0.105361     | 0.113744      | -0.060397     | 0.193537      | 0.099383      | -0.088444     | -0.137312     | -0.036        |
| ev_5  | -0.267111     | -0.062494     | 0.659857      | 0.210428      | -0.196963     | 0.108322      | 0.070451      | 0.241535      | -0.197        |
| ev_6  | 0.001058      | 0.109837      | -0.244030     | 0.502298      | 0.536801      | 0.234953      | -0.118318     | -0.203609     | -0.392        |
| ev_7  | -0.060505     | -0.035790     | 0.260950      | -0.337998     | 0.205978      | 0.106405      | -0.653943     | -0.359636     | 0.323         |
| ev_8  | 0.007082      | -0.042138     | 0.110591      | -0.228984     | 0.021452      | 0.611846      | 0.240576      | -0.283762     | -0.246        |
| ev_9  | -0.007179     | 0.036969      | -0.168208     | 0.280662      | 0.000296      | 0.046954      | -0.516073     | 0.477417      | 0.085         |
| ev_10 | -0.030856     | -0.012127     | 0.028821      | 0.347170      | 0.109553      | -0.312179     | 0.306829      | -0.438902     | 0.476         |
| ev_11 | -0.055118     | 0.073923      | -0.164065     | -0.564637     | 0.439765      | -0.237465     | 0.182878      | 0.285884      | -0.134        |
| ev_12 | 0.054462      | -0.208127     | 0.460676      | 0.099347      | 0.544672      | -0.309550     | 0.049135      | 0.146230      | -0.095        |
| ev_13 | 0.126217      | -0.151632     | 0.127489      | 0.037287      | -0.042637     | 0.078385      | -0.132493     | 0.011286      | 0.011         |
| ev_14 | 0.063697      | 0.013026      | 0.124078      | -0.002254     | 0.014269      | 0.013684      | 0.025779      | 0.028368      | -0.052        |
| ev_15 | -0.187835     | -0.001980     | -0.040303     | -0.006651     | -0.010939     | -0.019940     | 0.005683      | -0.005373     | 0.011         |
| ev_16 | 0.651904      | 0.681946      | 0.269061      | 0.001451      | -0.005318     | -0.005411     | 0.005299      | 0.020747      | 0.009         |
| ev_17 | 0.004430      | 0.003104      | 0.000078      | -0.028615     | -0.201527     | -0.371899     | -0.188291     | -0.269484     | -0.434        |

In [93]:

'''

코드는 데이터프레임 `df\_eigen`의 첫 번째 행에서 절댓값이 가장 큰 값을 가지는 열(특성)의 이름을 반환합니다.

### 코드 설명:

- `df\_eigen.iloc[0]`: `df\_eigen` 데이터프레임의 첫 번째 행을 선택합니다. 이 행은 첫 번째 주성분(eigenvector)에 해당합니다.
- `.abs()`: 선택한 행의 각 원소(가중치)에 대해 절댓값을 취합니다. 이는 각 특성(변수)의 중요도를 나타냅니다.
- `.idxmax()`: 절댓값이 가장 큰 값을 가지는 열(특성)의 이름을 반환합니다.

따라서 코드는 첫 번째 주성분에서 가장 중요한 특성(변수)을 결정하는 데 사용됩니다.  
가장 중요한 특성은 해당 주성분에서 가장 큰 가중치를 가진 특성입니다.

제가 설명한대로 코드는 주어진 데이터프레임에서 첫 번째 행(첫 번째 주성분)에서  
절댓값이 가장 큰 값을 가지는 열(특성)의 이름을 반환합니다.

이는 첫 번째 주성분이 가장 큰 영향을 미치는 특성을 나타냅니다.

...

# 제1 주성분에서 영향도가 가장 높은 속성을 구해봅니다.

```
print('df_eigen.iloc[0]')
```

```
print(df_eigen.iloc[0])
```

```
print('-----')
```

```
print('df_eigen.iloc[0].abs()')
```

```
print(df_eigen.iloc[0].abs())
```

```
print('-----')
```

```
df_eigen.iloc[0].abs().idxmax()
```

```
df_eigen.iloc[0]
measurement_0 0.022883
measurement_1 -0.019582
measurement_2 -0.015097
measurement_3 0.050394
measurement_4 0.204060
measurement_5 0.371117
measurement_6 0.190529
measurement_7 0.268949
measurement_8 0.427064
measurement_9 0.164705
measurement_10 -0.009905
measurement_11 -0.022599
measurement_12 0.002799
measurement_13 -0.033895
measurement_14 0.011526
measurement_15 -0.020182
measurement_16 -0.027936
measurement_17 0.704078
Name: ev_0, dtype: float64
```

```

df_eigen.iloc[0].abs()
measurement_0 0.022883
measurement_1 0.019582
measurement_2 0.015097
measurement_3 0.050394
measurement_4 0.204060
measurement_5 0.371117
measurement_6 0.190529
measurement_7 0.268949
measurement_8 0.427064
measurement_9 0.164705
measurement_10 0.009905
measurement_11 0.022599
measurement_12 0.002799
measurement_13 0.033895
measurement_14 0.011526
measurement_15 0.020182
measurement_16 0.027936
measurement_17 0.704078
Name: ev_0, dtype: float64

```

Out[93]: 'measurement\_17'

In [94]: '''

코드는 주어진 데이터프레임에 대해 주성분 분석(PCA)을 수행하고, 변환된 주성분을 새로운 특성으로 추가하는 작업을 수행합니다.

### 코드 설명:

1. PCA 변환 수행:

- `pca.transform()` 함수를 사용하여 주어진 특성(변수) 데이터 (`df_prob3_train[X_pca]`)를 주성분 축으로 변환합니다.
- 변환된 주성분 데이터는 원래 데이터셋과 동일한 모양의 배열로 반환됩니다.

2. 새로운 특성 추가:

- PCA로 변환된 주성분 데이터를 데이터프레임 형태로 변환하여 새로운 특성으로 추가합니다.
- `pd.DataFrame()` 함수를 사용하여 변환된 주성분 데이터를 데이터프레임으로 변환합니다.  
이 데이터프레임은 주성분을 열(특성)로 가지며, 각 행은 원래 데이터프레임의 인덱스에 해당합니다.
- 변환된 주성분 데이터프레임은 기존 데이터프레임 `df_prob3_train`에 추가됩니다.  
새로운 열 이름은 `'pca_0'`부터 `'pca_17'`까지 순차적으로 지정됩니다.

3. 테스트 데이터에도 적용:

- 테스트 데이터셋에 대해서도 동일한 과정을 수행합니다.
- `pca.transform()` 함수를 사용하여 테스트 데이터를 주성분 축으로 변환한 후, 데이터프레임으로 변환하여 새로운 특성으로 추가합니다.
- `pd.concat()` 함수를 사용하여 원래의 테스트 데이터프레임과 새롭게 추가된 주성분 데이터프레임을 열 방향(`axis=1`)으로 결합합니다.
- 이렇게 하면 테스트 데이터프레임인 `df_prob3_test`에 새로운 주성분 특성이 추가된 `df_prob4_test` 데이터프레임이 생성됩니다.

이러한 작업을 통해 기존의 특성들을 주성분으로 변환하여 데이터의 차원을 축소하고, 새로운 주성분 특성을 추가하여 모델 학습이나 분석에 활용할 수 있습니다.

'''

```
X_comp = ['pca_{}'.format(i) for i in range(18)]
```

```
print('X_comp = ', X_comp)
```

```
print('-----')
```

```
print('df_prob3_test.columns')
```

```
print(df_prob3_test.columns)
```

```
print('-----')
```

```
df_prob4_train = df_prob3_train.copy()
```

```
df_prob4_train[X_comp] = pd.DataFrame(pca.transform(df_prob3_train[X_pca]), index=df_prob3_train.index, columns=X_comp)
```

```
df_prob4_test = pd.concat([
 df_prob3_test,
 pd.DataFrame(pca.transform(df_prob3_test[X_pca]), index=df_prob3_test.index, columns=X_comp)
```

```
], axis=1)

print(df_prob4_test)
print('-----df_prob4_test-----')
print('df_prob4_test.columns')
print(df_prob4_test.columns)
```

```
X_comp = ['pca_0', 'pca_1', 'pca_2', 'pca_3', 'pca_4', 'pca_5', 'pca_6', 'pca_7', 'pca_8', 'pca_9', 'pca_10', 'pca_11', 'pca_12', 'pca_13', 'pca_14', 'pca_15', 'pca_16', 'pca_17']
```

```
df_prob3_test.columns
```

```
Index(['product_code', 'loading', 'attribute_0', 'attribute_1', 'attribute_2',
 'attribute_3', 'measurement_0', 'measurement_1', 'measurement_2',
 'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6',
 'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10',
 'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14',
 'measurement_15', 'measurement_16', 'measurement_17', 'failure',
 'isna_3', 'isna_4', 'isna_5', 'isna_6', 'isna_7', 'isna_8', 'isna_9',
 'isna_10', 'isna_11', 'isna_12', 'isna_13', 'isna_14', 'isna_15',
 'isna_16', 'isna_17', 'na_1', 'na_2'],
 dtype='object')
```

```

 product_code loading attribute_0 attribute_1 attribute_2 \
id
6626 B -0.646778 material_5 material_5 8
6302 B -0.776364 material_5 material_5 8
2068 A 1.220389 material_7 material_8 9
4661 A 0.213270 material_7 material_8 9
10303 B 0.694332 material_5 material_5 8
...
7731 B -0.408690 material_5 material_5 8
13022 C -0.652178 material_7 material_8 5
9559 B -1.067932 material_5 material_5 8
10604 C 0.160305 material_7 material_8 5
22261 E -0.334127 material_7 material_6 6
```

```
 attribute_3 measurement_0 measurement_1 measurement_2 \
id
6626 8 -0.779343 -0.210286 0.912362
6302 8 -0.308363 0.509078 1.216412
2068 5 1.104580 -1.409226 -0.911937
4661 5 -0.072872 -1.169438 -0.607887
10303 8 -0.072872 -0.689862 0.304263
...
7731 8 -0.779343 -0.450074 0.912362
13022 8 0.869089 0.748866 -0.607887
9559 8 0.162618 -1.409226 1.520462
10604 8 -0.543853 0.029502 0.000213
22261 9 -0.543853 -1.169438 1.824512
```

|       | measurement_3 | ... | pca_8     | pca_9     | pca_10    | pca_11    | pca_12    | \ |
|-------|---------------|-----|-----------|-----------|-----------|-----------|-----------|---|
| id    |               | ... |           |           |           |           |           |   |
| 6626  | 0.694817      | ... | -0.435347 | -2.451195 | -0.212859 | -0.474401 | -0.152049 |   |
| 6302  | -1.793689     | ... | 0.600191  | -1.158239 | -1.832178 | -1.066241 | -1.273782 |   |
| 2068  | 0.082494      | ... | 0.442147  | -1.731923 | -0.481832 | 0.195241  | 0.004504  |   |
| 4661  | 0.287607      | ... | 1.318845  | -0.761127 | 1.713534  | -1.731517 | 1.039950  |   |
| 10303 | 1.477063      | ... | 0.507550  | 0.996317  | 1.105341  | -0.423133 | 1.067850  |   |
| ...   | ...           | ... | ...       | ...       | ...       | ...       | ...       |   |
| 7731  | -1.419659     | ... | 0.364147  | -4.708135 | 0.912742  | -1.173427 | 0.986579  |   |
| 13022 | 0.575168      | ... | -1.352617 | 0.167372  | 0.789154  | 0.945992  | 0.483356  |   |
| 9559  | -2.331609     | ... | 0.816120  | -0.143480 | -0.063370 | 0.819752  | 1.042688  |   |
| 10604 | 0.240351      | ... | 1.056565  | 0.938345  | 0.799403  | 0.887117  | 0.755378  |   |
| 22261 | 0.635495      | ... | -0.166010 | -1.036776 | 0.399477  | -1.632600 | 0.589810  |   |

|       | pca_13    | pca_14    | pca_15    | pca_16    | pca_17    |
|-------|-----------|-----------|-----------|-----------|-----------|
| id    |           |           |           |           |           |
| 6626  | 0.415381  | -0.211874 | 1.251870  | -0.318274 | -0.142292 |
| 6302  | 0.121801  | 1.330900  | -0.396394 | 0.157529  | -0.862240 |
| 2068  | -0.244213 | -2.249243 | 0.099636  | -0.293499 | -0.412578 |
| 4661  | -2.236054 | 0.923431  | -1.178613 | -1.247759 | 0.304439  |
| 10303 | 0.665243  | -1.121216 | 0.637357  | -0.238106 | 0.043725  |
| ...   | ...       | ...       | ...       | ...       | ...       |
| 7731  | -0.724821 | -0.086433 | -0.417077 | -0.829020 | -1.007561 |
| 13022 | 0.622101  | 0.088954  | 0.893792  | 1.069329  | -0.126067 |
| 9559  | 0.731836  | 1.495723  | 1.750813  | -0.153709 | -0.432824 |
| 10604 | 0.129232  | 0.340020  | -0.807974 | -0.379527 | -0.425447 |
| 22261 | 0.615239  | 0.705375  | -0.302151 | -0.880253 | 0.065533  |

[4292 rows x 60 columns]

-----df\_prob4\_test-----

df\_prob4\_test.columns

```
Index(['product_code', 'loading', 'attribute_0', 'attribute_1', 'attribute_2',
 'attribute_3', 'measurement_0', 'measurement_1', 'measurement_2',
 'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6',
 'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10',
 'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14',
 'measurement_15', 'measurement_16', 'measurement_17', 'failure',
 'isna_3', 'isna_4', 'isna_5', 'isna_6', 'isna_7', 'isna_8', 'isna_9',
 'isna_10', 'isna_11', 'isna_12', 'isna_13', 'isna_14', 'isna_15',
 'isna_16', 'isna_17', 'na_1', 'na_2', 'pca_0', 'pca_1', 'pca_2',
 'pca_3', 'pca_4', 'pca_5', 'pca_6', 'pca_7', 'pca_8', 'pca_9', 'pca_10',
```



```
'pca_11', 'pca_12', 'pca_13', 'pca_14', 'pca_15', 'pca_16', 'pca_17'],
dtype='object')
```

In [95]: # 위 코드 실행 결과

```
...
X_comp = ['pca_0', 'pca_1', 'pca_2', 'pca_3', 'pca_4', 'pca_5', 'pca_6', 'pca_7', 'pca_8', 'pca_9', 'pca_10', 'pca_11', 'pca_12']

df_prob3_test.columns
Index(['product_code', 'loading', 'attribute_0', 'attribute_1', 'attribute_2',
 'attribute_3', 'measurement_0', 'measurement_1', 'measurement_2',
 'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6',
 'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10',
 'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14',
 'measurement_15', 'measurement_16', 'measurement_17', 'failure',
 'isna_3', 'isna_4', 'isna_5', 'isna_6', 'isna_7', 'isna_8', 'isna_9',
 'isna_10', 'isna_11', 'isna_12', 'isna_13', 'isna_14', 'isna_15',
 'isna_16', 'isna_17', 'na_1', 'na_2'],
 dtype='object')

 product_code loading attribute_0 attribute_1 attribute_2 \
id
6626 B -0.646778 material_5 material_5 8
6302 B -0.776364 material_5 material_5 8
2068 A 1.220389 material_7 material_8 9
4661 A 0.213270 material_7 material_8 9
10303 B 0.694332 material_5 material_5 8
...
7731 B -0.408690 material_5 material_5 8
13022 C -0.652178 material_7 material_8 5
9559 B -1.067932 material_5 material_5 8
10604 C 0.160305 material_7 material_8 5
22261 E -0.334127 material_7 material_6 6

 attribute_3 measurement_0 measurement_1 measurement_2 \
id
6626 8 -0.779343 -0.210286 0.912362
6302 8 -0.308363 0.509078 1.216412
2068 5 1.104580 -1.409226 -0.911937
4661 5 -0.072872 -1.169438 -0.607887
10303 8 -0.072872 -0.689862 0.304263
...
7731 8 -0.779343 -0.450074 0.912362
```

```

13022 8 0.869089 0.748866 -0.607887
9559 8 0.162618 -1.409226 1.520462
10604 8 -0.543853 0.029502 0.000213
22261 9 -0.543853 -1.169438 1.824512

 measurement_3 ... pca_8 pca_9 pca_10 pca_11 pca_12 \
id ...
6626 0.694817 ... -0.435347 -2.451195 -0.212859 -0.474401 -0.152049
6302 -1.793689 ... 0.600191 -1.158239 -1.832178 -1.066241 -1.273782
2068 0.082494 ... 0.442147 -1.731923 -0.481832 0.195241 0.004504
4661 0.287607 ... 1.318845 -0.761127 1.713534 -1.731517 1.039950
10303 1.477063 ... 0.507550 0.996317 1.105341 -0.423133 1.067850
...
7731 -1.419659 ... 0.364147 -4.708135 0.912742 -1.173427 0.986579
13022 0.575168 ... -1.352617 0.167372 0.789154 0.945992 0.483356
9559 -2.331609 ... 0.816120 -0.143480 -0.063370 0.819752 1.042688
10604 0.240351 ... 1.056565 0.938345 0.799403 0.887117 0.755378
22261 0.635495 ... -0.166010 -1.036776 0.399477 -1.632600 0.589810

 pca_13 pca_14 pca_15 pca_16 pca_17
id
6626 0.415381 -0.211874 1.251870 -0.318274 -0.142292
6302 0.121801 1.330900 -0.396394 0.157529 -0.862240
2068 -0.244213 -2.249243 0.099636 -0.293499 -0.412578
4661 -2.236054 0.923431 -1.178613 -1.247759 0.304439
10303 0.665243 -1.121216 0.637357 -0.238106 0.043725
...
7731 -0.724821 -0.086433 -0.417077 -0.829020 -1.007561
13022 0.622101 0.088954 0.893792 1.069329 -0.126067
9559 0.731836 1.495723 1.750813 -0.153709 -0.432824
10604 0.129232 0.340020 -0.807974 -0.379527 -0.425447
22261 0.615239 0.705375 -0.302151 -0.880253 0.065533

[4292 rows x 60 columns]
-----df_prob4_test-----
df_prob4_test.columns
Index(['product_code', 'loading', 'attribute_0', 'attribute_1', 'attribute_2',
 'attribute_3', 'measurement_0', 'measurement_1', 'measurement_2',
 'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6',
 'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10',
 'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14',
 'measurement_15', 'measurement_16', 'measurement_17', 'failure',

```

```
'isna_3', 'isna_4', 'isna_5', 'isna_6', 'isna_7', 'isna_8', 'isna_9',
'isna_10', 'isna_11', 'isna_12', 'isna_13', 'isna_14', 'isna_15',
'isna_16', 'isna_17', 'na_1', 'na_2', 'pca_0', 'pca_1', 'pca_2',
'pca_3', 'pca_4', 'pca_5', 'pca_6', 'pca_7', 'pca_8', 'pca_9', 'pca_10',
'pca_11', 'pca_12', 'pca_13', 'pca_14', 'pca_15', 'pca_16', 'pca_17'],
dtype='object')
...
print('')
```

In [96]: '''

코드는 다음을 수행합니다:

#### 1. 전체 분산 계산:

- 주어진 특성(변수)들의 전체 분산을 계산합니다. 이는 PCA를 적용하기 전의 원본 데이터셋의 분산입니다.

#### 2. PCA를 이용한 차원 축소 및 분산 계산:

- 18부터 1까지의 주성분 개수에 대해 반복합니다.
  - `PCA()` 함수를 사용하여 주어진 주성분 개수(`n\_components`)에 따른 PCA 모델을 생성합니다.
  - `fit()` 메서드를 사용하여 PCA 모델을 주어진 특성 데이터에 맞춥니다.
  - `transform()` 메서드를 사용하여 주어진 특성 데이터를 주성분 축으로 변환합니다.
  - PCA를 적용한 후의 주성분 개수에 따른 새로운 데이터를 `X\_tr`에 할당합니다.
  - 주어진 주성분 개수에 따른 설명된 분산의 합을 구하고, 이를 전체 분산에서 뺀 값을 출력합니다.
  - `inverse\_transform()` 메서드를 사용하여 주성분으로부터 다시 원래 차원으로 복원한 후, 이를 원본 데이터와 비교하여 재구성 오차의 분산을 계산합니다.
- 이는 PCA가 차원 축소를 얼마나 효과적으로 수행했는지를 나타냅니다.

코드는 PCA를 사용하여 데이터의 차원을 줄이면서 설명된 분산을 최대한 유지하는 데 얼마나 효과적인지를 평가하는 데 사용됩니다.

코드는 다음을 수행합니다:

#### 1. 전체 분산 계산:

- `total\_var` 변수에는 주어진 특성 `X\_pca`에 대한 전체 분산이 저장됩니다.

#### 2. PCA를 이용한 차원 축소 및 재구성 오차 계산:

- `for` 루프는 18에서 1까지 거꾸로 반복됩니다.
  - 각 반복에서 `PCA()` 함수를 사용하여 지정된 주성분 개수 `i`에 대한 PCA 모델을 만듭니다.
  - `fit()` 메서드를 사용하여 PCA 모델을 훈련 데이터에 맞춥니다.
  - `transform()` 메서드를 사용하여 훈련 데이터를 주성분 축으로 변환하여 새로운 데이터를 생성합니다.
  - `pca\_exp.explained\_variance\_.sum()`은 주어진 주성분 개수에 대한 설명된 분산의 합을 계산합니다.
- 전체 분산에서 이 값을 뺀 것은 설명되지 않는 분산의 합입니다.
- `pca\_exp.inverse\_transform(X\_tr)`을 사용하여 주성분으로부터 원래 차원으로 재구성된 데이터를 얻습니다.

- 원본 데이터셋에서 재구성된 데이터를 빼고 `.var().sum()`을 통해 재구성 오차의 분산을 계산합니다.  
이 오차는 PCA로 인해 잃어버린 정보의 양을 나타냅니다.

코드는 PCA를 사용하여 데이터의 차원을 줄이면서 설명된 분산을 최대한 유지하는 데 얼마나 효과적인지를 평가하는 데 사용됩니다.  
코드는 PCA를 사용하여 데이터의 차원을 줄일 때 얼마나 많은 정보가 유지되는지와  
PCA에 의해 손실된 정보의 양을 계산하여 PCA의 효과를 평가합니다.

```
'''
```

```
참고 pca.inverse_transform를 통해 PCA의 역변환을 할 수 있습니다.
이를 통해 원래 변수로 변환할 수 있습니다.
복원하면 차원축소 과정에서 제외된 성분이 지닌 변동성을 잃어버리게 됩니다.
```

```
print('df_prob4_train[X_pca].var()')
print(df_prob4_train[X_pca].var())
print('-----')
```

```
total_var = df_prob4_train[X_pca].var().sum()
print("Var: {}".format(total_var))
print('-----')
```

```
for i in range(18, 0, -1):
 pca_exp = PCA(n_components=i, random_state=123)
 pca_exp.fit(df_prob3_train[X_pca])

 X_tr = pca_exp.transform(df_prob4_train[X_pca])
 print(
 i, total_var - pca_exp.explained_variance_.sum(),
 (pca_exp.inverse_transform(X_tr) - df_prob4_train[X_pca]).var().sum()
)
```

```
df_prob4_train[X_pca].var()
measurement_0 1.000058
measurement_1 1.000058
measurement_2 1.000058
measurement_3 1.000058
measurement_4 1.000058
measurement_5 1.000058
measurement_6 1.000058
measurement_7 1.000058
measurement_8 1.000058
measurement_9 1.000058
measurement_10 1.000058
measurement_11 1.000058
measurement_12 1.000058
measurement_13 1.000058
measurement_14 1.000058
measurement_15 1.000058
measurement_16 1.000058
measurement_17 1.000058
dtype: float64
```

```

Var: 18.001048645499374

```

```
18 -1.8118839761882555e-13 2.0618907041729574e-29
17 0.15283534058138315 0.15283534058156684
16 0.8325254315599793 0.8325254315601601
15 1.540238311500751 1.5402383115009306
14 2.3542326347710176 2.354232634771195
13 3.2295581105971323 3.2295581105973223
12 4.201261233029362 4.201261233029561
11 5.1774020506765925 5.177402050676784
10 6.162931078706899 6.162931078707116
9 7.155067310617856 7.155067310618055
8 8.1616448809485 8.161644880948693
7 9.170867563764425 9.170867450007886
6 10.19621240601028 10.19475152955464
5 11.248118119364563 11.24623355431854
4 12.323356201212567 12.31979955975568
3 13.49045305697423 13.484471574782997
2 14.730648352824165 14.72897824831269
1 16.14491715645344 16.144913730254192
```

## 단계 4-3

초기에 loading을 입력 변수로 하여 prob4\_train을 학습하고, prob4\_test에 대한 성능을 측정한다.

여기에 pca\_0에서 pca\_17까지 입력 변수를 하나씩 추가 하면서,

즉 분산 설명율이 높은 순으로 컴포넌트를 하나씩 추가하여 prob4\_train를 학습하고 prob4\_test의 성능을 측정 했을 때,

최적의 성능을 보인 컴포넌트들의 분산 설명율의 합을 B라고 한다. (만일 없다면 B = 0이다.)

입력 변수: 설명 참고

대상 변수: failure

성능 지표: AUC(area under of ROC curve)

### 함수가이드

`sklearn.linear_model.LogisticRegression, solver='lbfgs'`

In [97]:

```
'''
이 코드는 로지스틱 회귀(Logistic Regression)를 사용하여, 모델이 데이터의 `failure` 클래스를 예측하는
성능을 평가하는 과정을 보여줍니다. 구체적으로, 분산 설명율이 높은 순서로 PCA 컴포넌트를 하나씩 추가하여 변수를 구성하고,
각 단계에서 모델의 테스트 및 훈련 성능을 평가합니다.

1. 변수 설정:
 X_comp = ['pca_{}'.format(i) for i in range(18)]
 - `X_comp` 리스트는 `pca_0`, `pca_1`, ..., `pca_17`로 구성된 문자열 리스트를 생성합니다.
 이는 PCA로 변환된 데이터의 컴포넌트들을 의미합니다.
2. 리스트 초기화:
 X_lr = list()
 X_lr_list = list()
 - `X_lr`와 `X_lr_list`는 비어 있는 리스트로 초기화됩니다. `X_lr`은 모델 학습에 사용할 변수들을 저장하고,
 `X_lr_list`는 각 단계에서 사용된 변수를 저장합니다.
3. 로지스틱 회귀 모델 초기화:
 clf_lr = LogisticRegression(solver='lbfgs')
```

```

- `clf_lr`는 로지스틱 회귀 모델을 `lbfgs` 솔버를 사용하여 초기화합니다.
 `lbfgs`는 L-BFGS 알고리즘으로, 수치 최적화에 사용됩니다.
4. 성능 저장 리스트 초기화:
 test_scores = list()
 train_scores = list()
 - `test_scores`와 `train_scores` 리스트를 초기화합니다. 각각 테스트 및 훈련 데이터에서의 ROC AUC 점수를 저장합니다.
5. 컴포넌트 추가 및 모델 학습:
 for i in ['loading'] + X_comp:
 - `['loading'] + X_comp`는 `['loading', 'pca_0', 'pca_1', ..., 'pca_17']` 리스트를 생성합니다.
 첫 번째 요소인 `loading`을 포함한 변수들을 순차적으로 추가합니다.
6. 변수 추가:
 X_lr.append(i)
 - 현재 변수 `i`를 `X_lr` 리스트에 추가합니다.
7. 모델 학습 및 예측:
 clf_lr.fit(df_prob4_train[X_lr], df_prob4_train['failure'])
 - `X_lr`에 포함된 변수들을 사용하여 로지스틱 회귀 모델을 학습합니다.
 `df_prob4_train` 데이터프레임에서 해당 변수들과 타겟 변수인 `failure`를 사용합니다.
8. 테스트 데이터 성능 평가:
 test_scores.append(
 roc_auc_score(df_prob4_test['failure'], clf_lr.predict_proba(df_prob4_test[X_lr]))[:, 1])
)
 - 학습된 모델을 사용하여 테스트 데이터의 ROC AUC 점수를 계산하고, `test_scores` 리스트에 추가합니다.
 `predict_proba` 메서드는 클래스 확률을 반환하고, `[:, 1]`은 양성 클래스의 확률을 선택합니다.
9. 훈련 데이터 성능 평가:
 train_scores.append(
 roc_auc_score(df_prob4_train['failure'],
 clf_lr.predict_proba(df_prob4_train[X_lr]))[:, 1])
)
 - 학습된 모델을 사용하여 훈련 데이터의 ROC AUC 점수를 계산하고, `train_scores` 리스트에 추가합니다.
10. 결과 출력:
 print(X_lr[-1], test_scores[-1], train_scores[-1])
 - 현재 추가된 변수 (`X_lr`의 마지막 요소)와 해당 변수 조합에서의 테스트 및 훈련 데이터 성능 점수를 출력합니다.
11. 변수 리스트 저장:
 X_lr_list.append(X_lr.copy())
 - 현재 `X_lr` 리스트를 복사하여 `X_lr_list`에 저장합니다. 이를 통해 각 단계에서 사용된 변수들의 기록을 남깁니다.

이 과정은 PCA 컴포넌트를 하나씩 추가하며 모델의 성능이 어떻게 변하는지 확인하고,
최종적으로 모든 변수를 사용했을 때의 성능을 평가합니다.
...

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

```

```
X_lr = list()
X_lr_list = list()

clf_lr = LogisticRegression(solver='lbfgs')
test_scores = list()
train_scores = list()

분산 설명율이 높은 순으로 컴포넌트를 하나씩 추가하여 변수를 구성해 갑니다.
for i in ['loading'] + X_comp:
 X_lr.append(i)
 clf_lr.fit(df_prob4_train[X_lr], df_prob4_train['failure'])
 test_scores.append(
 roc_auc_score(df_prob4_test['failure'], clf_lr.predict_proba(df_prob4_test[X_lr]))[:, 1])
)

 train_scores.append(
 roc_auc_score(df_prob4_train['failure'], clf_lr.predict_proba(df_prob4_train[X_lr]))[:, 1])
)

print(X_lr[-1], test_scores[-1], train_scores[-1])

X_lr_list.append(X_lr.copy())
```



```
loading 0.5778277926972065 0.5886355699157924
pca_0 0.579565141627812 0.5902607636650189
pca_1 0.5784170060859327 0.5906173884232395
pca_2 0.5786498903164131 0.5906803365700851
pca_3 0.5801256923420173 0.5911363799178113
pca_4 0.5805735466314028 0.5912654510937876
pca_5 0.5811103203542446 0.5914601161518473
pca_6 0.581757510516433 0.5929702204182378
pca_7 0.5808025223153578 0.5934465748718649
pca_8 0.5801787834323228 0.5935233418973748
pca_9 0.5801664063683251 0.593551407042703
pca_10 0.5794231311040503 0.5937232780741002
pca_11 0.5785678108393766 0.5940223479149978
pca_12 0.5782202759108135 0.594022551433093
pca_13 0.5782310244137587 0.593998210668907
pca_14 0.5776818736269195 0.5938648656129314
pca_15 0.5772988360674158 0.5941303346163114
pca_16 0.5768773644670778 0.5941881134035389
pca_17 0.5771252314592396 0.5945461424366163
```

In [98]: # 위 코드 실행 결과

```
...
loading 0.5778277926972065 0.5886355699157924
pca_0 0.579565141627812 0.5902607636650189
pca_1 0.5784170060859327 0.5906173884232395
pca_2 0.5786498903164131 0.5906803365700851
pca_3 0.5801256923420173 0.5911363799178113
pca_4 0.5805735466314028 0.5912654510937876
pca_5 0.5811103203542446 0.5914601161518473
pca_6 0.581757510516433 0.5929702204182378
pca_7 0.5808025223153578 0.5934465748718649
pca_8 0.5801787834323228 0.5935233418973748
pca_9 0.5801664063683251 0.593551407042703
pca_10 0.5794231311040503 0.5937232780741002
pca_11 0.5785678108393766 0.5940223479149978
pca_12 0.5782202759108135 0.594022551433093
pca_13 0.5782310244137587 0.593998210668907
pca_14 0.5776818736269195 0.5938648656129314
pca_15 0.5772988360674158 0.5941303346163114
pca_16 0.5768773644670778 0.5941881134035389
pca_17 0.5771252314592396 0.5945461424366163
```



```
Out[99]: [0.5778277926972065,
 0.579565141627812,
 0.5784170060859327,
 0.5786498903164131,
 0.5801256923420173,
 0.5805735466314028,
 0.5811103203542446,
 0.581757510516433,
 0.5808025223153578,
 0.5801787834323228,
 0.5801664063683251,
 0.5794231311040503,
 0.5785678108393766,
 0.5782202759108135,
 0.5782310244137587,
 0.5776818736269195,
 0.5772988360674158,
 0.5768773644670778,
 0.5771252314592396]
```

```
In [100... print('np.argmax(test_scores) = ', np.argmax(test_scores))

np.argmax(test_scores) = 7
```

```
In [101... '''
코드는 테스트 데이터에 대한 ROC AUC 점수가 최대인 변수 조합을 찾기 위해 사용됩니다.

1. `np.argmax(test_scores)` :
 `test_scores` 리스트에서 가장 큰 값(최대 ROC AUC 점수)의 인덱스를 반환합니다.
 이는 테스트 데이터에 대해 가장 높은 성능을 보이는 변수 조합의 인덱스를 나타냅니다.
2. `X_lr_list[np.argmax(test_scores)]` :
 `X_lr_list` 리스트에서 해당 인덱스에 해당하는 변수 조합을 선택합니다.
 이는 최대 ROC AUC 점수를 가진 변수 조합을 나타냅니다.

결과적으로 코드는 테스트 데이터에 대해 가장 높은 성능을 보이는 변수 조합을 출력합니다.
이는 모델을 구성할 때 사용할 최적의 변수 조합을 제공합니다.
'''

print('np.argmax(test_scores) = ', np.argmax(test_scores))
print('-----')

X_lr_list[np.argmax(test_scores)]
```

```
np.argmax(test_scores) = 7
```

```
Out[101]: ['loading', 'pca_0', 'pca_1', 'pca_2', 'pca_3', 'pca_4', 'pca_5', 'pca_6']
```

```
In [102...
```

```
'''
코드는 주어진 PCA(주성분 분석) 객체인 `pca`를 사용하여 주성분의 설명된 분산 비율을 계산하고,
처음 7개 주성분의 설명된 분산 비율을 합산하여 결과를 반환합니다.

1. `pca.explained_variance_ratio_`:
 PCA 객체인 `pca`의 `explained_variance_ratio_` 속성은 각 주성분이 설명하는 분산의 비율을 나타내는 배열입니다.
 이 배열의 각 원소는 주성분의 설명된 분산 비율을 나타냅니다.
2. `[:7]`:
 처음 7개의 주성분에 대한 설명된 분산 비율만 선택합니다. 즉, 0부터 6번 인덱스까지의 원소를 선택합니다.
3. `np.sum()`: 선택된 설명된 분산 비율의 합을 계산합니다.

결과적으로, 코드는 주어진 PCA 객체에서 처음 7개 주성분의 설명된 분산 비율을 합산하여 그 값을 반환합니다.
이 값은 주어진 데이터셋에서 선택된 주성분들이 설명하는 분산의 전체 비율을 나타냅니다.
'''

print('pca.explained_variance_ratio_[:7] = ', pca.explained_variance_ratio_[:7])
print('-----')

B = np.sum(pca.explained_variance_ratio_[:7])
B
```

```
pca.explained_variance_ratio_[:7] = [0.10311277 0.07882566 0.06974445 0.06462319 0.0592398 0.05833005
0.05666113]
```

```
Out[102]: 0.4905370453061068
```

```
In [103...
```

```
#A, B, A + B, round(A + B, 2)
A, B, A + B, (A + B).round(2)
```

```
Out[103]: (array([[1, 2, 3],
 [2, 4, 5],
 [3, 5, 6]]),
 0.4905370453061068,
 array([[1.49053705, 2.49053705, 3.49053705],
 [2.49053705, 4.49053705, 5.49053705],
 [3.49053705, 5.49053705, 6.49053705]]),
 array([[1.49, 2.49, 3.49],
 [2.49, 4.49, 5.49],
 [3.49, 5.49, 6.49]]))
```

A + B를 소수점 셋째 자리에서 반올림하여 둘째 자리까지 구하라.

**1.19**

## 문제 5

랜덤포레스트 분류기(Random-Forest Classifier)의

최적의 하이퍼 파라미터(Hyper-Parameter, 초매개변수)를 탐색하고자 한다.

문제3에서 사용했던, 전처리(loading 결측치 처리와 표준화 과정을 거친)

과정을 거친 prob3\_train과 prob3\_test를 사용한다.

In [ ]:

## 단계 5-1

sklearn에서 제공하는 랜덤포레스트 분류기(Random-Forest Classifier)의 하이퍼 파라미터 중

n\_estimators, max\_depth 그리고 min\_samples\_split의 최적 조합을 탐색한다.

탐색 값은 아래에 제공한 하이퍼 파라미터의 모든 조합이다.

prob3\_train을 대상으로 5-겹 층화교차검증(5-fold stratified cross validation)으로

각각 겹외셋(OOF set, Out-Of-Fold set)의 성능에 대한

평균을 기준으로 하이퍼 파라미터를 선택한다.

- n\_estimators: [5, 10, 15]
- max\_depth: [5, 6, 7]
- min\_samples\_split: [256, 512]

Hint] 모든 하이퍼 파라미터의 조합의 수는 18개이다

입력 변수: loading, measurement\_0 ~ 17, na\_1, na\_2 (순서에 유의)

대상 변수: failure

성능 지표: AUC (area under of ROC curve)

---

## 함수가이드

sklearn.ensemble.RandomForestClassifier, random\_state=123

itertools.product 필요시 사용

sklearn.model\_selection.cross\_val\_score 필요시 사용

sklearn.model\_selection.StratifiedKFold, random\_state=123, shuffle=True

sklearn.model\_selection.GridSearchCV 필요시 사용

---

In [104...

```
'''
아래 예제에서는 Iris 데이터셋을 로드하고, 이를 훈련 세트와 테스트 세트로 분할합니다.
선형 커널을 사용하여 SVC 모델을 적합시키고, 예측을 수행한 후 정확도를 평가합니다. 또한, 확률 예측도 수행합니다.
'''

from sklearn import datasets
from sklearn.model_selection import train_test_split
```

```

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

데이터셋 로드
iris = datasets.load_iris()
X = iris.data
y = iris.target

```

In [105...

```

훈련 세트와 테스트 세트로 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

SVC 모델 생성 및 적합
svc = SVC(C=1.0, kernel='linear', probability=True, random_state=42)
svc.fit(X_train, y_train)

예측 수행
y_pred = svc.predict(X_test)

정확도 평가
accuracy = accuracy_score(y_test, y_pred)
print(f"정확도: {accuracy:.2f}")
print('-----')

확률 예측 수행
print('len(X_test)', len(X_test))
print('-----')
y_proba = svc.predict_proba(X_test)
print("예측 확률:\n", y_proba)
print('-----')
print('len(y_proba)', len(y_proba))

```

정확도 : 1.00

len(X\_test) 45

예측 확률:

```
[[7.41921920e-03 9.04032778e-01 8.85480033e-02]
[9.42381156e-01 3.87782740e-02 1.88405702e-02]
[9.68952385e-04 2.95794772e-05 9.99001468e-01]
[9.02418630e-03 8.95945406e-01 9.50304078e-02]
[6.23324659e-03 8.76734004e-01 1.17032750e-01]
[9.41673474e-01 4.01790434e-02 1.81474822e-02]
[4.52612697e-02 9.43494999e-01 1.12437316e-02]
[1.33108787e-02 1.32689710e-01 8.53999412e-01]
[1.57436689e-02 6.38072077e-01 3.46184254e-01]
[2.03170637e-02 9.71468836e-01 8.21410080e-03]
[1.48546723e-02 2.70654756e-01 7.14490572e-01]
[9.47625598e-01 3.70962935e-02 1.52781088e-02]
[9.67546183e-01 2.07086206e-02 1.17451963e-02]
[9.43817763e-01 3.96450719e-02 1.65371654e-02]
[9.61421557e-01 2.41759282e-02 1.44025148e-02]
[7.25284990e-03 9.11068440e-01 8.16787102e-02]
[6.00539466e-03 4.06324147e-03 9.89931364e-01]
[1.88004221e-02 9.71052829e-01 1.01467490e-02]
[9.62877258e-03 9.03612691e-01 8.67585362e-02]
[7.24266289e-03 5.22826107e-03 9.87529076e-01]
[9.33693200e-01 4.70353976e-02 1.92714022e-02]
[1.70591686e-02 4.06901002e-01 5.76039829e-01]
[9.32201577e-01 4.73125755e-02 2.04858477e-02]
[7.79538924e-03 7.60870467e-03 9.84595906e-01]
[4.39965125e-03 2.84410350e-02 9.67159314e-01]
[1.26455166e-02 5.36150091e-02 9.33739474e-01]
[6.41300658e-03 6.22813746e-03 9.87358856e-01]
[5.28389100e-03 4.42856966e-03 9.90287539e-01]
[9.39647590e-01 4.29477981e-02 1.74046119e-02]
[9.27325662e-01 5.26081974e-02 2.00661404e-02]
[9.83558594e-01 9.07782227e-03 7.36358398e-03]
[9.73188613e-01 1.47873371e-02 1.20240504e-02]
[8.19051536e-03 9.80521525e-01 1.12879599e-02]
[9.41698995e-01 4.03079316e-02 1.79930729e-02]
[9.61058961e-01 2.57750293e-02 1.31660101e-02]
[1.64861149e-02 9.12412911e-02 8.92272594e-01]
[7.58708761e-03 9.64969942e-01 2.74429704e-02]]
```



```
[9.54186707e-01 3.05649519e-02 1.52483413e-02]
[9.65374963e-01 2.18600091e-02 1.27650275e-02]
[9.74535944e-01 1.45339963e-02 1.09300599e-02]
[1.69699439e-02 5.36760927e-02 9.29353963e-01]
[9.21017066e-03 9.52034224e-01 3.87556054e-02]
[5.71434396e-03 9.35877132e-01 5.84085241e-02]
[9.72419762e-01 1.61889618e-02 1.13912759e-02]
[9.60825585e-01 2.51293651e-02 1.40450498e-02]]
```

```
len(y_proba) 45
```

In [106...

```
'''
`SVC` (Support Vector Classification)는 `scikit-learn` 라이브러리의 `svm` 모듈에 있는 클래스입니다.
SVC는 서포트 벡터 머신(SVM)을 사용하여 분류 작업을 수행합니다.
SVM은 높은 차원의 공간에서 데이터를 분류하기 위해 최적의 초평면을 찾는 강력한 분류 알고리즘입니다.

클래스의 인자
1. C:
 - 유형: 실수 (기본값은 1.0)
 - 설명: 규제 매개변수입니다. 값이 작을수록 규제가 강해집니다. 이는 과적합을 방지하는 데 사용됩니다.
2. kernel:
 - 유형: 문자열 (기본값은 'rbf')
 - 설명: 사용할 커널 타입을 지정합니다. 가능한 값은 'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'입니다.
3. degree:
 - 유형: 정수 (기본값은 3)
 - 설명: 다항 커널('poly')을 사용할 경우 커널 함수의 차수를 지정합니다.
4. gamma:
 - 유형: 'scale', 'auto' 또는 실수 (기본값은 'scale')
 - 설명: 'rbf', 'poly', 'sigmoid' 커널에 대한 커널 계수입니다.
 - 'scale': 1 / (n_features * X.var())로 설정합니다.
 - 'auto': 1 / n_features로 설정합니다.
5. coef0:
 - 유형: 실수 (기본값은 0.0)
 - 설명: 다항 및 시그모이드 커널 함수에서 상수 항을 지정합니다.
6. shrinking:
 - 유형: 부울 (기본값은 True)
 - 설명: 수축 휴리스틱을 사용할지 여부를 지정합니다.
7. probability:
 - 유형: 부울 (기본값은 False)
 - 설명: 확률 추정을 사용할지 여부를 지정합니다. True로 설정하면 fit 메서드가 더 오래 걸립니다.
8. tol:
 - 유형: 실수 (기본값은 1e-3)
```

- 설명: 종료 조건에 대한 허용 오차입니다.

#### 9. cache\_size:

- 유형: 실수 (기본값은 200)
- 설명: 커널 캐시에 사용할 메모리 크기를 MB 단위로 지정합니다.

#### 10. class\_weight:

- 유형: 'balanced', 딕셔너리 또는 None (기본값은 None)
- 설명: 클래스의 가중치를 지정합니다. 'balanced'로 설정하면 클래스 빈도에 따라 자동으로 조정됩니다.

#### 11. verbose:

- 유형: 부울 (기본값은 False)
- 설명: 자세한 출력을 활성화할지 여부를 지정합니다.

#### 12. max\_iter:

- 유형: 정수 (기본값은 -1)
- 설명: 최대 반복 횟수를 지정합니다. -1인 경우, 반복 횟수에 제한이 없습니다.

#### 13. decision\_function\_shape:

- 유형: 'ovo', 'ovr' (기본값은 'ovr')
- 설명: 다중 클래스 전략을 지정합니다. 'ovr'은 일대다(one-vs-rest) 방식, 'ovo'는 일대일(one-vs-one) 방식을 사용합니다.

#### 14. break\_ties:

- 유형: 부울 (기본값은 False)
- 설명: True로 설정하면 결정 함수 값이 동일한 경우 타이를 깨기 위해 추가적인 계산을 수행합니다.

#### 15. random\_state:

- 유형: 정수, RandomState 인스턴스 또는 None (기본값은 None)
- 설명: 난수 생성 시드를 설정합니다. 재현 가능한 결과를 위해 사용됩니다.

### ### 주요 메서드

#### 1. fit(X, y, sample\_weight=None):

- 설명: 모델을 데이터에 적합시킵니다.
- 인자:
  - `X`: 훈련 데이터 배열.
  - `y`: 타겟 레이블 배열.
  - `sample\_weight`: 샘플 가중치를 지정하는 배열 (기본값은 None).
- 리턴값: 적합된 `SVC` 인스턴스.

#### 2. predict(X):

- 설명: 입력 데이터에 대한 클래스 레이블을 예측합니다.
- 인자:
  - `X`: 입력 데이터 배열.
- 리턴값: 예측된 클래스 레이블 배열.

#### 3. predict\_proba(X):

- 설명: 각 클래스에 대한 확률을 예측합니다. (probability=True인 경우에만 사용 가능)
- 인자:
  - `X`: 입력 데이터 배열.
- 리턴값: 각 클래스에 대한 확률 배열.

4. `decision_function(X)`:
  - 설명: 각 샘플에 대한 결정 함수 값을 계산합니다.
  - 인자:
    - `X`: 입력 데이터 배열.
  - 리턴값: 각 샘플에 대한 결정 함수 값 배열.
5. `score(X, y, sample_weight=None)`:
  - 설명: 입력 데이터와 타겟 레이블에 대한 모델의 정확도를 계산합니다.
  - 인자:
    - `X`: 입력 데이터 배열.
    - `y`: 타겟 레이블 배열.
    - `sample\_weight`: 샘플 가중치를 지정하는 배열 (기본값은 None).
  - 리턴값: 모델의 정확도.
6. `get_params(deep=True)`:
  - 설명: 추정기의 매개변수를 반환합니다.
  - 인자:
    - `deep`: 부울 값, True인 경우 하위 객체의 매개변수도 포함합니다.
  - 리턴값: 매개변수의 딕셔너리.
7. `set_params(params)`:
  - 설명: 추정기의 매개변수를 설정합니다.
  - 인자:
    - `params`: 설정할 매개변수.
  - 리턴값: 적합된 `SVC` 인스턴스.

### 주요 속성

1. `support_`:
  - 설명: 서포트 벡터의 인덱스를 나타내는 배열입니다.
2. `support_vectors_`:
  - 설명: 서포트 벡터를 나타내는 배열입니다.
3. `n_support_`:
  - 설명: 각 클래스의 서포트 벡터 수를 나타내는 배열입니다.
4. `dual_coef_`:
  - 설명: 결정 함수에서 서포트 벡터의 계수입니다.
5. `coef_`:
  - 설명: 선형 모델의 계수 (`kernel='linear'`인 경우).
6. `intercept_`:
  - 설명: 결정 함수의 절편입니다.

### 요약

`SVC` 클래스는 SVM을 사용하여 분류 작업을 수행하는 강력한 도구입니다.

다양한 인자들을 통해 모델의 동작 방식을 설정할 수 있으며, 주요 메서드와 속성을 사용하여 모델을 적합시키고 예측 및 평가를 수행할 수 있습니다. SVM의 특성상 높은 차원의 데이터를 다루는 데 효과적이며, 커널 트릭을 통해 비선형 분류도 가능합니다.

```
'''
```

```
print('')
```

In [107...

```
'''
```

```
아래 예제에서는 5차원 데이터셋을 생성하고, 이를 2차원으로 축소하는 PCA 모델을 적합시킵니다.
주성분과 설명된 분산 비율을 출력하고, 변환된 데이터를 시각화합니다.
```

```
'''
```

```
from sklearn.datasets import load_iris
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

데이터 로드
iris = load_iris()
X, y = iris.data, iris.target

모델과 파라미터 그리드 설정
svc = SVC()
param_grid = {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf', 'poly'], 'gamma': ['scale', 'auto']}

GridSearchCV 인스턴스 생성
grid_search = GridSearchCV(estimator=svc, param_grid=param_grid, cv=5, scoring='accuracy', n_jobs=-1)

모델 적합
grid_search.fit(X, y)

결과 출력
print("최적의 파라미터:", grid_search.best_params_)
print('-----')
print("최고 교차 검증 점수:", grid_search.best_score_)
print('-----')
print("최적의 추정기:", grid_search.best_estimator_)
print('-----')

최적의 모델로 예측 수행
y_pred = grid_search.predict(X)
print(y_pred)
```

최적의 파라미터: {'C': 0.1, 'gamma': 'scale', 'kernel': 'poly'}

최고 교차 검증 점수: 0.9866666666666667

최적의 추정기: SVC(C=0.1, cache\_size=200, class\_weight=None, coef0=0.0, decision\_function\_shape='ovr', degree=3, gamma='scale', kernel='poly', max\_iter=-1, probability=False, random\_state=None, shrinking=True, tol=0.001, verbose=False)

```
[0 0
 0 0 0 0 0 0 0 0 0 0 0 0 1
 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 1 2 2 2 2
 2
 2 2]
```

In [108...

```
'''
`GridSearchCV`는 `scikit-learn` 라이브러리의 `model_selection` 모듈에 있는 클래스입니다.
이 클래스는 하이퍼파라미터 튜닝을 자동으로 수행하여 주어진 모델의
최적의 하이퍼파라미터 조합을 찾는 데 사용됩니다.
`GridSearchCV`는 지정된 파라미터 그리드를 통해 모든 가능한 조합을 시도하고 교차 검증을 통해 성능을 평가합니다.

클래스의 인자
1. estimator:
 - 유형: 추정기 객체
 - 설명: 모델을 나타내는 객체입니다 (예: `sklearn.svm.SVC()`).
2. param_grid:
 - 유형: 딕셔너리 또는 리스트
 - 설명: 하이퍼파라미터 조합을 지정하는 딕셔너리 또는 딕셔너리의 리스트입니다.
 키는 하이퍼파라미터 이름, 값은 가능한 매개변수 값의 리스트입니다.
3. scoring:
 - 유형: 문자열, 호출 가능 객체 또는 None (기본값은 None)
 - 설명: 모델 평가에 사용할 평가 기준입니다. None인 경우, 추정기의 기본 평가 기준이 사용됩니다.
4. n_jobs:
 - 유형: 정수 (기본값은 None)
 - 설명: 병렬 처리할 작업 수를 지정합니다. -1로 설정하면 모든 프로세서를 사용합니다.
5. iid:
 - 유형: 부울 (기본값은 'deprecated')
 - 설명: 이전 버전에서 사용된 인자로 현재는 더 이상 사용되지 않으며, 무시됩니다.
6. refit:
 - 유형: 부울 또는 문자열 (기본값은 True)
 - 설명: 최적의 하이퍼파라미터로 전체 데이터에 대해 추정기를 다시 적합시킬지 여부를 지정합니다.
 True로 설정하면, 최적의 하이퍼파라미터로 모델을 재적합시킵니다.
'''
```

```

7. cv:
 - 유형: 정수, 교차 검증 생성기 또는 반복자 (기본값은 None)
 - 설명: 교차 검증을 생성하는 데 사용할 방법을 지정합니다. None인 경우,
 기본적으로 5-겹 교차 검증이 사용됩니다.
8. verbose:
 - 유형: 정수 (기본값은 0)
 - 설명: 메시지 출력의 상세 수준을 지정합니다. 값이 클수록 더 많은 메시지가 출력됩니다.
9. pre_dispatch:
 - 유형: 정수 또는 문자열 (기본값은 '2*n_jobs')
 - 설명: 병렬 작업을 실행하기 전에 준비할 작업 수를 지정합니다.
10. error_score:
 - 유형: 'raise' 또는 숫자 (기본값은 np.nan)
 - 설명: 모델 적합 도중 오류가 발생할 경우 반환할 점수입니다.
11. return_train_score:
 - 유형: 부울 (기본값은 False)
 - 설명: 훈련 세트 점수를 반환할지 여부를 지정합니다.

주요 메서드
1. fit(X, y=None, fit_params):
 - 설명: 파라미터 그리드의 각 조합에 대해 교차 검증을 수행하고 최적의 파라미터를 찾습니다.
 - 인자:
 - `X`: 훈련 데이터 배열.
 - `y`: 타겟 레이블 배열.
 - `fit_params`: 추가적으로 전달할 적합 매개변수.
 - 리턴값: 적합된 `GridSearchCV` 인스턴스.
2. predict(X):
 - 설명: 최적의 추정기를 사용하여 입력 데이터에 대한 예측을 수행합니다.
 - 인자:
 - `X`: 입력 데이터 배열.
 - 리턴값: 예측된 값의 배열.
3. score(X, y=None):
 - 설명: 최적의 추정기를 사용하여 입력 데이터에 대한 점수를 계산합니다.
 - 인자:
 - `X`: 입력 데이터 배열.
 - `y`: 타겟 레이블 배열.
 - 리턴값: 점수.
4. predict_proba(X):
 - 설명: 클래스 확률을 예측합니다 (가능한 경우).
 - 인자:
 - `X`: 입력 데이터 배열.
 - 리턴값: 각 클래스의 확률 배열.
5. decision_function(X):

```

```

- 설명: 각 클래스의 결정 함수 값을 계산합니다 (가능한 경우).
- 인자:
 - `X`: 입력 데이터 배열.
- 리턴값: 각 클래스의 결정 함수 값 배열.
6. transform(X):
- 설명: 입력 데이터를 변환합니다 (가능한 경우).
- 인자:
 - `X`: 입력 데이터 배열.
- 리턴값: 변환된 데이터 배열.
7. inverse_transform(X):
- 설명: 변환된 데이터를 원래 특성 공간으로 변환합니다 (가능한 경우).
- 인자:
 - `X`: 변환된 데이터 배열.
- 리턴값: 원래 특성 공간의 데이터 배열.

주요 속성
1. cv_results_:
- 설명: 교차 검증의 결과를 담고 있는 딕셔너리입니다. 각 키는 다양한 성능 메트릭 및 파라미터 설정에 해당합니다.
2. best_estimator_:
- 설명: 최적의 파라미터 설정으로 적합한 추정기입니다.
3. best_score_:
- 설명: 최적의 파라미터 설정에서의 최고 교차 검증 점수입니다.
4. best_params_:
- 설명: 최적의 파라미터 설정을 나타내는 딕셔너리입니다.
5. best_index_:
- 설명: 최적의 파라미터 설정에서의 cv_results_ 배열 인덱스입니다.
6. scorer_:
- 설명: 사용된 스코어러 객체입니다.
7. n_splits_:
- 설명: 교차 검증 시 사용된 분할 수입니다.

요약
`GridSearchCV` 클래스는 하이퍼파라미터 튜닝을 자동화하고 최적의 모델을 찾는 강력한 도구입니다.
다양한 인자들을 통해 교차 검증 및 평가 방식을 설정할 수 있으며, 주요 메서드와 속성을 사용하여
모델을 적합시키고 예측 및 평가를 수행할 수 있습니다.
'''
print('')

```

In [109...

```

'''
`StratifiedKFold`는 `scikit-learn` 라이브러리의 `model_selection` 모듈에 있는 클래스입니다.
이 클래스는 데이터셋을 여러 개의 "층화된" (stratified) 폴드로 나누는 데 사용됩니다.

```

층화된 폴드는 각 클래스의 비율이 전체 데이터셋의 클래스 비율과 동일하도록 나뉘어져 있어, 분류 작업에서 특히 유용합니다.

### 클래스의 인자

1. `n_splits`:

- 유형: 정수 (기본값은 5)
- 설명: 데이터셋을 나눌 폴드의 수입니다.

2. `shuffle`:

- 유형: 부울 (기본값은 `False`)
- 설명: 데이터를 분할하기 전에 무작위로 섞을지 여부를 지정합니다.

### 주요 메서드

1. `split(X, y, groups=None)`:

- 설명: 데이터셋을 폴드로 나눕니다.
- 인자:
  - ``X``: 입력 데이터 배열 (특성 행렬).
  - ``y``: 타겟 레이블 배열.
  - ``groups``: 그룹 레이블 배열 (기본값은 `None`).
- 리턴값: 각 폴드에 대한 훈련 세트와 테스트 세트의 인덱스를 생성하는 제너레이터.

### 주요 속성

- `n_splits`:

- 설명: 폴드의 수를 나타내는 정수입니다.

### 요약

``StratifiedKFold`` 클래스는 층화된 폴드를 생성하여 교차 검증을 수행하는 데 유용한 도구입니다. 주요 인자들을 통해 폴드의 수와 데이터를 무작위로 섞을지 여부 등을 설정할 수 있으며, 주요 메서드를 사용하여 데이터를 폴드로 나누고 훈련 및 평가를 수행할 수 있습니다. 이는 특히 분류 작업에서 데이터의 클래스 비율을 유지하면서 교차 검증을 수행하는 데 유용합니다.

...

`print('')`

In [110...

...

코드는 ``RandomForestClassifier``를 사용하여 랜덤포레스트 모델의 최적의 하이퍼파라미터를 찾기 위해 그리드 탐색(Grid Search)을 수행합니다.

1. ``X_gscv`` 리스트 초기화:

- ``X_gscv``: 모델의 입력 변수들을 담은 리스트입니다. 여기서는 'loading' 변수와 18개의 'measurement\_' 변수, 그리고 'na\_1', 'na\_2' 변수를 포함합니다.

2. ``GridSearchCV`` 초기화:

- ``RandomForestClassifier(random_state=123)``: ``RandomForestClassifier`` 모델을 생성합니다.
- ``param_grid``: 그리드 탐색을 위해 탐색할 하이퍼파라미터들의 조합을 딕셔너리 형태로 지정합니다.



여기서는 'n\_estimators', 'max\_depth', 'min\_samples\_split'의 가능한 값들을 지정합니다.

- ``scoring='roc_auc'``: 모델의 성능을 평가할 지표로 ROC AUC를 사용합니다.
- ``cv=StratifiedKFold(5, random_state=123, shuffle=True)``: 5-겹 층화교차검증(5-fold stratified cross validation)을 사용하여 모델의 성능을 평가합니다. 이때 각 겹의 데이터셋을 무작위로 섞습니다.
- ``iid=False``: 각각 겹외셋(Out-Of-Fold set)의 성능에 대한 평균을 계산할 때, 가중치를 동일하게 설정합니다.

3. 그리드 탐색 실행:

- ``gscv.fit(df_prob3_train[X_gscv], df_prob3_train['failure'])``: 그리드 탐색을 훈련 데이터에 대해 실행합니다. 여기서는 입력 변수로 ``X_gscv``를, 목표 변수로 `'failure'`를 사용합니다.

결과적으로, 이 코드는 랜덤 포레스트 모델을 사용하여 주어진 데이터에 대해 그리드 탐색을 수행하고, 최적의 하이퍼파라미터를 찾아 모델을 훈련시킵니다. 이렇게 하여 모델의 성능을 최대화하고 일반화할 수 있도록 합니다.

```
'''
...
'''
```

코드를 한 줄씩 자세히 설명해보겠습니다.

#### 1. ``X_gscv`` 리스트 초기화:

- ``'loading'``: 첫 번째 변수로 `'loading'`을 포함합니다. 이 변수는 모델에 항상 포함됩니다.
- ``['measurement_{}'.format(i) for i in range(18)]``: 18개의 `'measurement_'` 변수를 생성합니다. 이 변수들은 `'measurement_0'`부터 `'measurement_17'`까지의 이름을 갖습니다.
- ``['na_1', 'na_2']``: `'na_1'`과 `'na_2'` 변수를 추가합니다. 이 변수들은 결측값을 나타냅니다.
- 따라서 ``X_gscv`` 리스트에는 총 21개의 변수가 포함됩니다.

#### 2. ``GridSearchCV`` 초기화:

- ``RandomForestClassifier(random_state=123)``: ``RandomForestClassifier`` 모델을 생성합니다. 이 모델은 랜덤 포레스트를 구축하는 데 사용됩니다.
- ``param_grid``: 탐색할 하이퍼파라미터 그리드를 지정합니다.
  - `'n_estimators'`: 랜덤 포레스트에서 사용할 결정 트리의 수입니다. ``[5, 10, 15]`` 세 개의 값으로 탐색합니다.
  - `'max_depth'`: 각 결정 트리의 최대 깊이입니다. ``[5, 6, 7]`` 세 개의 값으로 탐색합니다.
  - `'min_samples_split'`: 내부 노드를 분할하기 위해 필요한 최소 샘플 수입니다. ``[256, 512]`` 두 개의 값으로 탐색합니다.
- ``scoring='roc_auc'``: 모델의 성능을 평가할 지표로 ROC AUC를 사용합니다. 이는 이진분류 문제에서 많이 사용되는 평가지표 중 하나입니다.
- ``cv=StratifiedKFold(5, random_state=123, shuffle=True)``: 5-겹 층화교차검증을 사용하여 모델을 평가합니다. 데이터를 5개의 폴드로 나누고, 각 폴드에 대해 훈련 및 검증을 수행합니다. ``shuffle=True``는 데이터를 무작위로 섞어 폴드를 생성합니다.
- ``iid=False``: 각 겹의 성능을 평균할 때, 각각 겹의 데이터셋의 크기를 가중치로 사용하지 않습니다.

#### 3. 그리드 탐색 실행:

- ``gscv.fit(df_prob3_train[X_gscv], df_prob3_train['failure'])``: 그리드 탐색을 훈련 데이터에 대해 실행합니다. 여기서는 입력 변수로 ``X_gscv``를, 목표 변수로 `'failure'`를 사용합니다. 그리드 탐색은 주어진 하이퍼파라미터 그리드의 모든 조합에 대해 모델을 훈련하고 교차 검증을 수행하여 최적의 하이퍼파라미터를 선택합니다.

```
...
'''
```

*# 방법 1: `sklearn.model_selection.GridSearchCV` 이용*

```

from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold

X_gscv = ['loading'] + ['measurement_{}'.format(i) for i in range(18)] + ['na_1', 'na_2']
print('X_gscv = ', X_gscv)
print('-----')

gscv = GridSearchCV(
 RandomForestClassifier(random_state=123),
 param_grid = {
 'n_estimators': [5, 10, 15],
 'max_depth': [5, 6, 7],
 'min_samples_split': [256, 512]
 },
 scoring='roc_auc', # AUC
 # 5-겹 층화교차검증(5-fold stratified cross validation)
 cv = StratifiedKFold(5, random_state=123, shuffle=True),
 # 각각 겹의셋(OOF set, Out-Of-Fold set)의 성능에 대한 평균
 iid=False,
)

gscv.fit(df_prob3_train[X_gscv], df_prob3_train['failure'])

X_gscv = ['loading', 'measurement_0', 'measurement_1', 'measurement_2', 'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6', 'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10', 'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14', 'measurement_15', 'measurement_16', 'measurement_17', 'na_1', 'na_2']

```

```
Out[110]: GridSearchCV(cv=StratifiedKFold(n_splits=5, random_state=123, shuffle=True),
 error_score='raise-deprecating',
 estimator=RandomForestClassifier(bootstrap=True, class_weight=None,
 criterion='gini', max_depth=None,
 max_features='auto',
 max_leaf_nodes=None,
 min_impurity_decrease=0.0,
 min_impurity_split=None,
 min_samples_leaf=1,
 min_samples_split=2,
 min_weight_fraction_leaf=0.0,
 n_estimators='warn', n_jobs=None,
 oob_score=False, random_state=123,
 verbose=0, warm_start=False),

 iid=False, n_jobs=None,
 param_grid={'max_depth': [5, 6, 7],
 'min_samples_split': [256, 512],
 'n_estimators': [5, 10, 15]},
 pre_dispatch='2*n_jobs', refit=True, return_train_score=False,
 scoring='roc_auc', verbose=0)
```

In [111...

'''

코드는 그리드 탐색(Grid Search)을 통해 찾은 최적의 하이퍼파라미터와 그에 해당하는 모델의 성능을 확인합니다.

### 1. `best\_params = gscv.best\_params`:

- `gscv.best\_params`: 그리드 탐색을 통해 찾은 최적의 하이퍼파라미터를 나타냅니다.
- 이 부분은 최적의 하이퍼파라미터들을 `best\_params` 변수에 저장합니다.

### 2. `gscv.best\_score`:

- `gscv.best\_score`: 그리드 탐색을 통해 찾은 최적의 모델의 평균 검증 점수를 나타냅니다.
- 이 부분은 최적의 모델의 평균 검증 점수를 반환합니다.

### 3. 주석 처리된 코드:

- `# gscv.cv\_results`:  
그리드 탐색의 모든 결과를 담은 딕셔너리를 반환합니다. 주석 처리되어 있으므로 현재 사용되지 않습니다.
- `# gscv.best\_estimator`:  
최적의 모델을 반환합니다. 주석 처리되어 있으므로 현재 사용되지 않습니다.

따라서 코드는 그리드 탐색을 통해 찾은 최적의 하이퍼파라미터와 해당 모델의 평균 검증 점수를 확인합니다.  
최적의 하이퍼파라미터를 사용하여 모델을 구축하고 평가할 수 있습니다.

'''

```
best_params = gscv.best_params_

#, gscv.cv_results_#, gscv.best_estimator_

print('gscv.best_params_ = ', gscv.best_params_)
print('-----')
print('gscv.best_score_ = ', gscv.best_score_)
print('-----')
print('gscv.cv_results_ = ', gscv.cv_results_)
print('-----')
print('gscv.best_estimator_ = ', gscv.best_estimator_)
```

```

gscv.best_params_ = {'max_depth': 7, 'min_samples_split': 512, 'n_estimators': 15}

gscv.best_score_ = 0.5745226991354744

gscv.cv_results_ = {'mean_fit_time': array([0.07910085, 0.13808031, 0.19349985, 0.07307949, 0.13368912,
 0.19510412, 0.0791749 , 0.14212847, 0.22192163, 0.08063745,
 0.15509987, 0.2131135 , 0.08609877, 0.15734677, 0.23029342,
 0.08083482, 0.15305429, 0.22742176]), 'std_fit_time': array([0.00111829, 0.00623198, 0.00296251, 0.00178238, 0.00802632,
 0.00769235, 0.00105199, 0.00194959, 0.00393943, 0.00225243,
 0.00834199, 0.00520051, 0.00246132, 0.00248158, 0.00298839,
 0.00140826, 0.00236599, 0.0036519]), 'mean_score_time': array([0.00550437, 0.00570378, 0.0067925 , 0.00502424, 0.0061926
8,
 0.00673656, 0.00450363, 0.00540409, 0.00710635, 0.00500631,
 0.00590491, 0.00631394, 0.00450072, 0.00580201, 0.00685377,
 0.00450587, 0.00561752, 0.00659447]), 'std_score_time': array([0.00055013, 0.00024248, 0.00023373, 0.00057678, 0.0006326
7,
 0.00047116, 0.00054943, 0.00080226, 0.00058646, 0.00031625,
 0.00037323, 0.00042245, 0.00044718, 0.00051096, 0.00043334,
 0.00031937, 0.00022214, 0.00020351]), 'param_max_depth': masked_array(data=[5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 7, 7, 7,
7, 7, 7],
 mask=[False, False, False, False, False, False, False, False,
 False, False, False, False, False, False, False, False, False,
 False, False],
 fill_value='?',
 dtype=object), 'param_min_samples_split': masked_array(data=[256, 256, 256, 512, 512, 512, 256, 256, 256, 512, 512,
512, 256, 256, 256, 512, 512, 512],
 mask=[False, False, False, False, False, False, False, False, False,
 False, False, False, False, False, False, False, False, False,
 False],
 fill_value='?',
 dtype=object), 'param_n_estimators': masked_array(data=[5, 10, 15, 5, 10, 15, 5, 10, 15, 5, 10, 15, 5, 10, 15,
5, 10, 15],
 mask=[False, False, False, False, False, False, False, False, False,
 False, False, False, False, False, False, False, False, False,
 False],
 fill_value='?',
 dtype=object), 'params': [{'max_depth': 5, 'min_samples_split': 256, 'n_estimators': 5}, {'max_depth': 5, 'min_sampl
es_split': 256, 'n_estimators': 10}, {'max_depth': 5, 'min_samples_split': 256, 'n_estimators': 15}, {'max_depth': 5, 'min_sampl
es_split': 512, 'n_estimators': 5}, {'max_depth': 5, 'min_samples_split': 512, 'n_estimators': 10}, {'max_depth': 5, 'min_sampl
es_split': 512, 'n_estimators': 15}, {'max_depth': 6, 'min_samples_split': 256, 'n_estimators': 5}, {'max_depth': 6, 'min_sampl
es_split': 256, 'n_estimators': 10}, {'max_depth': 6, 'min_samples_split': 256, 'n_estimators': 15}, {'max_depth': 6, 'min_sampl
es_split': 512, 'n_estimators': 5}, {'max_depth': 6, 'min_samples_split': 512, 'n_estimators': 10}, {'max_depth': 6, 'min_sampl
es_split': 512, 'n_estimators': 15}]}

```

```

split': 512, 'n_estimators': 15}, {'max_depth': 7, 'min_samples_split': 256, 'n_estimators': 5}, {'max_depth': 7, 'min_samples_split': 256, 'n_estimators': 10}, {'max_depth': 7, 'min_samples_split': 256, 'n_estimators': 15}, {'max_depth': 7, 'min_samples_split': 512, 'n_estimators': 5}, {'max_depth': 7, 'min_samples_split': 512, 'n_estimators': 10}, {'max_depth': 7, 'min_samples_split': 512, 'n_estimators': 15}], 'split0_test_score': array([0.55772586, 0.57576266, 0.57711973, 0.57327539, 0.58392692,
0.58177993, 0.56223118, 0.57547528, 0.57840253, 0.57493357,
0.58452661, 0.58386919, 0.55355904, 0.5721083 , 0.57683438,
0.56125001, 0.57735269, 0.57975756]), 'split1_test_score': array([0.52999849, 0.53750454, 0.53979785, 0.53169036, 0.54268
624,
0.54692838, 0.54107909, 0.55040574, 0.55333331, 0.53551755,
0.54046722, 0.54207361, 0.52359432, 0.54050513, 0.54044763,
0.53852246, 0.54476609, 0.54656151]), 'split2_test_score': array([0.56252283, 0.57111448, 0.57365737, 0.57488874, 0.58178
343,
0.58061871, 0.55498219, 0.57098829, 0.57263512, 0.57316762,
0.57541997, 0.57577437, 0.57118851, 0.57885077, 0.57833939,
0.57968805, 0.58344731, 0.57892277]), 'split3_test_score': array([0.54878867, 0.55096215, 0.55671909, 0.55001878, 0.55494
504,
0.55037623, 0.5565036 , 0.55392916, 0.55444715, 0.55214722,
0.55061894, 0.54970203, 0.56068111, 0.55877299, 0.56500925,
0.55914775, 0.56435361, 0.56524814]), 'split4_test_score': array([0.57452188, 0.59193235, 0.59874257, 0.57870194, 0.59039
568,
0.59850037, 0.56970374, 0.58985988, 0.59426128, 0.575854 ,
0.58939099, 0.59823374, 0.57479385, 0.58489036, 0.58907806,
0.58239937, 0.59211045, 0.60212351]), 'mean_test_score': array([0.55471155, 0.56545524, 0.56920732, 0.56171504, 0.5707474
6,
0.57164072, 0.55689996, 0.56813167, 0.57061588, 0.56232399,
0.56808475, 0.56993059, 0.55676337, 0.56702551, 0.56994174,
0.56420153, 0.57240603, 0.5745227]), 'std_test_score': array([0.0148887 , 0.01913426, 0.01987769, 0.01807776, 0.0185401
8,
0.01983798, 0.0094498 , 0.01449223, 0.01538785, 0.01601124,
0.01921394, 0.02104618, 0.01821529, 0.01585093, 0.01660397,
0.01590384, 0.01651333, 0.01831073]), 'rank_test_score': array([18, 12, 8, 15, 4, 3, 16, 9, 5, 14, 10, 7, 17, 11,
6, 13, 2,
1]))}

```

```

gscv.best_estimator_ = RandomForestClassifier(bootstrap=True, class_weight=None, criterion='gini',
max_depth=7, max_features='auto', max_leaf_nodes=None,
min_impurity_decrease=0.0, min_impurity_split=None,
min_samples_leaf=1, min_samples_split=512,
min_weight_fraction_leaf=0.0, n_estimators=15,
n_jobs=None, oob_score=False, random_state=123,
verbose=0, warm_start=False)

```

In [112...

```
'''
코드는 그리드 탐색(Grid Search)을 수동으로 구현하여 랜덤 포레스트(Random Forest) 모델의 최적의 하이퍼파라미터를 탐색합니다.
이를 위해 `product` 함수를 사용하여 가능한 모든 하이퍼파라미터 조합을 생성하고,
각 조합에 대해 교차 검증을 수행하여 평균 ROC AUC 점수를 계산합니다.
'''
```

#### 1. 하이퍼파라미터 조합 생성:

```
- `product([5, 10, 15], [5, 6, 7], [256, 512])`:
 `product` 함수를 사용하여 'n_estimators', 'max_depth', 'min_samples_split'의 모든 가능한 조합을 생성합니다.
 즉, 'n_estimators'는 5, 10, 15 중 하나, 'max_depth'는 5, 6, 7 중 하나,
 'min_samples_split'은 256, 512 중 하나의 값을 가집니다.
 이렇게 생성된 조합은 `params` 리스트에 딕셔너리 형태로 저장됩니다.
```

#### 2. 교차 검증 수행:

```
- `cv = StratifiedKFold(5, random_state=123, shuffle=True)`:
 5-겹 층화 교차 검증을 설정합니다. 이때 데이터를 5개의 폴드로 나누고, 각 폴드에 대해 훈련 및 검증을 수행합니다.
 `shuffle=True`는 데이터를 무작위로 섞어 폴드를 생성합니다.
```

#### 3. 그리드 탐색 수행:

```
- `for param in params:`: 가능한 모든 하이퍼파라미터 조합에 대해 반복합니다.
- `RandomForestClassifier(random_state=123, param)``:
 현재 하이퍼파라미터 조합을 사용하여 랜덤 포레스트 분류기를 초기화합니다.
- 교차 검증을 통해 평균 ROC AUC 점수 계산:
 - `for train_idx, valid_idx in cv.split(df_prob3_train[X_gscv], df_prob3_train['failure']):`
 교차 검증에서 각 폴드의 인덱스를 반환합니다.
 - `clf_rf.fit(df_cv_train[X_gscv], df_cv_train['failure'])`:
 훈련 데이터로 랜덤 포레스트 모델을 훈련합니다.
 - `roc_auc_score(df_valid['failure'], clf_rf.predict_proba(df_valid[X_gscv])[:, 1])`:
 검증 데이터에 대한 ROC AUC 점수를 계산하고, 이를 `cv_scores` 리스트에 추가합니다.
- `test_scores.append(cv_scores_mean)`:
 현재 하이퍼파라미터 조합에 대한 교차 검증 평균 ROC AUC 점수를 `test_scores` 리스트에 추가합니다.
- `print(param, test_scores[-1])`:
 현재 하이퍼파라미터 조합과 이에 대한 교차 검증 평균 ROC AUC 점수를 출력합니다.
```

```
결과적으로 이 코드는 모든 가능한 하이퍼파라미터 조합에 대해 랜덤 포레스트 모델을 훈련하고,
각 모델의 평균 교차 검증 ROC AUC 점수를 계산하여 출력합니다. 이를 통해 최적의 하이퍼파라미터 조합을 선택할 수 있습니다.
'''
```

```
방법 2 itertools.product + 직접 교차 검증
```

```
from itertools import product
from sklearn.ensemble import RandomForestClassifier
```

```

from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score
from sklearn.metrics import roc_auc_score

params = list()
for a, b, c in product([5, 10, 15], [5, 6, 7], [256, 512]):
 params.append({'n_estimators': a, 'max_depth': b, 'min_samples_split': c})

print('params = ', params)
print('-----')

cv = StratifiedKFold(5, random_state=123, shuffle=True)

test_scores = list()

X_gscv = ['loading'] + ['measurement_{}'.format(i) for i in range(18)] + ['na_1', 'na_2']

for param in params:
 clf_rf = RandomForestClassifier(random_state=123, **param)
 # cross_val_score은 아래 작업을 할 수 있습니다.
 # cv_scores_mean = np.mean(cross_val_score(clf_rf,
 # df_prob3_train[X_gscv],
 # df_prob3_train['failure'],
 # scoring='roc_auc', cv=cv))
 cv_scores = list()
 # splitter의 split 메소드는 폴드(fold)의
 # train_idx와 test_idx를 넘겨 줍니다.
 for train_idx, valid_idx in cv.split(df_prob3_train[X_gscv], df_prob3_train['failure']):
 # train_idx, test_idx는 위치 인덱스입니다.
 df_cv_train, df_valid = df_prob3_train.iloc[train_idx], df_prob3_train.iloc[valid_idx]
 clf_rf.fit(df_cv_train[X_gscv], df_cv_train['failure'])
 cv_scores.append(
 roc_auc_score(df_valid['failure'], clf_rf.predict_proba(df_valid[X_gscv])[:, 1])
)
 cv_scores_mean = np.mean(cv_scores)
 test_scores.append(cv_scores_mean)
 print(param, test_scores[-1])
 print('-----')

```



```

params = [{'n_estimators': 5, 'max_depth': 5, 'min_samples_split': 256}, {'n_estimators': 5, 'max_depth': 5, 'min_samples_split': 512}, {'n_estimators': 5, 'max_depth': 6, 'min_samples_split': 256}, {'n_estimators': 5, 'max_depth': 6, 'min_samples_split': 512}, {'n_estimators': 5, 'max_depth': 7, 'min_samples_split': 256}, {'n_estimators': 5, 'max_depth': 7, 'min_samples_split': 512}, {'n_estimators': 10, 'max_depth': 5, 'min_samples_split': 256}, {'n_estimators': 10, 'max_depth': 5, 'min_samples_split': 512}, {'n_estimators': 10, 'max_depth': 6, 'min_samples_split': 256}, {'n_estimators': 10, 'max_depth': 6, 'min_samples_split': 512}, {'n_estimators': 10, 'max_depth': 7, 'min_samples_split': 256}, {'n_estimators': 10, 'max_depth': 7, 'min_samples_split': 512}, {'n_estimators': 15, 'max_depth': 5, 'min_samples_split': 256}, {'n_estimators': 15, 'max_depth': 5, 'min_samples_split': 512}, {'n_estimators': 15, 'max_depth': 6, 'min_samples_split': 256}, {'n_estimators': 15, 'max_depth': 6, 'min_samples_split': 512}, {'n_estimators': 15, 'max_depth': 7, 'min_samples_split': 256}, {'n_estimators': 15, 'max_depth': 7, 'min_samples_split': 512}]

```

```

{'n_estimators': 5, 'max_depth': 5, 'min_samples_split': 256} 0.5547115468443294

```

```

{'n_estimators': 5, 'max_depth': 5, 'min_samples_split': 512} 0.5617150416141751

```

```

{'n_estimators': 5, 'max_depth': 6, 'min_samples_split': 256} 0.5568999583940097

```

```

{'n_estimators': 5, 'max_depth': 6, 'min_samples_split': 512} 0.5623239919998733

```

```

{'n_estimators': 5, 'max_depth': 7, 'min_samples_split': 256} 0.5567633668997499

```

```

{'n_estimators': 5, 'max_depth': 7, 'min_samples_split': 512} 0.5642015275666604

```

```

{'n_estimators': 10, 'max_depth': 5, 'min_samples_split': 256} 0.5654552370913933

```

```

{'n_estimators': 10, 'max_depth': 5, 'min_samples_split': 512} 0.5707474621000601

```

```

{'n_estimators': 10, 'max_depth': 6, 'min_samples_split': 256} 0.5681316681217556

```

```

{'n_estimators': 10, 'max_depth': 6, 'min_samples_split': 512} 0.5680847464819798

```

```

{'n_estimators': 10, 'max_depth': 7, 'min_samples_split': 256} 0.5670255107788329

```

```

{'n_estimators': 10, 'max_depth': 7, 'min_samples_split': 512} 0.5724060304780464

```

```

{'n_estimators': 15, 'max_depth': 5, 'min_samples_split': 256} 0.5692073209929452

```

```

{'n_estimators': 15, 'max_depth': 5, 'min_samples_split': 512} 0.5716407241067258

```

```

{'n_estimators': 15, 'max_depth': 6, 'min_samples_split': 256} 0.5706158790202794

```

```

{'n_estimators': 15, 'max_depth': 6, 'min_samples_split': 512} 0.5699305861415821

```

```

{'n_estimators': 15, 'max_depth': 7, 'min_samples_split': 256} 0.5699417404975209

{'n_estimators': 15, 'max_depth': 7, 'min_samples_split': 512} 0.5745226991354744

```

```
In [113... np.argmax(test_scores)
```

```
Out[113]: 17
```

```
In [114... '''
코드는 교차 검증을 통해 얻은 각 하이퍼파라미터 조합에 대한 평균 ROC AUC 점수를 비교하여
가장 높은 점수를 갖는 하이퍼파라미터 조합을 찾습니다. 그리고 그 조합과 해당 점수를 출력합니다.

1. `best_params = params[np.argmax(test_scores)]`:
 - `np.argmax(test_scores)`: `test_scores` 리스트에서 가장 큰 값(최대 ROC AUC 점수)의 인덱스를 반환합니다.
 이는 가장 높은 성능을 보이는 하이퍼파라미터 조합의 인덱스를 나타냅니다.
 - `params[np.argmax(test_scores)]`: `params` 리스트에서 해당 인덱스에 해당하는 하이퍼파라미터 조합을 선택합니다.
 이는 최대 ROC AUC 점수를 갖는 하이퍼파라미터 조합을 나타냅니다.
 - 이 부분은 최적의 하이퍼파라미터 조합을 `best_params` 변수에 저장합니다.

2. `np.max(test_scores)`:
 - `np.max(test_scores)`: `test_scores` 리스트에서 가장 큰 값(최대 ROC AUC 점수)을 반환합니다.
 - 이 부분은 최대 ROC AUC 점수를 반환합니다.

따라서 이 코드는 교차 검증을 통해 얻은 결과 중에서 가장 높은 성능을 보이는 하이퍼파라미터 조합과 해당 점수를 찾아서 출력합니다.
이를 통해 최적의 모델을 결정하는 데 사용할 수 있습니다.
'''

best_params = params[np.argmax(test_scores)]

print('gscv.best_params = ', best_params)
print('-----')
print('np.max(test_scores) = ', np.max(test_scores))

gscv.best_params = {'n_estimators': 15, 'max_depth': 7, 'min_samples_split': 512}

np.max(test_scores) = 0.5745226991354744
```

## 단계 5-2

단계 5-1에서 구한 최적 하이퍼 파라미터로 설정한

랜덤포레스트 분류기(Random-Forest Classifier)를 사용하여 prob3\_train 학습하고,

prob3\_test로 성능을 측정하여 이 값을 A라고 한다.

입력 변수: loading, measurement\_0 ~ 17, na\_1, na\_2 (순서에 유의)

대상 변수: failure

성능 지표: AUC (area under of ROC curve)

## 함수가이드

sklearn.ensemble.RandomForestClassifier, random\_state=123

In [115...

```
'''
코드는 이전 단계에서 찾은 최적의 하이퍼파라미터를 사용하여
랜덤 포레스트(Random Forest) 분류기를 훈련하고,
테스트 데이터셋과 학습 데이터셋에 대해 ROC AUC 점수를 계산합니다.

1. `X_rf = ['loading'] + ['measurement_{}'.format(i) for i in range(18)] + ['na_1', 'na_2']`:
 - `X_rf` 리스트에는 모델의 입력 변수들이 저장됩니다.
 - 여기서는 'loading' 변수와 18개의 'measurement_' 변수, 그리고 'na_1', 'na_2' 변수를 포함합니다.
2. `clf_rf = RandomForestClassifier(random_state=123, best_params)` :
 - 최적의 하이퍼파라미터를 사용하여 랜덤 포레스트 분류기를 초기화합니다.
 - `best_params`는 이전에 찾은 최적의 하이퍼파라미터 조합을 분류기에 전달합니다.
3. `clf_rf.fit(df_prob3_train[X_rf], df_prob3_train['failure'])` :
 - 훈련 데이터셋을 사용하여 랜덤 포레스트 분류기를 학습시킵니다.
 - `df_prob3_train[X_rf]`는 입력 변수들을, `df_prob3_train['failure']`은 목표 변수를 나타냅니다.
4. `roc_auc_score()` :
 - `roc_auc_score()` 함수는 ROC AUC 점수를 계산합니다.
 - 첫 번째 호출은 테스트 데이터셋에 대한 ROC AUC 점수를 계산합니다.
 - 두 번째 호출은 학습 데이터셋에 대한 ROC AUC 점수를 계산합니다.
 - `df_prob3_test['failure']`는 테스트 데이터셋의 목표 변수를,
 `df_prob3_train['failure']`은 학습 데이터셋의 목표 변수를 나타냅니다.
 - `clf_rf.predict_proba()`는 각 클래스에 대한 예측 확률을 반환합니다.
 여기서는 두 번째 열에 대한 확률을 사용하여 ROC AUC 점수를 계산합니다.
'''
```

## 5. 결과:

- 변수 `A`에는 테스트 데이터셋에 대한 ROC AUC 점수가 저장됩니다.
- 이어지는 튜플에서는 테스트 데이터셋과 학습 데이터셋에 대한 ROC AUC 점수가 차례로 표시됩니다.

코드는 최적의 하이퍼파라미터를 사용하여 훈련된 랜덤 포레스트 분류기의 성능을 평가하고, 이 모델이 얼마나 잘 일반화되는지 확인합니다.

```
'''
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
```

```
X_rf = ['loading'] + ['measurement_{}'.format(i) for i in range(18)] + ['na_1', 'na_2']
print('X_rf = ', X_rf)
```

```
clf_rf = RandomForestClassifier(random_state=123, **best_params)
clf_rf.fit(df_prob3_train[X_rf], df_prob3_train['failure'])
```

```
A = roc_auc_score(
 df_prob3_test['failure'],
 clf_rf.predict_proba(df_prob3_test[X_rf])[:, 1]
)
(
 A,
 roc_auc_score(df_prob3_train['failure'], clf_rf.predict_proba(df_prob3_train[X_rf])[:, 1])
)
```

```
X_rf = ['loading', 'measurement_0', 'measurement_1', 'measurement_2', 'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6', 'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10', 'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14', 'measurement_15', 'measurement_16', 'measurement_17', 'na_1', 'na_2']
```

Out[115]: (0.5687712018291998, 0.6416568062149542)

In [116...

```
'''
```

코드는 테스트 데이터셋을 사용하여 그리드 탐색을 통해 얻은 최적의 모델의 ROC AUC 점수를 계산합니다.

## 1. `roc\_auc\_score()`:

- `roc\_auc\_score()` 함수는 ROC AUC 점수를 계산합니다.
- 첫 번째 매개변수는 실제 클래스 레이블을 나타내는 테스트 데이터셋의 목표 변수입니다.  
여기서는 `df\_prob3\_test['failure']`을 사용합니다.
- 두 번째 매개변수는 예측된 클래스 확률을 나타내는 배열입니다.  
여기서는 `gscv.best\_estimator\_.predict\_proba(df\_prob3\_test[X\_rf])[:, 1]`을 사용합니다.
- `gscv.best\_estimator\_`는 그리드 탐색을 통해 얻은 최적의 모델을 나타냅니다.
- `predict\_proba()` 메서드는 각 클래스에 대한 예측 확률을 반환합니다.

이중 두 번째 열은 **positive** 클래스(이진 분류에서의 '1')에 대한 확률을 나타냅니다.  
 - `[:, 1]`은 예측 확률 배열에서 두 번째 열의 값을 선택합니다.

따라서 코드는 최적의 모델을 사용하여 테스트 데이터셋에 대한 **ROC AUC** 점수를 계산합니다.  
 이는 최적의 모델이 테스트 데이터에서 얼마나 잘 수행되는지를 나타내는 중요한 성능 지표입니다.  
 ...

```
roc_auc_score(
 df_prob3_test['failure'], gscv.best_estimator_.predict_proba(df_prob3_test[X_rf])[:, 1]
)
```

Out[116]: 0.5687712018291998

In [117... A, round(A, 3)

Out[117]: (0.5687712018291998, 0.569)

A값을 소수점 넷째 자리에서 반올림하여 3째 자리까지 출력하시오.

**0.569**

In [ ]: